Fundamentals of Wireless Communications

无线通信基础

深圳大学
SHENZHEN UNIVERSITY

课程内容

第1单元 / 无线通信概论

第2单元 / 无线电波传播

第3单元 / 窄带无线信道

第4单元 / 宽带和方向性信道

第5单元 / 数字调制解调

第6单元 / 信道编码

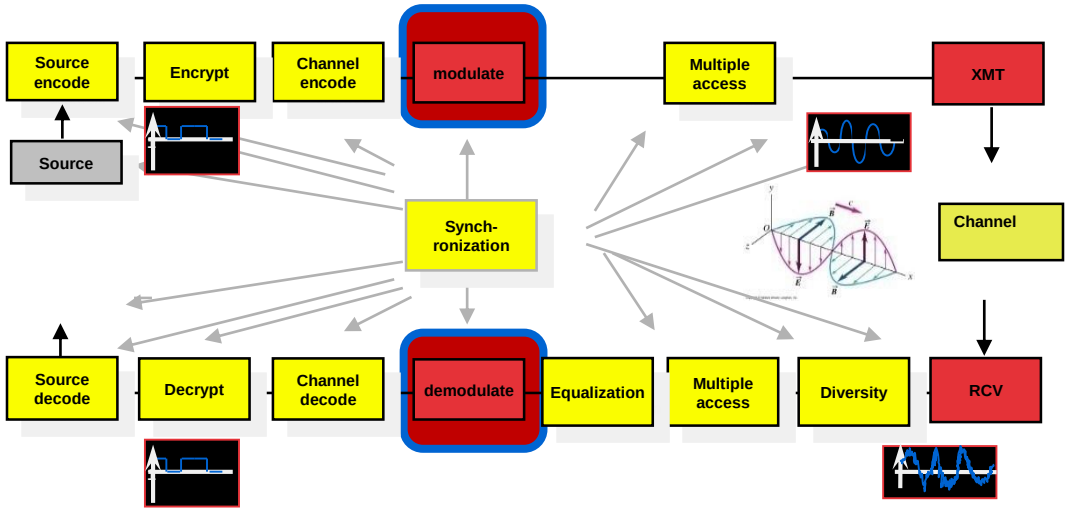
第7单元 / 均衡

第8单元 / 分集

第9单元 / 扩展频谱通信

第10单元/ 正交频分复用

第11单元/ 多天线技术



信道编码

第六单元

信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

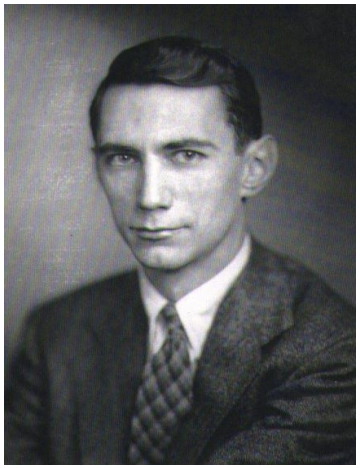
信道编码的产生的背景

- 无线信道在传输过程中，由于噪声、干扰、衰落等原因，会导致接收信号中存在错误（误码），从而降低了通信的可靠性。
- 不同的业务对传输可靠性有不同的要求（BER）
  - 语音业务：10^{-2}~10^{-3}
  - 数据业务：10^{-6}~10^{-9}
  - 敏感数据： 10^{-9} ~ 10^{-10}

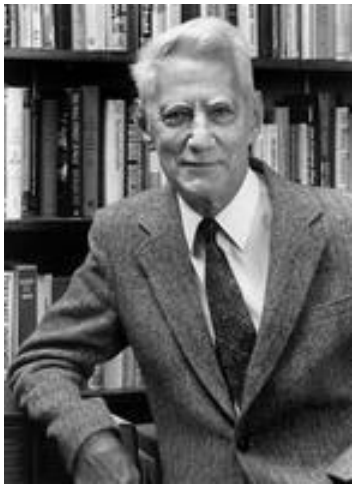
- 人们很早就发现采用信道编码（如重复码）可以提高传输可靠性，但同时也会降低传输的有效性。
- 在1940年代以前，普遍认为可靠性与有效性是一对矛盾，要实现任意小的误码率（不断地重复），信息速率只有趋近于0。

信道编码的产生与发展 有噪信道编码定理

- 1948年香农建立了信息论，提出只要信息速率R<信道容量C，通过信道编码可达到任意小的误码率，而信息速率无需趋近于0。
- 香农信息论的提出标志着通信由工程技术成为了科学，香农也被誉为信息论及数字通信时代的奠基人 。



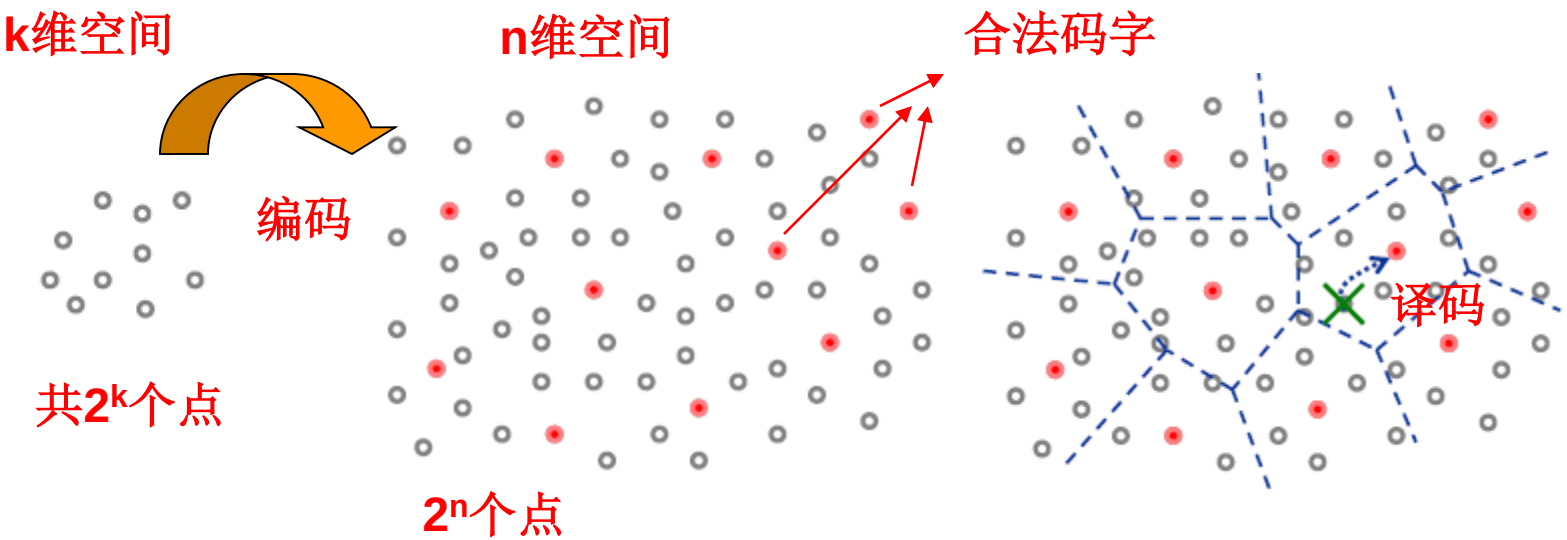
克劳德·香农
(Claude Elwood Shannon，1916-2001)



- 信道编码的基本思路是根据一定的规律在待发送的信息码元中人为地加入一定的多余码元(称为**监督码**)，以引入一定的冗余为代价来提高传输可靠性。

与信源编码思路刚好相反

信道编码的基本原理



一个k bit长的信息序列，对应k维离散空间中的一个点

n维空间中有2^n个点（对应长度为n的序列），其中只有2^k个为合法码字

信道编码的产生与发展

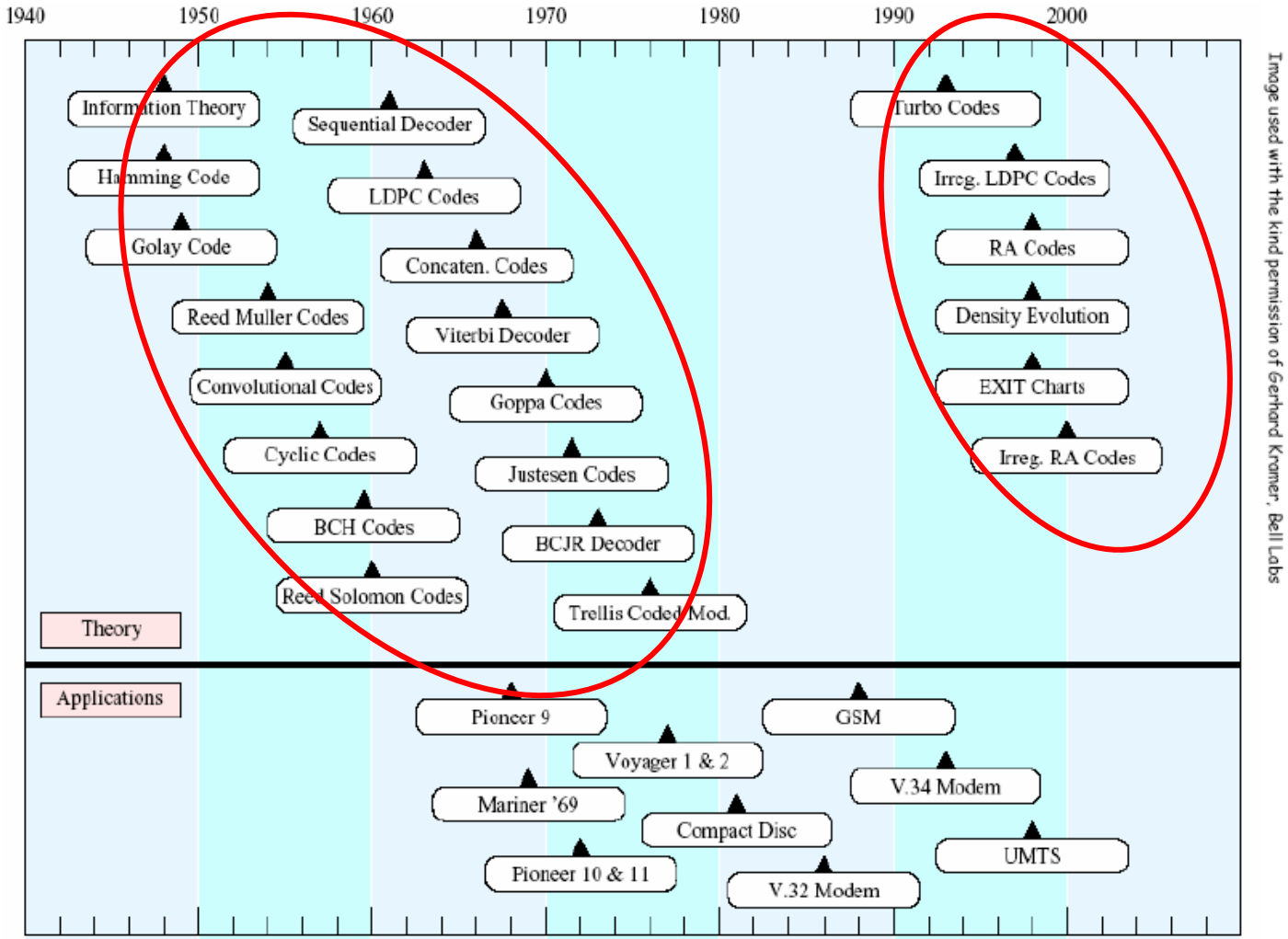


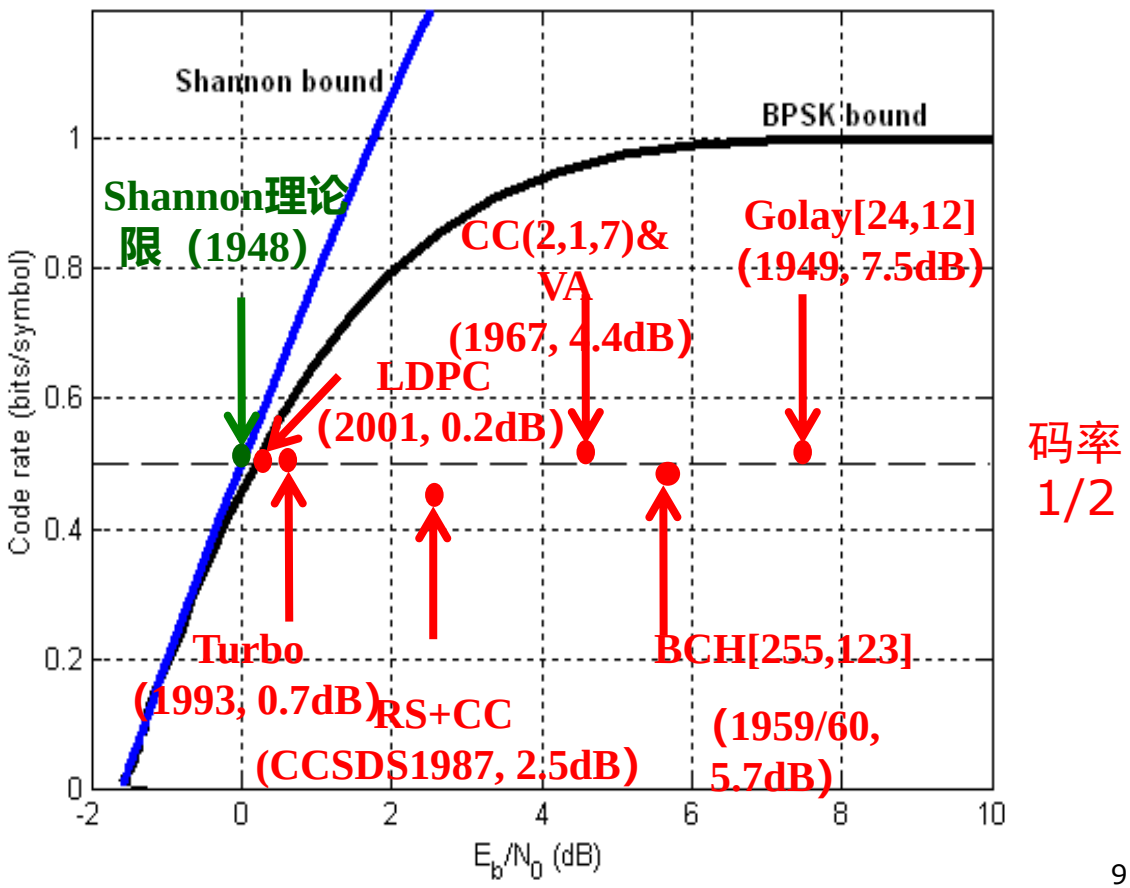
Image used with the kind permission of Gerhard Kramer, Bell Labs

两个活跃阶段

编码性能与Shannon限

未编码	9.6dB
扩展格雷码	7.5dB
BCH编码	5.7dB
卷积编码	4.4dB
RS+卷积码	2.5dB
Turbo码	0.7dB
LDPC码	0.2dB

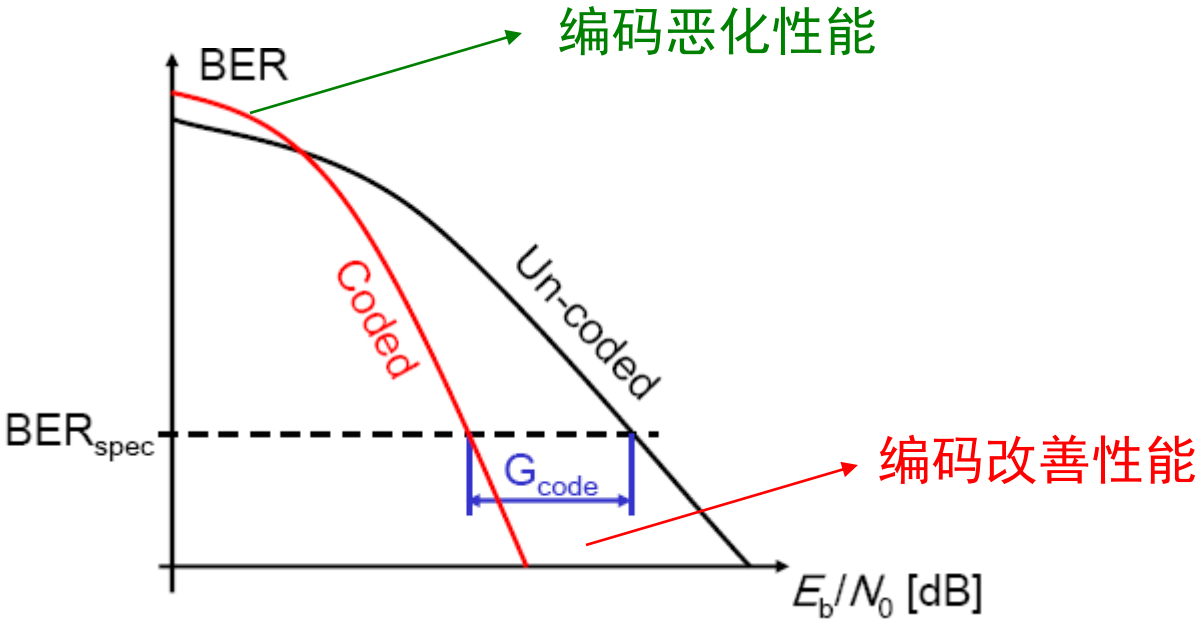
码率1/2时，BPSK传输下与香农极限（0dB）差，BER为10^{-5}可认为无差错传输



逼近Shannon限的信道编码

编码增益(G_{code})

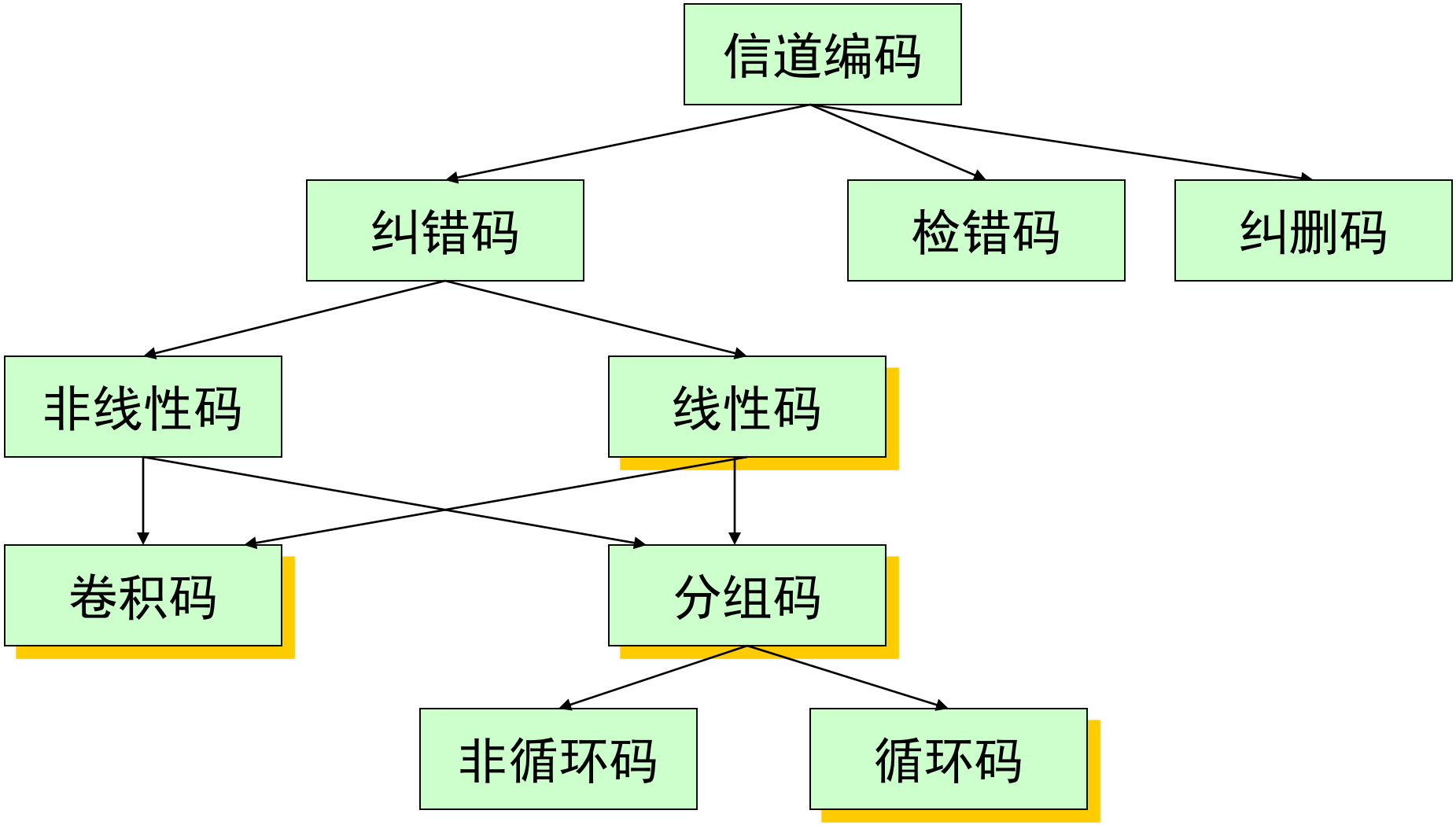
在一定BER要求下，采用信道编码相比于不采用信道编码，所需信噪比的减少量



在信噪比E_b/N_0较低时，编码为什么会导致性能的恶化?

信道编码不是万能的，信道编码是锦上添花，而非雪中送炭

信道编码分类



信道编码分类（按照差错控制功能）

检错码仅具备识别错误功能，而无纠正错误功能（不知道哪错了）

1 0 0 0 1 1 0 → 1 0 1 0 1 1 0 ✗

纠错码不仅具备识别错误功能，同时具备纠正错误功能

1 0 0 0 1 1 0 → 1 0 1 0 1 1 0 → 1 0 0 0 1 1 0

纠删码具备将丢失数据恢复功能

1 0 0 0 1 1 0 → 1 0 ? 0 1 1 0 → 1 0 0 0 1 1 0

检错码

- 检错码虽然无法直接纠正错误，但可以通过反向链路，通知发射机重新发送该码字，再次进行接收，直到正确接收为止，即自动反馈重发（Automatic Repeat Queuing，ARQ），从而能够显著提高传输可靠性
- 循环冗余校验码（CRC）是当前应用最为广泛的检错码
- 主要优点包括：
  - 检错能力强，冗余小
  - 编译码复杂度低，实现电路简单，处理时延小

循环冗余校验码（CRC）

- 假定信息序列长度为k 位，CRC校验码长度为m 位，编码过程：
  - 根据信息序列 **u**，得到对应的信息多项式为$u(x)$
  - 将信息多项式左移m位，得到 $x^m u(x)$
  - 用 $x^m u(x)$ 除以生成多项式$g(x)$，得到余数多项式 $p(x)$，其对应的就是CRC校验码
  - 一般将CRC校验码附在信息序列后，完成了CRC编码
- 实际上，就是通过编码，将信息序列对应的k–1阶信息多项式$u(x)$，转换为可以被 $g(x)$ 整除的 n–1 阶多项式 $x^m u(x) - p(x)$

CRC应用

CRC校验码	生成多项式$g(x)$	典型应用
CRC-6	x^6+x^5+1	5G
CRC-8	$x^8 + x^7 + x^4 + x^3 + x + 1$ x^8+x^2+x+1	LTE
CRC-11	$x^{11}+x^{10}+x^9+ x^5+1$	5G
CRC-12	$x^{12}+x^{11}+x^3+x^2+x+1$	
CRC-16	$x^{16} + x^{12} + x^5 + 1$	LTE，5G，USB
CRC-24A	$x^{24}+x^{23}+x^{18}+x^{17}+x^{14} + x^{11} + x^{10} + x^7 + x^6 + x^5 + x^4 + x^3 + x +1$	LTE，5G
CRC-24B	$x^{24} + x^{23} + x^6 + x^5 + x + 1$	LTE，5G
CRC-24C	$x^{24}+x^{23}+x^{21}+x^{20}+x^{17} + x^{15} + x^{13} + x^{12} + x^8 + x^4 + x^2 + x +1$	5G
CRC-32	$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$	ZIP, RAR, IEEE 802 LAN/FDDI, IEEE 1394
CRC-32C	$x^{32}+x^{28}+x^{27}+x^{26}+x^{25}+x^{23}+x^{22}+x^{20}+x^{19}+x^{18}+x^{14}+x^{13}+x^{11}+x^{10}+x^9$ $+x^8+x^6+1$	iSCSI, SCTP
CRC-64	$x^{64}+x^4+x^3+ x +1$	ISO 3309

校验码越长，检测错误的能力越强，同时计算复杂度也越高

纠错码

- 纠错码不仅能够检测错误，而且具备纠正错误的能力。
- 因此纠错码也称前向纠错码（Forward Error Correction，FEC）。
- 纠错码可以不通过反馈重发就能够提高传输可靠性，具有较低的时延，特别适应于时延敏感的应用。
- 但是纠错码都有一定的纠错能力，当错误数超过了纠错能力后，就无法纠正所有错误。

1 0 0 0 1 1 0 → 1 0 1 0 1 1 0 → 1 0 0 0 1 1 0

纠错码

- 因此，为了进一步提高传输可靠性，保证纠错码译码结果极高的正确率，在实际应用中通常将纠错码和检错码联合使用。
- 首先，进行纠错码译码，对错误进行纠正。
- 然后，采用检错码对译码结果进行校验。当发现纠错译码结果仍然存在错误时，再反馈给发射机，要求进行重发。
- 该方案将FEC与ARQ相结合，因此也称为混合自动重传请求（Hybrid Automatic Repeat reQuest，HARQ）。

检错码

无线通信系统	纠错码
GSM	奇偶校验码、循环码、卷积码
WCDMA/CDMA2000/ TD-SCDMA	卷积码、Turbo码
LTE	卷积码、分组码、重复码、 Turbo码
WiMax	卷积码、Turbo码、LDPC码
5G	LDPC码（用于业务信道） Polar码（用于控制信道）

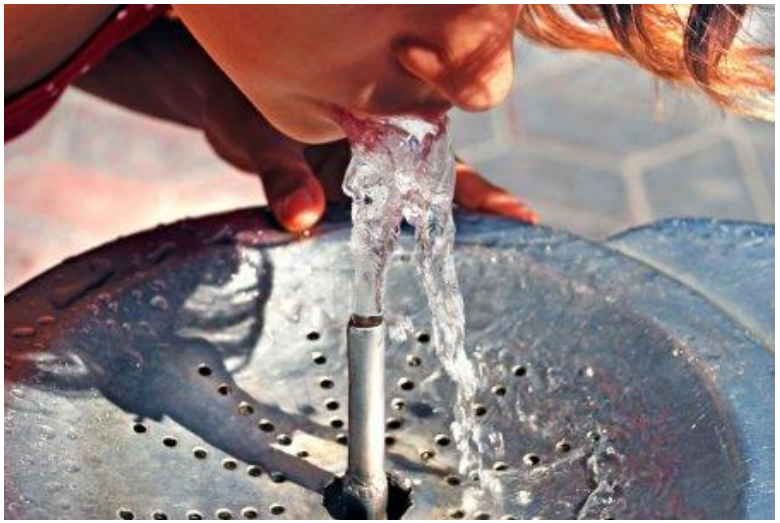
纠删码

- 在删除信道（Erasure Channel）中传输时，比特或者分组可能会发生丢失（删除）。
- 纠错码（例如RS码）也可以作为纠删码，但存在以下不足：
  - 纠错码，特别是纠正突发错误能力较强的纠错码，译码复杂度较高
  - 需要加入较多冗余比特才能实现较好的纠删能力，开销较大
  - 对信道适应性不好，难以根据信道状态进行灵活调整，特别是在广播与多播应用中，不同链路信道状态有显著差异，这一问题更加突出

1 0 0 0 1 1 0 → 1 0 ? 0 1 1 0 → 1 0 0 0 1 1 0

喷泉码（Fountain Code）

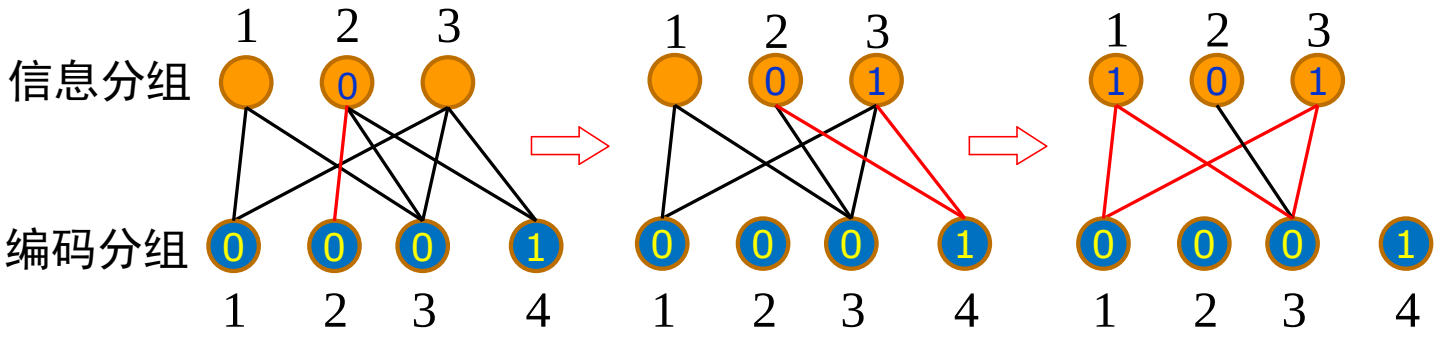
- 喷泉码根据k个信息符号可产生无限长的编码序列（无码率码）。
- 接收机只要能够正确接收其中任意$k(1+\varepsilon)$个编码符号，就能够以很高的概率（与ε有关）恢复k个信息符号，而且ε仅略大于0，即开销很小



- 接收机正确恢复所有编码符号后，给发射机发送一个确认信息（ACK）。当所有接收机都正确接收后，发射机停止编码和发送
- 2002年，M.Luby提出了第一种可行的喷泉码—LT码。
- Amin Shokrollahi对LT码进行了改进，提出了一种具有更低编译码复杂度的喷泉码—Raptor码。

喷泉码（Fountain Code）

- LT码译码就是根据接收到的编码分组，恢复（推断）未知信息分组过程。



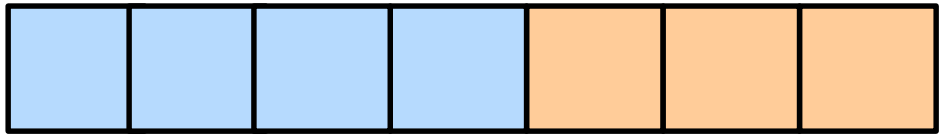
- LT码编码和译码复杂度均为 $k\ln k$。
- 当前喷泉码已经被DVB-H标准和3GPP的多媒体广播多播业务（MBMS）标准采用，而且不仅应用于无线通信系统，还广泛应用于数据存储等应用中。

信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

线性分组码

- 分组码
  - 信息序列以每 k 个码元为一组，按一定规律产生 r 个监督码元，输出序列每组长为$n=k+r$，每一码字的 r 个监督码元只与本码字的 k 个信息码元有关，通常将分组码记为(n, k)。



(7, 4)分组码

- 线性码
  - 信息码元与监督码元之间呈线性关系，它们的关系可用一组线性方程联系起来。
- 线性分组码
  - 信息码元与监督码元之间呈线性关系的分组码

线性分组码

- 码长：码字的长度n　　码率：$R=k/n$
- 码重：码字中非零元素的个数
- 汉明距离：码字间相同位置元素不同的个数
- 最小汉明距离：所有汉明距离中的最小者
- 错误图案：差错的形式也可以用二元序列来描述
  - 若发送码字为：$\boldsymbol{c}=[c_1,c_2,\ldots,c_n]$，接收码字为 $\boldsymbol{r}=[r_1,r_2,\ldots,r_n]$

    两者的差别：$\boldsymbol{e}=\boldsymbol{c}\oplus\boldsymbol{r}=[e_1,e_2,\ldots,e_n]$　　就是错误图案
  - 如：

$\boldsymbol{c}$:	1	0	0	0	1	1	0
$\boldsymbol{r}$:	1	1	0	0	1	1	0
$\boldsymbol{e}$:	0	1	0	0	0	0	0

发生错误

对应码字中发生了错误的位置，发生错误的位表示为1，未发生错误的位表示为0

线性分组码编码

- 输入 k 位信息序列 $\boldsymbol{u}=[u_1,u_2,\ldots,u_k]$
- 输出 n 位编码码字 $\boldsymbol{c}=[c_1,c_2,\ldots,c_n]$
- 生成矩阵

$$\mathbf{G}=\begin{bmatrix}\mathbf{g_1}\\ \mathbf{g_2}\\ \mathrm{M}\\ \mathbf{g_k}\end{bmatrix}=\begin{bmatrix}g_{11} & g_{12} & \mathrm{L} & g_{1n}\\ g_{21} & g_{22} & \mathrm{L} & g_{2n}\\ \mathrm{M} & \mathrm{M} & \mathrm{O} & \mathrm{M}\\ g_{k1} & g_{k2} & \mathrm{L} & g_{kn}\end{bmatrix}$$

$\mathbf{g}_1,\mathbf{g}_2,\mathrm{K},\mathbf{g}_k$是线性无关的

- 编码码字

$$\boldsymbol{c}=\mathbf{uG}$$

$$\boldsymbol{c}=u_1\mathbf{g}_1+u_2\mathbf{g}_2+,\mathrm{K},+u_k\mathbf{g}_k$$

码字是线性无关矢量的线性组合

例

- 信息序列 $\boldsymbol{u}=\begin{bmatrix}1 & 0 & 1 & 1\end{bmatrix}$

- 生成矩阵 $\mathbf{G}=\begin{bmatrix}1 & 0 & 0 & 0 & 1 & 1 & 0\\ 0 & 1 & 0 & 0 & 1 & 0 & 1\\ 0 & 0 & 1 & 0 & 0 & 1 & 1\\ 0 & 0 & 0 & 1 & 1 & 1 & 1\end{bmatrix}_{4\times 7}$

系统码的生成矩阵中包括单位阵

- 编码码字

$$\boldsymbol{c}=\boldsymbol{u}\mathbf{G}=\begin{bmatrix}1 & 0 & 1 & 1\end{bmatrix}\begin{bmatrix}1 & 0 & 0 & 0 & 1 & 1 & 0\\ 0 & 1 & 0 & 0 & 1 & 0 & 1\\ 0 & 0 & 1 & 0 & 0 & 1 & 1\\ 0 & 0 & 0 & 1 & 1 & 1 & 1\end{bmatrix}=\begin{bmatrix}1 & 0 & 1 & 1 & 0 & 1 & 0\end{bmatrix}$$

信息序列　　监督码元

- 编码码字中包含信息序列的信道编码称为系统码；反之，则称为非系统码

线性分组码译码

- 接收码字 $\boldsymbol{r}$
- 校验矩阵 $\mathbf{H}$ 有 $\mathbf{G}\mathbf{H}^T = \mathbf{0}$
- 伴随式 $\mathbf{s} = \mathbf{r}\mathbf{H}^{\mathrm{T}} = (\mathbf{c}+\mathbf{e})\mathbf{H}^{\mathrm{T}} = \mathbf{0} + \mathbf{e}\mathbf{H}^{\mathrm{T}}$
- 校验 $\mathbf{s} = 0$ 校验未发现错误（实际上并不一定无错误）

  $\mathbf{s} \neq 0$ 校验发现错误
- 纠错
  - $\boldsymbol{s}$ 与错误图案 $\boldsymbol{e}$ 和校验矩阵 $\mathbf{H}$ 有关
  - 码字中一位发生错误时，$\boldsymbol{s}^{\mathrm{T}}$对应 $\mathbf{H}$ 中的某一列
  - 码字中多位发生错误时， $\boldsymbol{s}^{\mathrm{T}}$对应 $\mathbf{H}$ 中的某些列相加
  - 一旦确定了错误图案，$\hat{\boldsymbol{c}} = \boldsymbol{r} + \boldsymbol{e}$

伴随式译码

译码的关键是根据伴随式确定错误图案

例

接收码字 $\boldsymbol{r} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \end{bmatrix}$

校验矩阵 $\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}_{3\times 7}$

确定接收码字是否有错。

伴随式 $\boldsymbol{s} = \boldsymbol{r}\mathbf{H}^{\mathrm{T}} = \begin{bmatrix} 0 & 1 & 1 \end{bmatrix}_{1\times 3}$

$\boldsymbol{s} \neq \mathbf{0}$ 表示接收码字有误

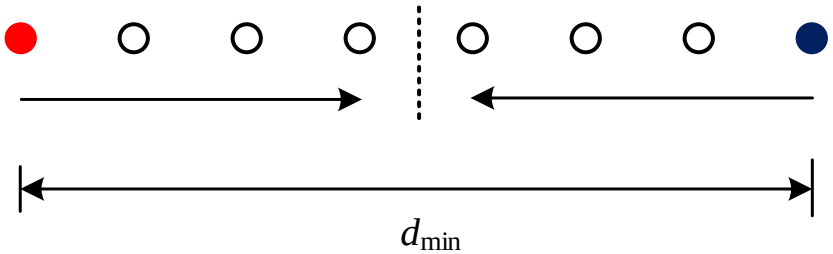
纠错 $\boldsymbol{s}$ 对应的错误图案 $\boldsymbol{e} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$

$$\hat{\boldsymbol{c}} = \boldsymbol{r} + \boldsymbol{e} = [1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1] + [0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0]$$
$$= [1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1]$$

线性分组码纠错能力

- 纠错能力

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

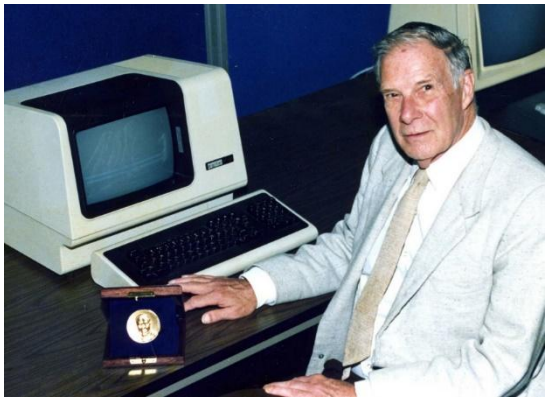


- 线性分组码总是满足

$$d_{\min} \le n - k + 1$$

- 汉明码是能够纠所有单个错误的完备码，且码率最高的线性分组码

$$(n, k) = (2^m - 1,\ 2^m - 1 - m)$$



理查德·汉明

例

- BPSK调制，高斯噪声，S/N_0=43766，R=4800bit/s，采用(15, 11)汉明码。请分别计算采用和不采用信道编码的误比特率。
- 解：

1）未采用信道编码

$$\frac{E_b}{N_0}=\frac{S}{RN_0}=9.12=9.6\text{dB}$$

误比特率 $p_u=Q\left(\sqrt{2E_b/N_0}\right)=Q\left(\sqrt{18.24}\right)=1.02\times10^{-5}$

例

2）采用信道编码后

$$R_c = 4800 \times \frac{15}{11} = 6545\text{bps}$$

$$\frac{E_c}{N_0} = \frac{S}{R_c N_0} = 6.688 = 8.25\text{dB}$$

由于加入了监督比特，在总信号能量不变的前提下，相比于不采用信道编码，信噪比降低了

译码前误比特率 $p = Q\left(\sqrt{2E_c / N_0}\right) = Q\left(\sqrt{13.38}\right) = 1.36 \times 10^{-4}$

译码后码字错误概率 $p_c = \sum_{j=2}^{15} \binom{15}{j} p^j (1-p)^{15-j} \approx 1.94 \times 10^{-6}$

根据码字错误概率与误比特率的近似关系式，误比特率

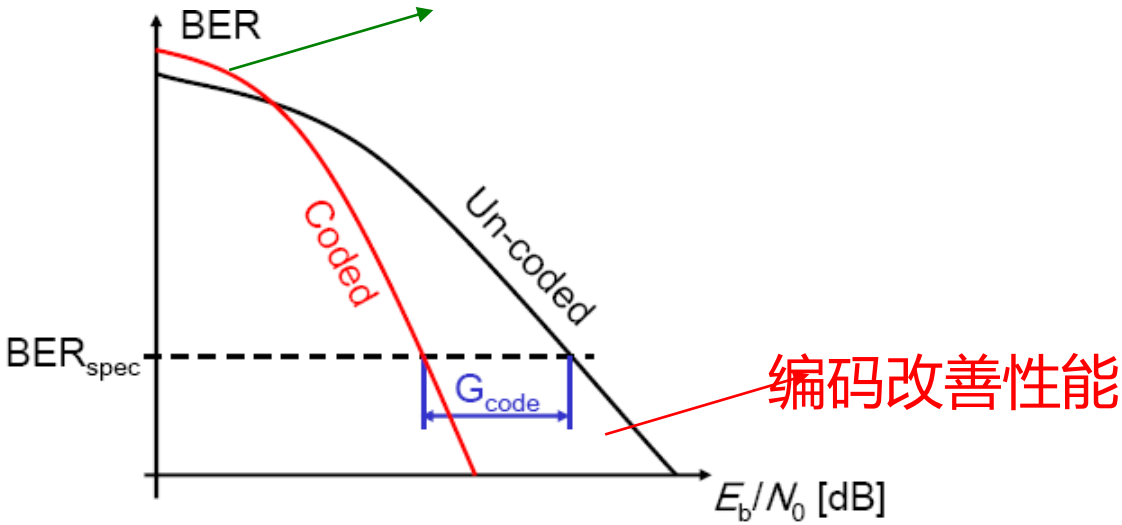
$$p_c \approx p - p(1-p)^{14} = 1.39 \times 10^{-6}$$

采用(15, 11)编码，在信号能量不变的情况下，误比特率下降1个数量级

编码增益

定义：要实现相同的误比特率，编码与未编码所需信噪比之差

编码恶化性能

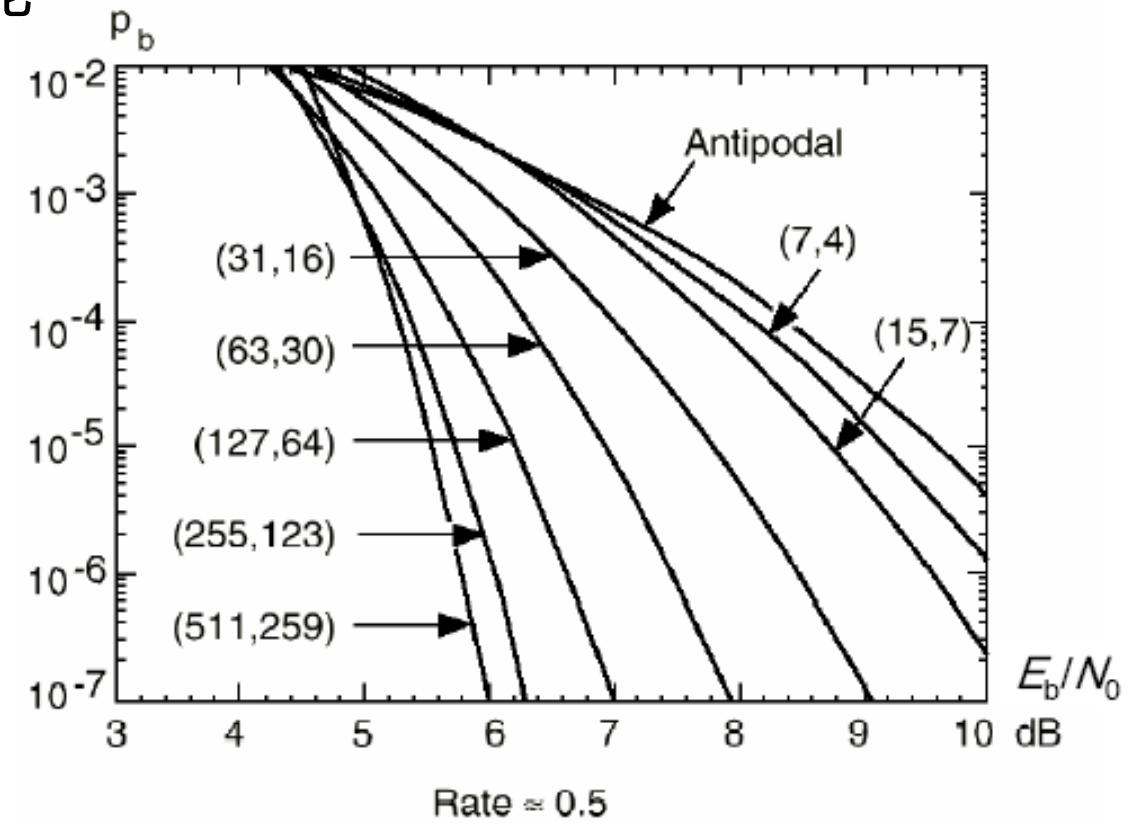


编码改善性能

信道编码的性能实际上是错误纠正和能量摊薄这正反两方面共同作用的结果。

线性分组码的性能

- 码长与性能



码率相同的条件下，码长越长，性能越好，但编译码复杂度也越高

循环码

- 循环码是一种特殊的线性分组码，任意一个码字（全零码除外），进行循环左移（或循环右移）所形成的码字仍然是该循环码的一个码字
- 如某（7，3）循环码的全部码字

第2行对应码字循环左移1位，得到的就是第3行对应的码字

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

- 由于具有循环的结构，编译码可用反馈移位寄存器来实现，硬件复杂度较低。

循环码

- 一般采用代数方法来构造和分析循环码。
- 码长为n的循环码的一个码字，可表示为多项式的形式：

$$C(x) = c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + \ldots + c_1 x + c_0$$

- 若$C(x)$是码长为n的循环码的一个码多项式，则 $x^i C(x)$按模 x^n+1 运算的余式，必为循环码中的另一码多项式，循环码同余 x^n+1
- 循环码生成多项式

$$G(x) = x^{n-k} + g_{n-k-1}x^{n-k-1} + \ldots + g_2 x^2 + g_1 x + 1$$

- 多项式最高阶次 $n-k$，所有码多项式都能被 $G(x)$ 整除，是x^n+1的一个因式，同时也是一个码字多项式

循环码编码

- 将信息序列多项式$U(x)$与x^{n-k}相乘，实际上就是左移 $n-k$ 位，然后再除以$G(x)$

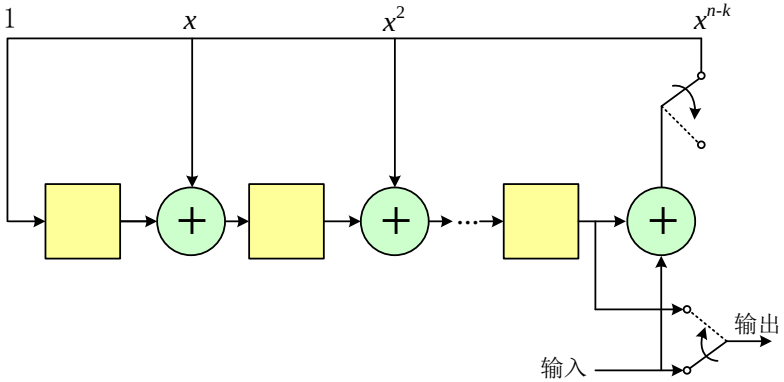
$$\frac{x^{n-k}U(x)}{G(x)} = Q(x) + \frac{P(x)}{G(x)}$$

得到余数多项式 $P(x) = p_{n-k-1}x^{n-k-1} + \ldots + p_2x^2 + p_1x + p_0$

则码字多项式 $C(x) = x^{n-k}U(x) + P(x)$

生成多项式 $G(x) = x^{n-k} + x^2 + x + 1$

对应抽头位置，即系数为1的对应位置有抽头

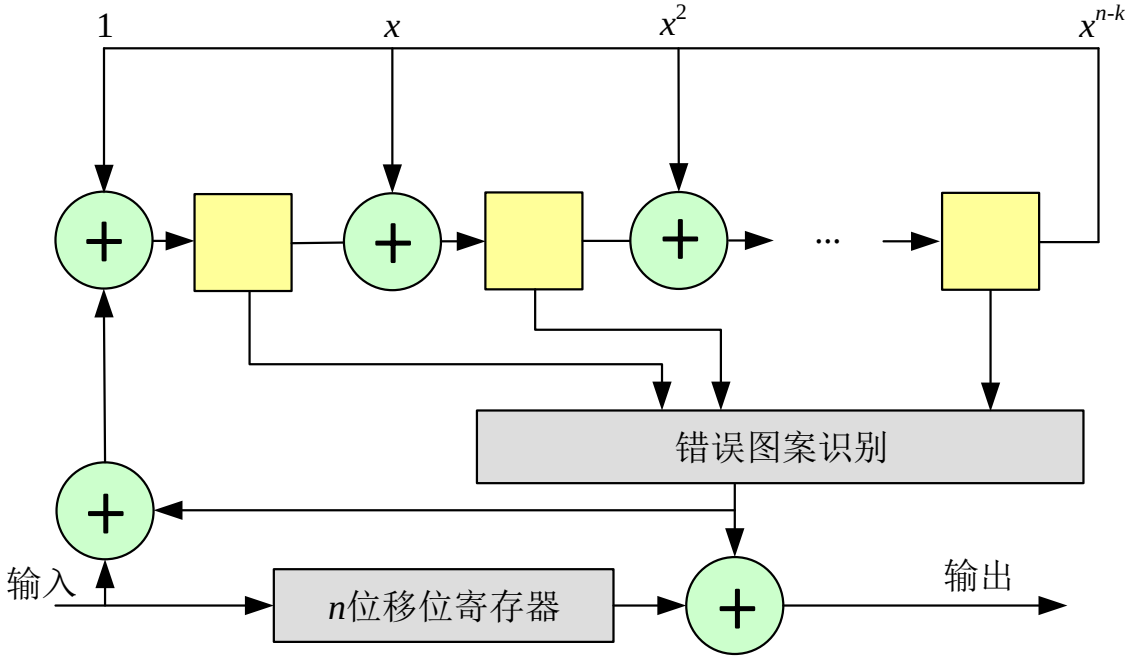


循环码译码

- 通常采用伴随式译码，即根据伴随式的值，确定错误图案，完成译码

$$S(x)=\left|R(x)\right|_{\mathrm{mod}\,G(x)}=\left|C(x)+e(x)\right|_{\mathrm{mod}\,G(x)}=0+e(x)\Big|_{\mathrm{mod}\,G(x)}$$

接收码字多项式　发送码字多项式　错误图案多项式



循环码编译码实例

某(7, 4)循环码，对 x^7+1 进行因式分解，得

$$x^7+1=(x+1)(x^3+x^2+1)(x^3+x+1)$$

选择 x^3+x+1 作为该循环码的生成多项式$G(x)$。

（1）假定信息序列为1101，完成循环码编码。

（2）假定编码序列最高位（左侧）在传输过程中发生了错误，完成译码。

解：信息序列为1101，对应的信息多项式为$U(x)=x^3+x^2+1$

$$\frac{x^{n-k}U(x)}{G(x)}=\frac{x^3(x^3+x^2+1)}{x^3+x+1}=\frac{x^6+x^5+x^3}{x^3+x+1}=x^3+x^2+x+1+\frac{1}{x^3+x+1}$$

得到余数多项式$P(x)=1$，则码字多项式 $C(x)=x^3U(x)+P(x)=x^6+x^5+x^3+1$

对应的编码序列为C=[1101001]。

循环码编译码实例

- 编码序列最高位（左侧）在传输过程中发生了错误，则接收到的码字序列为R=[0101001]，对应的接收码字多项式$R(x)=x^5+x^3+1$
- 计算伴随式

$$S(x)=\left|R(x)\right|_{\bmod G(x)}=x^2+1$$

- 由伴随式查询得到$e(x)=x^6$，即错误图案 $\boldsymbol{e}$=[1000000]，表明最高位发生错误
- 译码结果为：

$$\hat{\mathbf{c}}=\mathbf{r}+\mathbf{e}=[0\quad 1\quad 0\quad 1\quad 0\quad 0\quad 1]+[1\quad 0\quad 0\quad 0\quad 0\quad 0\quad 0]=[1\quad 1\quad 0\quad 1\quad 0\quad 0\quad 1]$$

信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

卷积码

- 卷积码不将信息序列分组，而是以连续的方式添加冗余，具有流的形式（类似流水线）

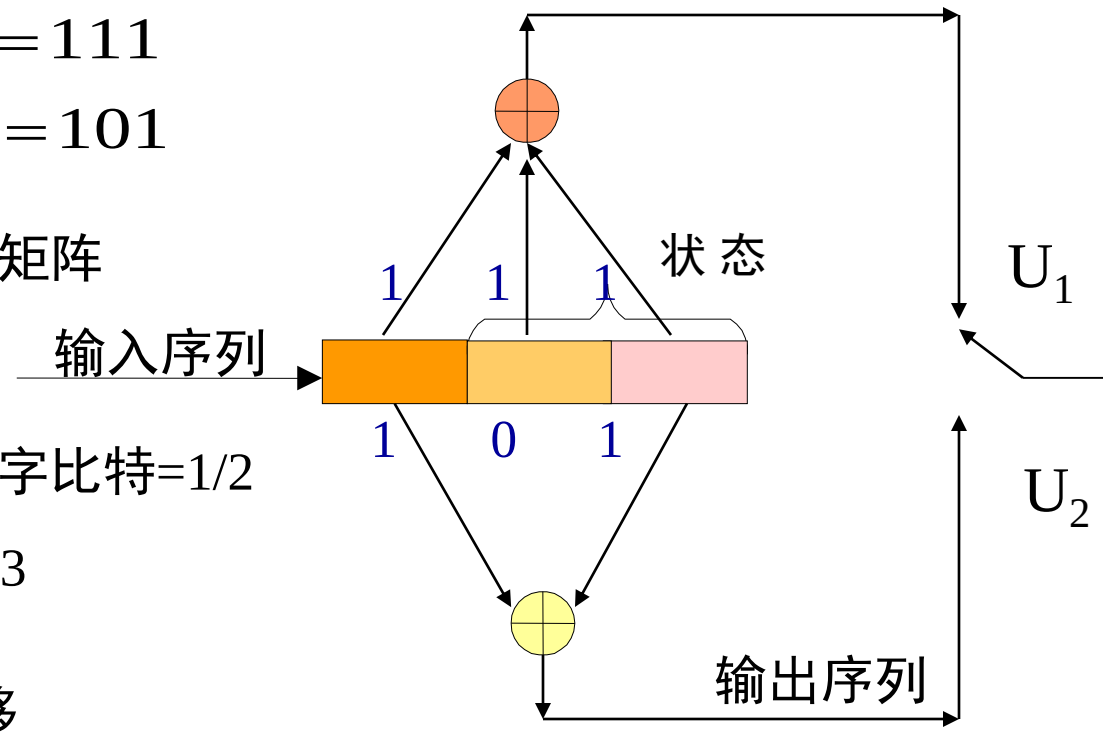
卷积码

- 卷积码连接关系决定了编码方式

生成式（连接关系）

$$g_1 = 111$$
$$g_2 = 101$$

相当于线性分组码的生成矩阵



码率R:输入信息比特/输出码字比特=1/2

约束长度K：编码相关长度=3

为确保最后一个信息比特移出寄存器，通常要在输入序列最后补K-1个零（尾比特），寄存器回到全0状态

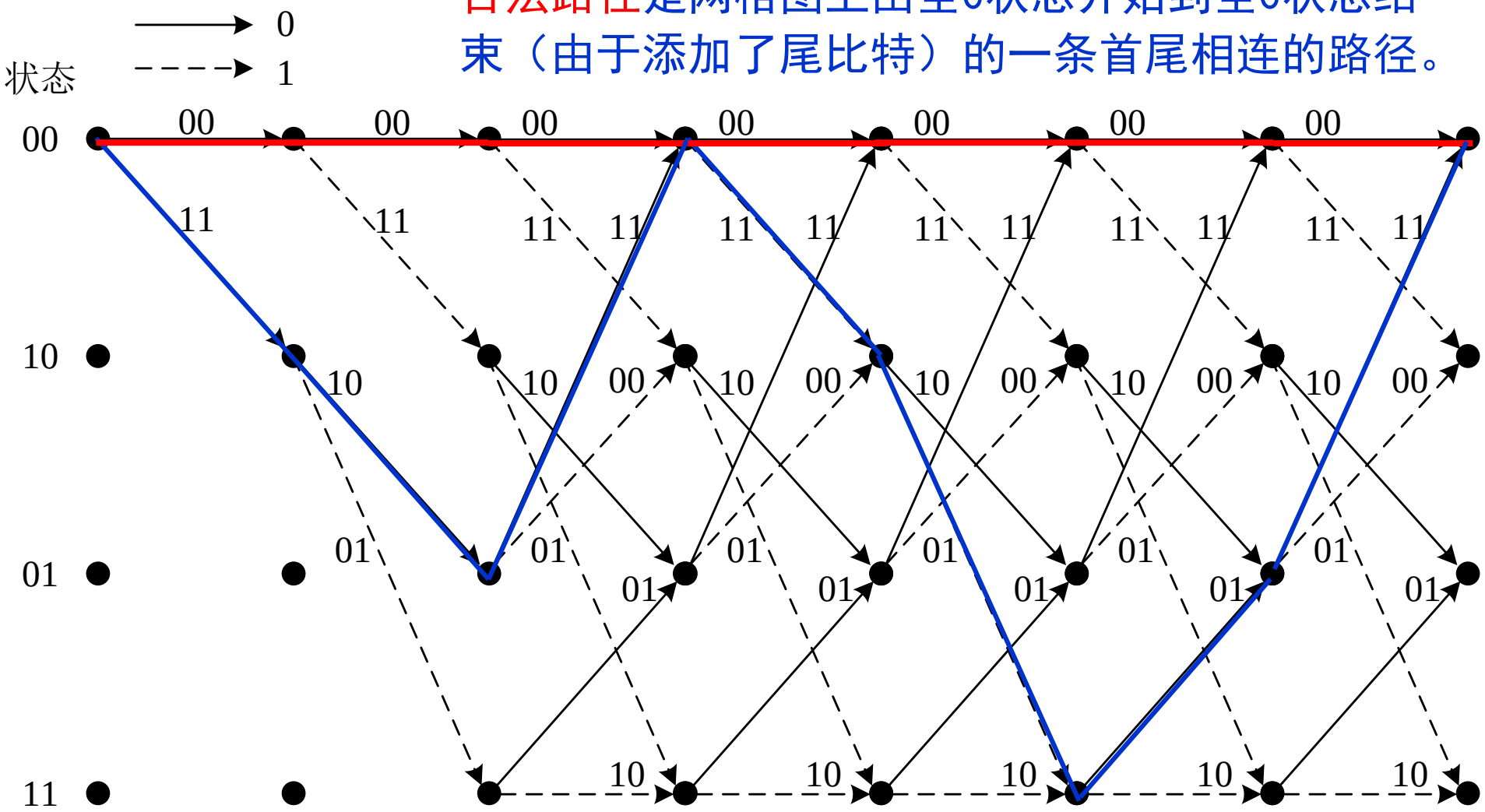
卷积码表示方法：

(输出，输入，约束长度)

例如(2, 1, 3)

(2，1，3)卷积码网格图

合法路径是网格图上由全0状态开始到全0状态结束（由于添加了尾比特）的一条首尾相连的路径。



状态数$2^{K-1}=4$

红色路径对应输入为0000000，蓝色路径对应输入为1001100

卷积码生成式

■卷积码生成式直接关系到卷积码的性能，针对不同码率R和约束长度K，研究人员给出了最佳的生成式

R=1/2

约束长度K	生成式(八进制)
3	5, 7
4	15, 17
5	23, 35
6	53, 75
7	133, 171
8	247, 371
9	561, 753
11	2335, 3661

R=1/3

约束长度K	生成式(八进制)
3	5, 7, 7
4	13, 15, 17
5	25, 33, 37
6	47, 53, 75
7	133, 145, 175
8	225, 331, 367
9	557, 663, 711
11	2353, 2671, 3175

译码算法

- 门限译码
- 序列译码
- Viterbi

最大似然序列估计(MLSE)

- 译码实际上是寻找与接收路径具有最大似然的一条合法路径
- 已知接收序列**r**，寻找最有可能的发送序列 $\hat{\mathbf{s}}$
- 译码所需的定量指标称为度量（matric）

$$\hat{\mathbf{s}} = \max_{\mathbf{s}} \mathrm{P}(\mathbf{s}|\mathbf{r})$$ 最大后验概率

$$\hat{\mathbf{s}} = \max_{\mathbf{s}} \mathrm{P}(\mathbf{r}|\mathbf{s})$$ 最大似然概率

$$\hat{\mathbf{s}} = \max_{\mathbf{s}} \prod_i \mathrm{P}(r_i|s_i)$$

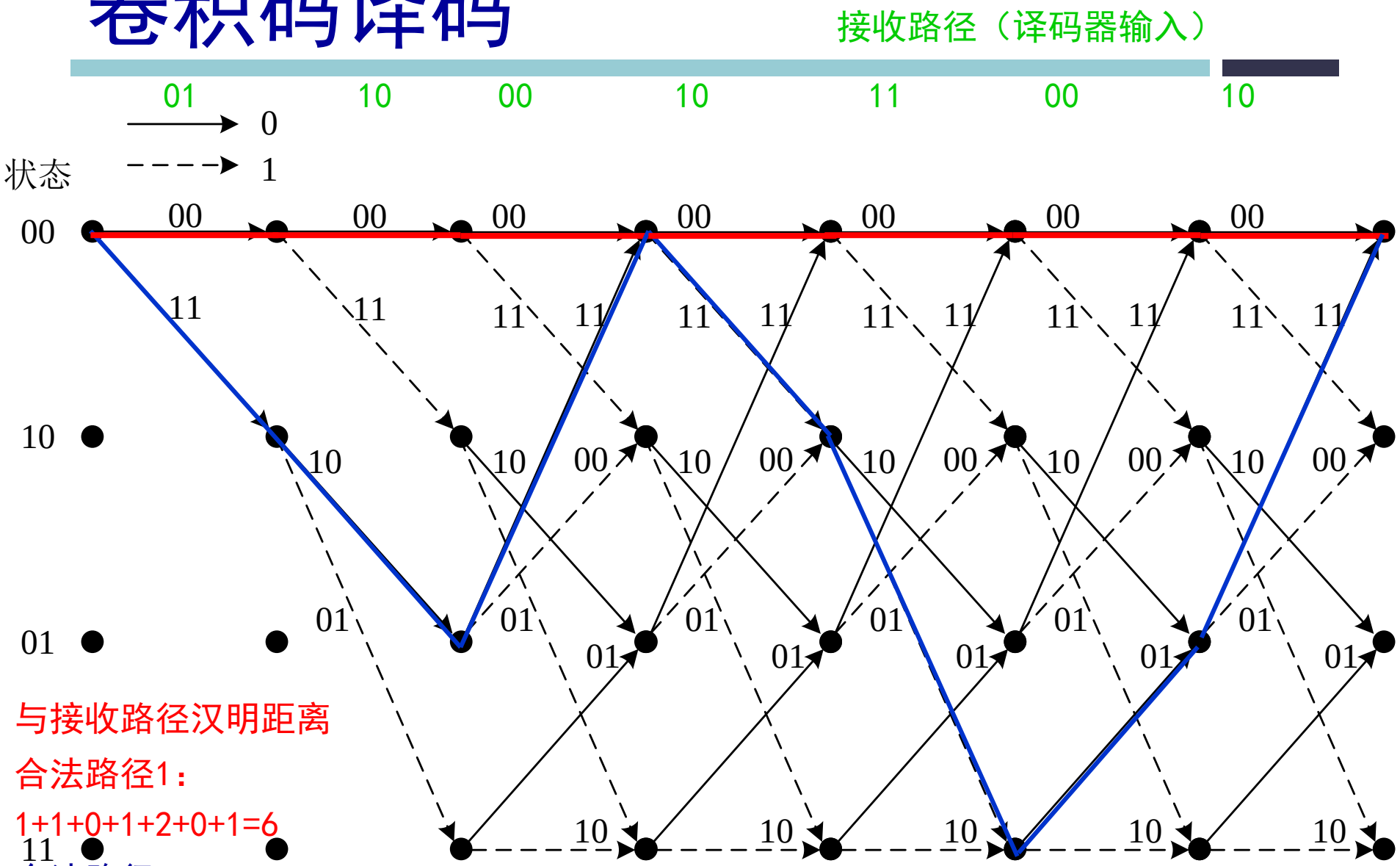
$$\hat{\mathbf{s}} = \max_{\mathbf{s}} \sum_i \log\left[\mathrm{P}(r_i|s_i)\right]$$

$$\hat{\mathbf{s}} = \min_{\mathbf{s}} \sum_i |r_i - s_i|^2$$ 最小欧氏距离

在什么条件下是等价的

后验概率、似然概率、欧氏距离、汉明距离、相关值等都可作为度量

卷积码译码



与接收路径汉明距离

合法路径1：

1+1+0+1+2+0+1=6

合法路径2：

1+0+2+1+1+1+1=7

合法路径1与接收路径的汉明距离更小

Viterbi译码

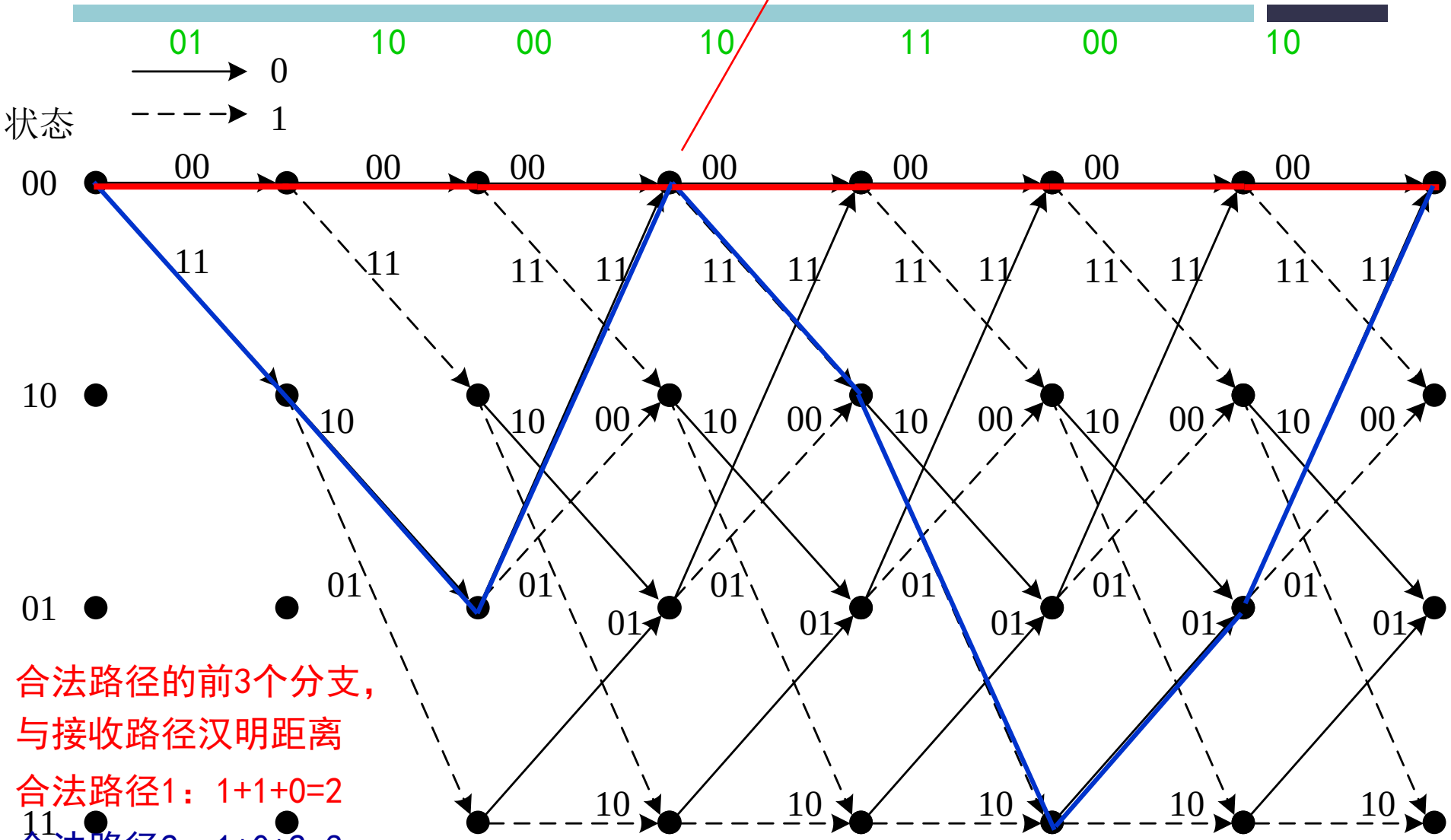
- 当输入信息序列长度为k bit时，共有2^k个合法路径，直接比较各条合法路径与接收路径的似然，复杂度难以承受
- Viterbi译码算法通过提前去除一些不可能成为最佳路径的路径，从而有效地降低了复杂度，而且仍然是一种最大似然译码算法，因此具有很好的性能

安德鲁·维特比(Andrew J. Viterbi)
CDMA之父，高通前首席科学家
美国国家科学奖章获得者

卷积码译码

此时就可以确定路径2（以及以111011开头的序列）不可能是具有最短距离的合法路径

接收路径（译码器输入）



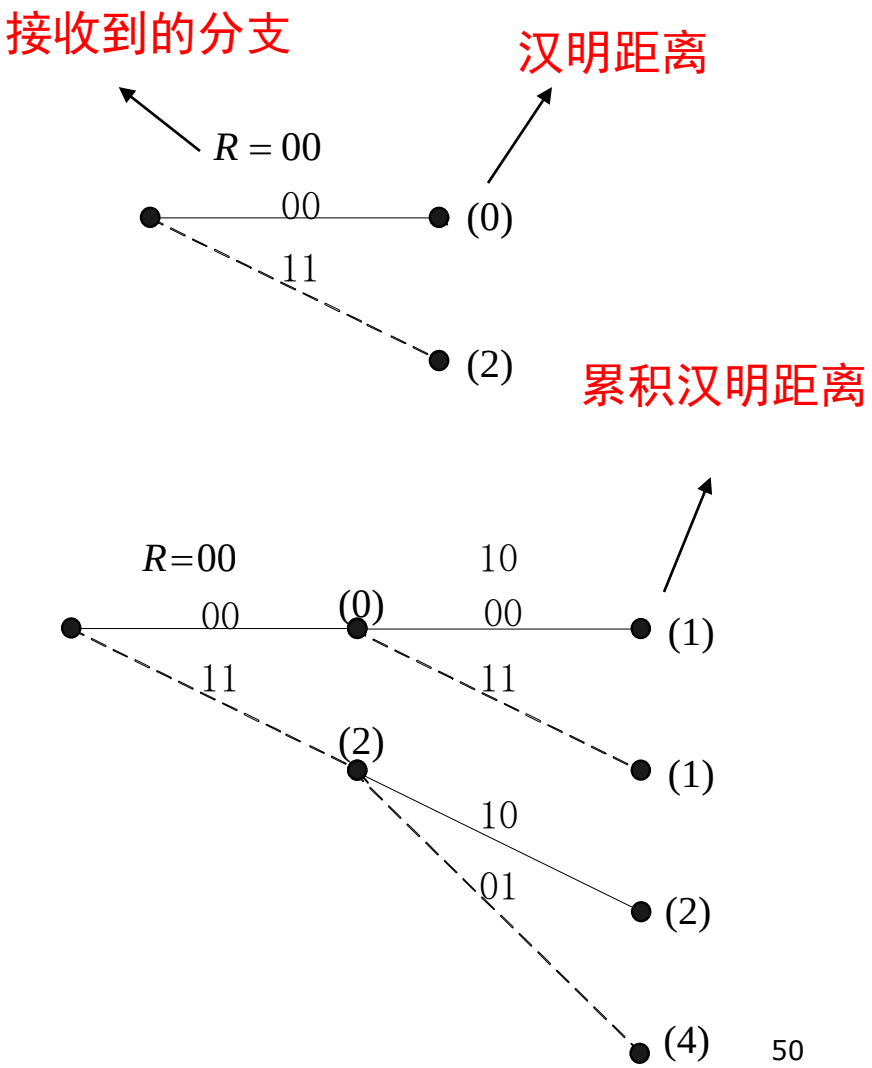
合法路径的前3个分支，与接收路径汉明距离

合法路径1：1+1+0=2

合法路径2：1+0+2=3

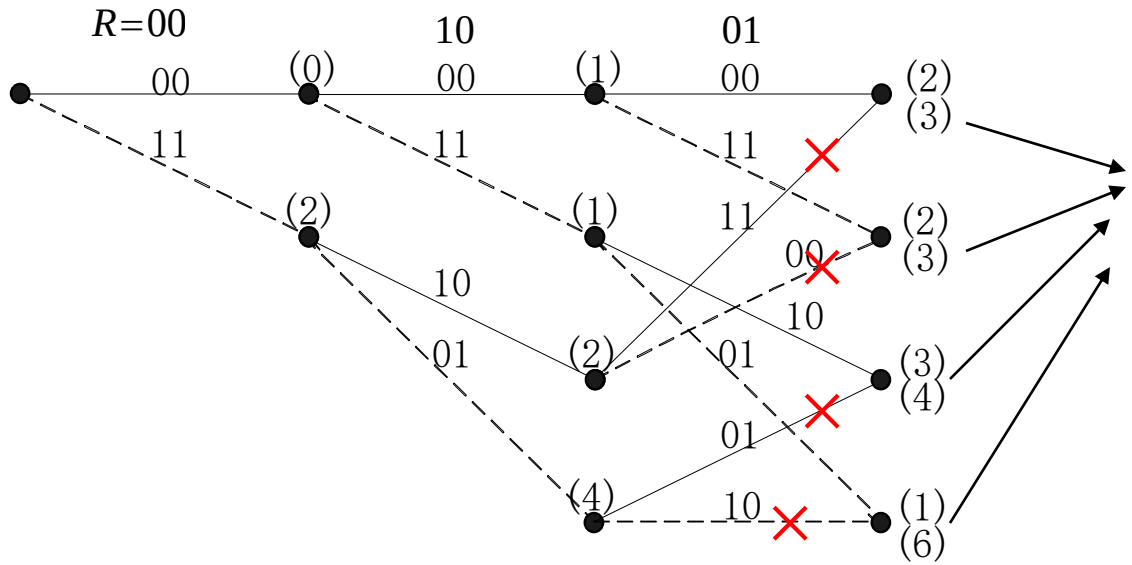
Viterbi译码示例

(2, 1, 3)卷积码，假设信息为全0序列，则编码输出也为全0序列，经信道传输后的接收序列为001001000…（存在误码）

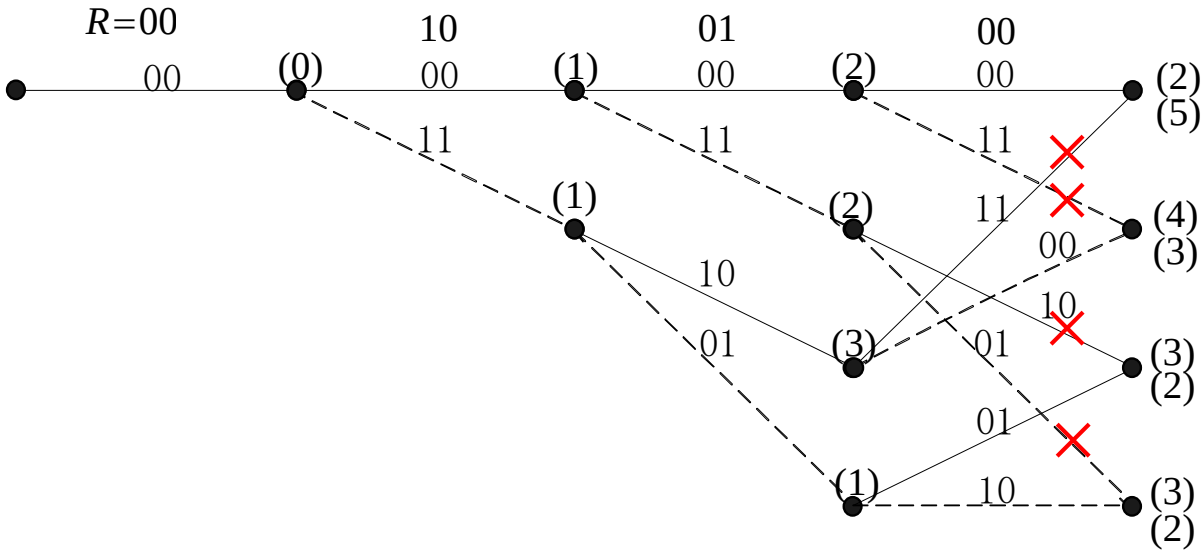


Viterbi译码

减少不必要的路径

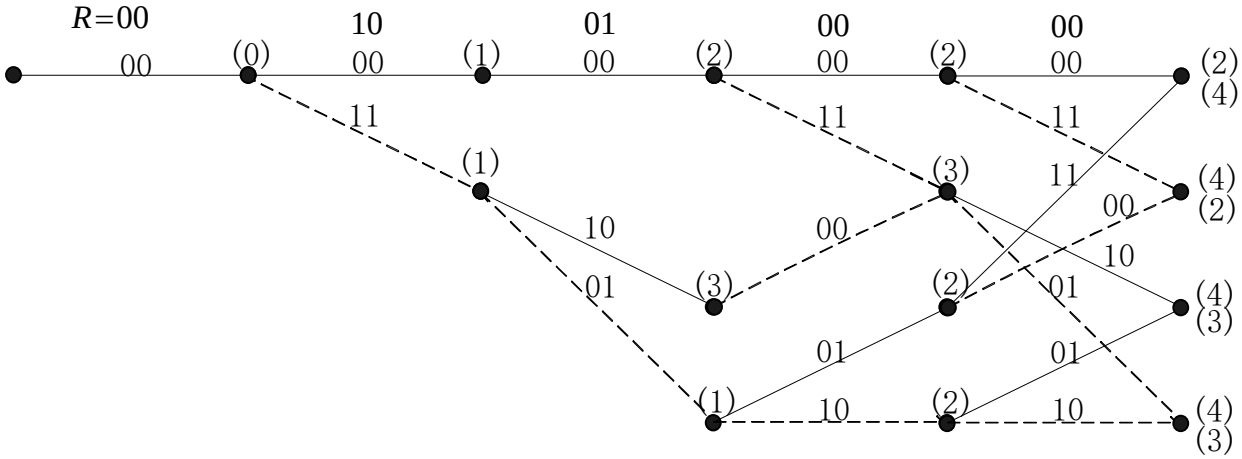


多个路径汇聚到同一节点，保留汉明距较小的路径（幸存路径），舍弃其他路径（不可能是最优路径），从而降低复杂度。



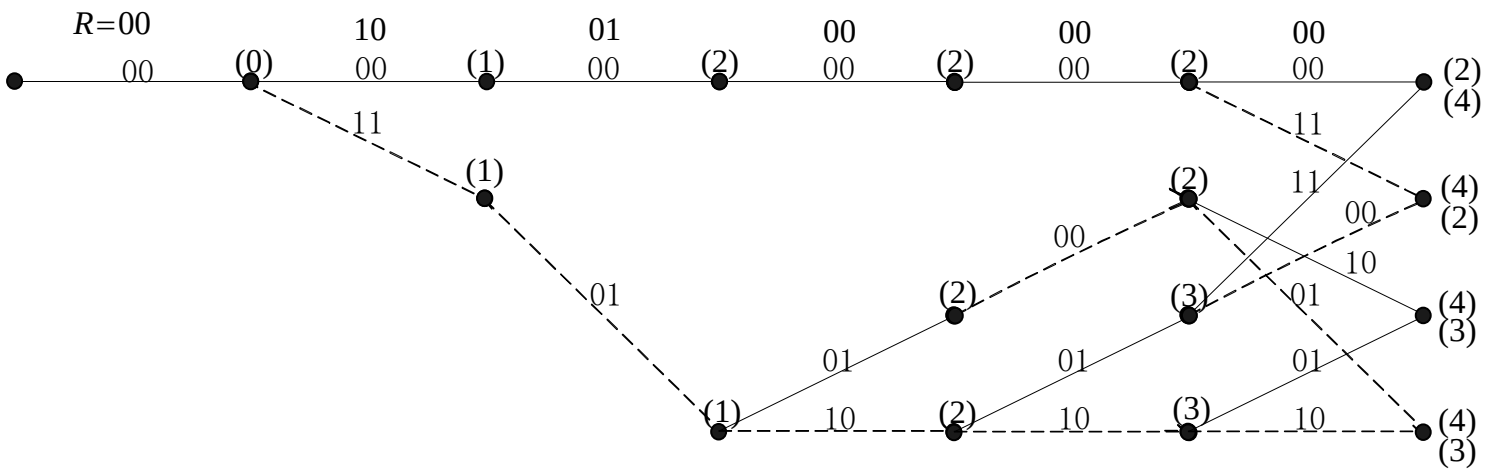
Viterbi译码可分为三步：一加，二比，三选。

Viterbi译码



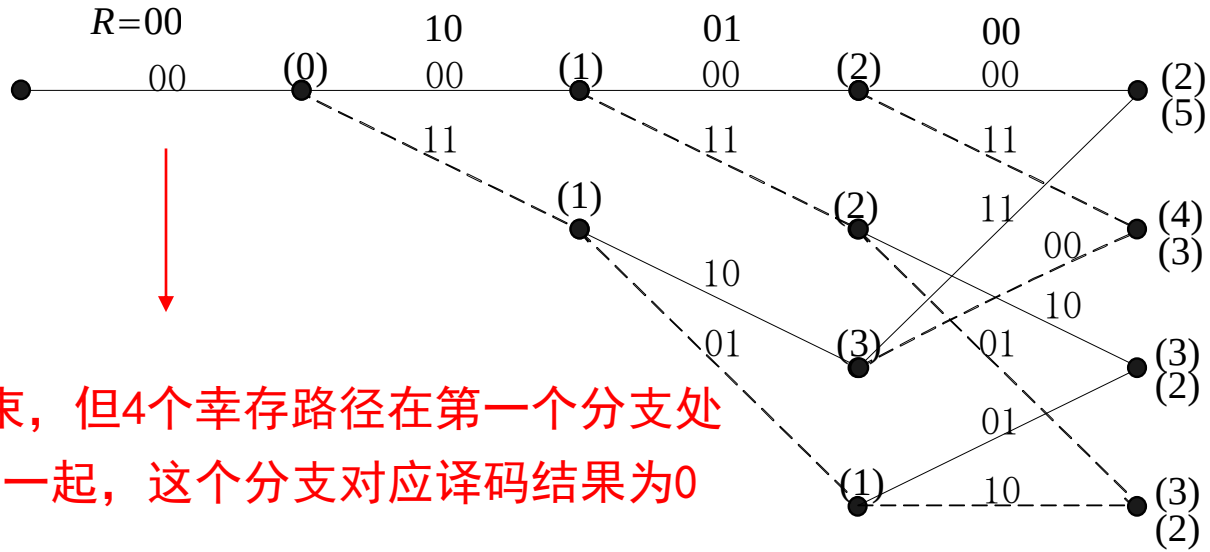
无论有多少个分支，幸存路径数不会超过状态数。

最后选择最佳度量的路径，根据路径可得译码输出（实线为0，虚线为1）



回溯长度L

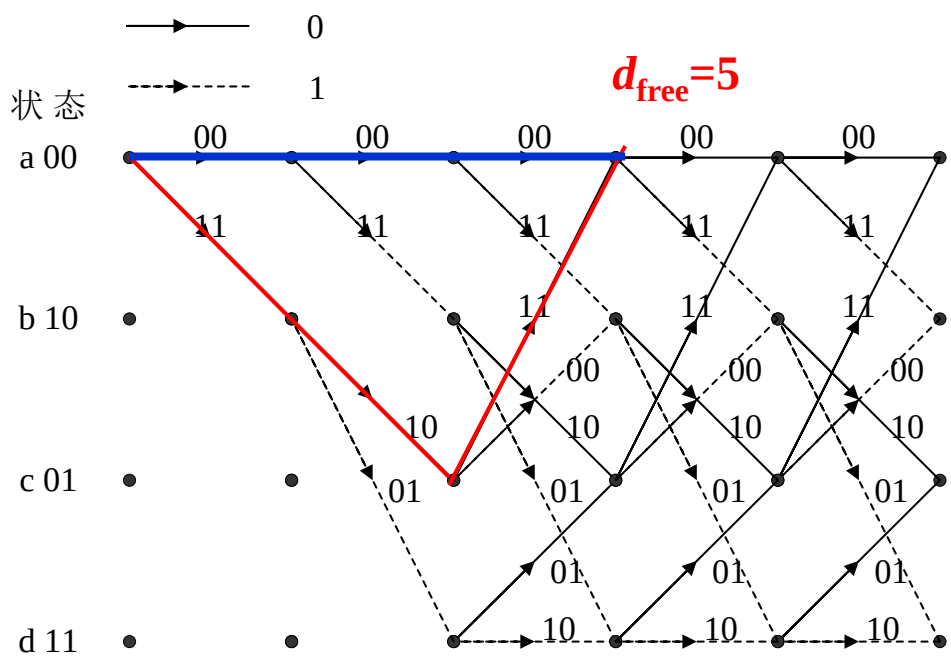
- 观察译码网格图，回溯一定的分支，可以发现所有幸存路径已经收敛到一起，即无论最终选择是哪一条合法路径，都不影响最初几个时刻的译码结果
- 因此此时就可以输出译码判决，而无需要走完整个网格图
- 回溯长度一般取(5~7)K
- 降低了译码时延和存储量
- 卷积码具有固定的译码时延特别适应时延敏感应用



译码尚未结束，但4个幸存路径在第一个分支处已经收敛到一起，这个分支对应译码结果为0

自由距离

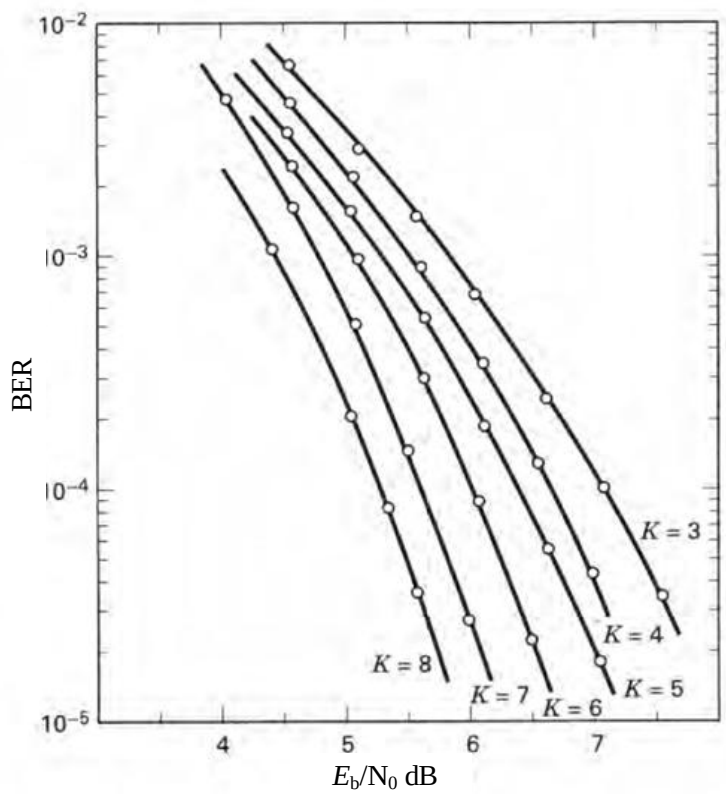
- 线性分组码纠错能力取决于最小汉明距离 $d_{\min}$
- 卷积码没有分组的概念，其纠错能力可用自由距离d_{free}表示
- 自由距离定义：两个合法路径之间的最小汉明距离
- 自由距离计算：从全零状态出发，经历非全零状态，又回到全零状态的这段路径的汉明重（即与全零路径的汉明距）
- 自由距离越大，纠正突发错误能力越强



约束长度选取

- 约束长度K越长，意味着卷积码输出码字前后相关范围更广，自由距离也随之增大，纠错能力更强，编码增益更大
- 但状态数和幸存路径数随约束长度指数增长
- 约束长度一般为5~9，更长约束长度（11~13）的卷积码一般应用于深空通信中

矛盾普遍存在
权衡在所难免

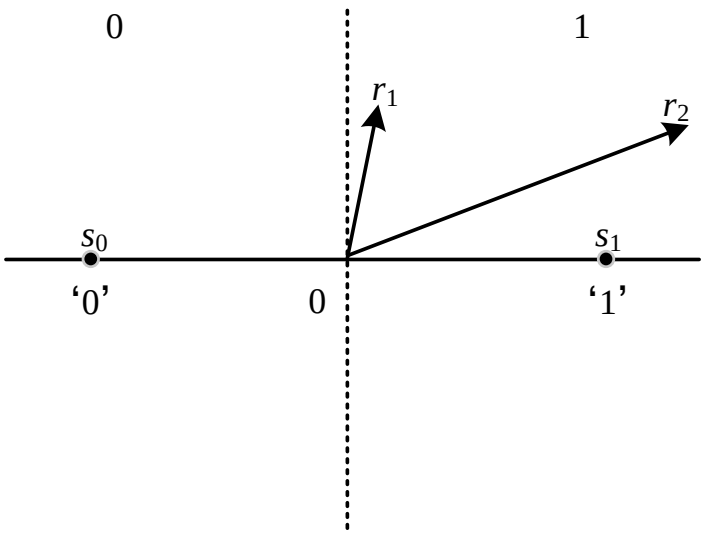


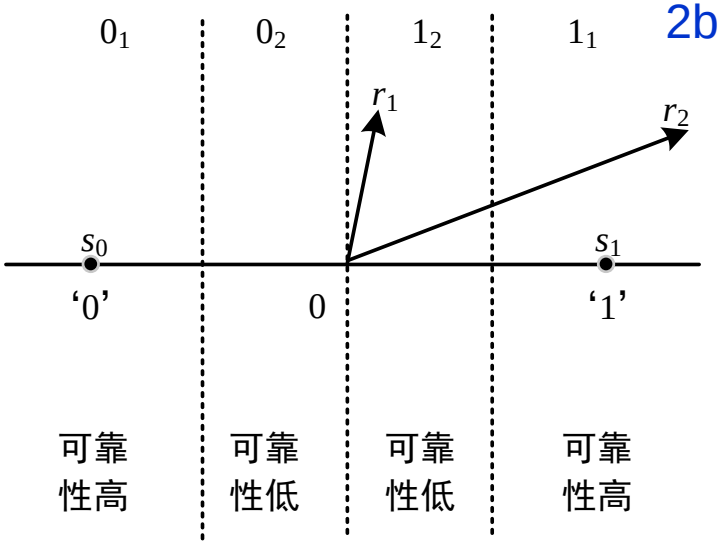
删余

- 卷积码的码率较为固定(如1/2 ，1/3等)，通过删除编码序列中的部分比特，减低冗余，提高码率，获得所需的码率(如3/4，3/5，2/5等)。译码时则在接收序列相应位置先添加0再行译码。
- 删余需要在一定的位置上进行（根据删除图案），避免某个比特所有信息都被删除，造成性能严重恶化。

Viterbi软判决

- 硬判决译码利用接收解调判决值进行译码，而没有利用判决值的可靠性信息（即软信息）。
- 利用软信息，对接收信号进行量化，尽可能保留接收信号携带的信息。



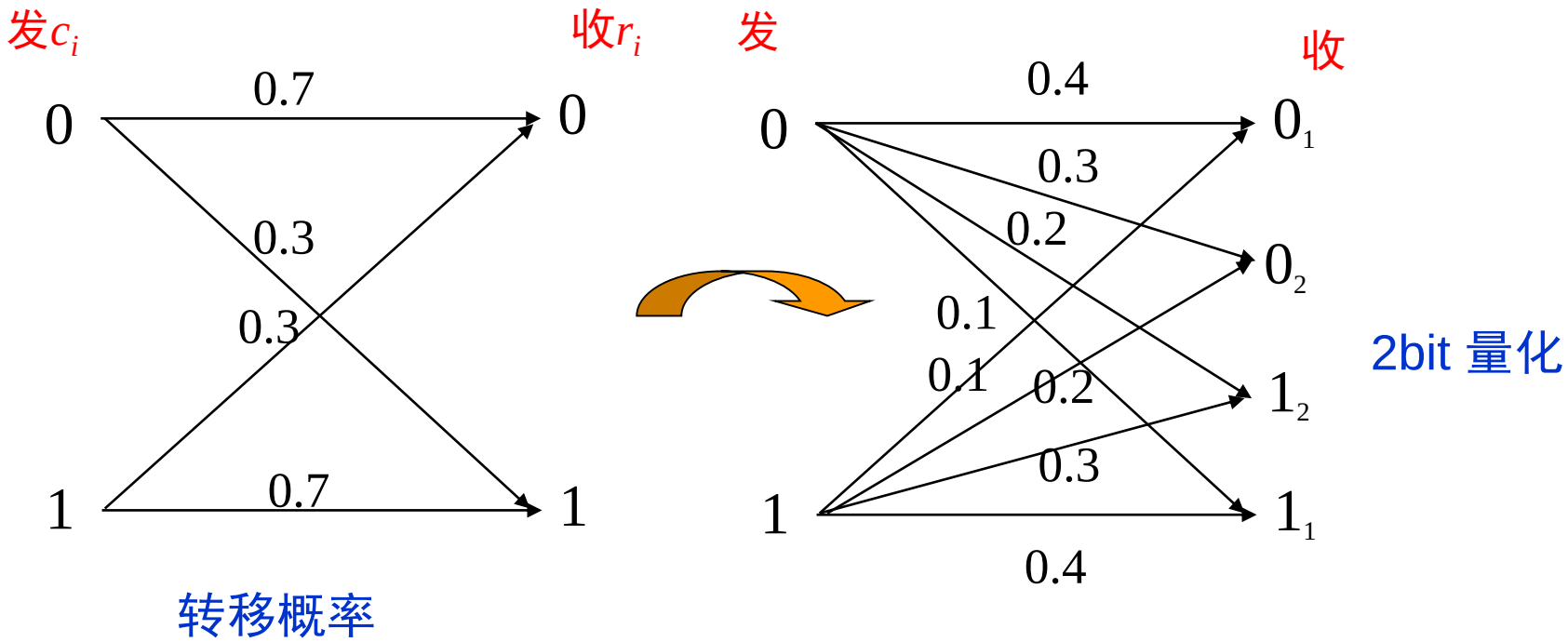


2bit量化

量化bit数越多，保留的信息也越多

Viterbi软判决

- 硬判决译码采用汉明距离，而软判决则采用欧氏距离、相关值或似然值做度量，可以提高译码性能。



Viterbi软判决

最大似然 $\max_{\mathbf{c}}[P(\mathbf{r}\,|\,\mathbf{c})] \Leftrightarrow \max_{\mathbf{c}} \sum_i \log P(r_i^{(j)}\,|\,c_i^{(j)})$ 调制和信道都无记忆

c \ r	0_1	0_2	1_2	1_1
0	-0.40	-0.52	-0.70	-1
1	-1	-0.70	-0.52	-0.40

$\log P(r_i^{(j)}\,|\,c_i^{(j)})$ 似然概率的对数值是负小数

通过线性变换为正整数，方便计算和比较（不是必须的）

c \ r	0_1	0_2	1_2	1_1
0	10	8	5	0
1	0	5	8	10

$b=1, a=16.8$

$a[\log P(r_i^{(j)}\,|\,c_i^{(j)})+b]$

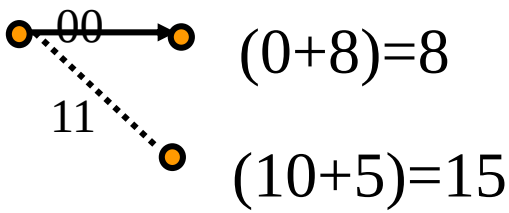
m=10110100

C=(11 10 00 01 01 00 10 11)

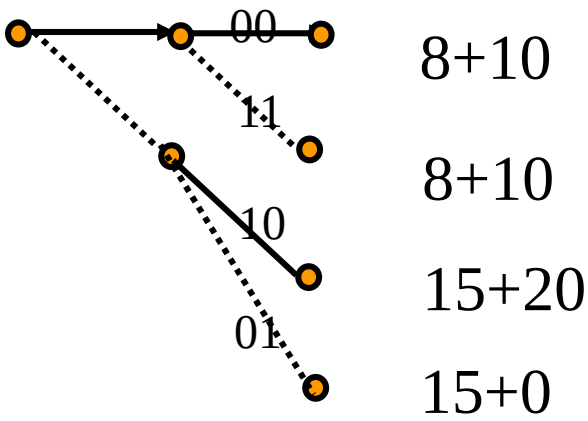
R=(1_1,0_2,1_1,0_1,0_1,0_2,0_1,1_1,1_2,1_1, 0_1,0_1, 1_1,1_2, 0_2,1_1)

Viterbi软判决

$r_0=(1_1,0_2)$

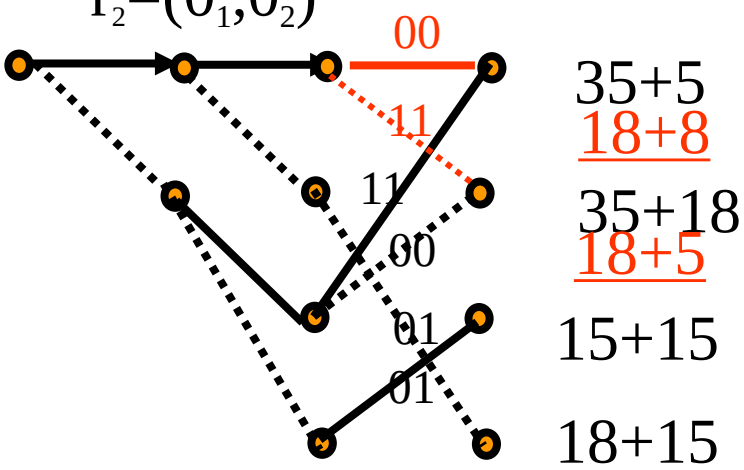


$r_1=(1_1,0_1)$

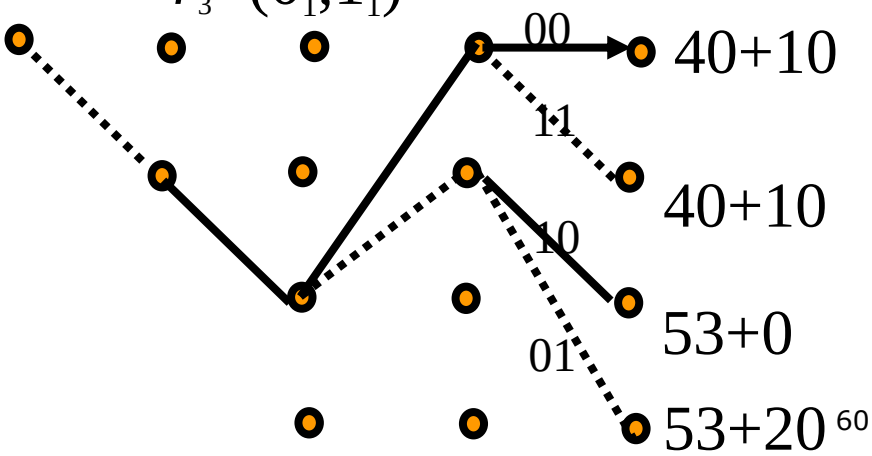


保留度量较大的路径

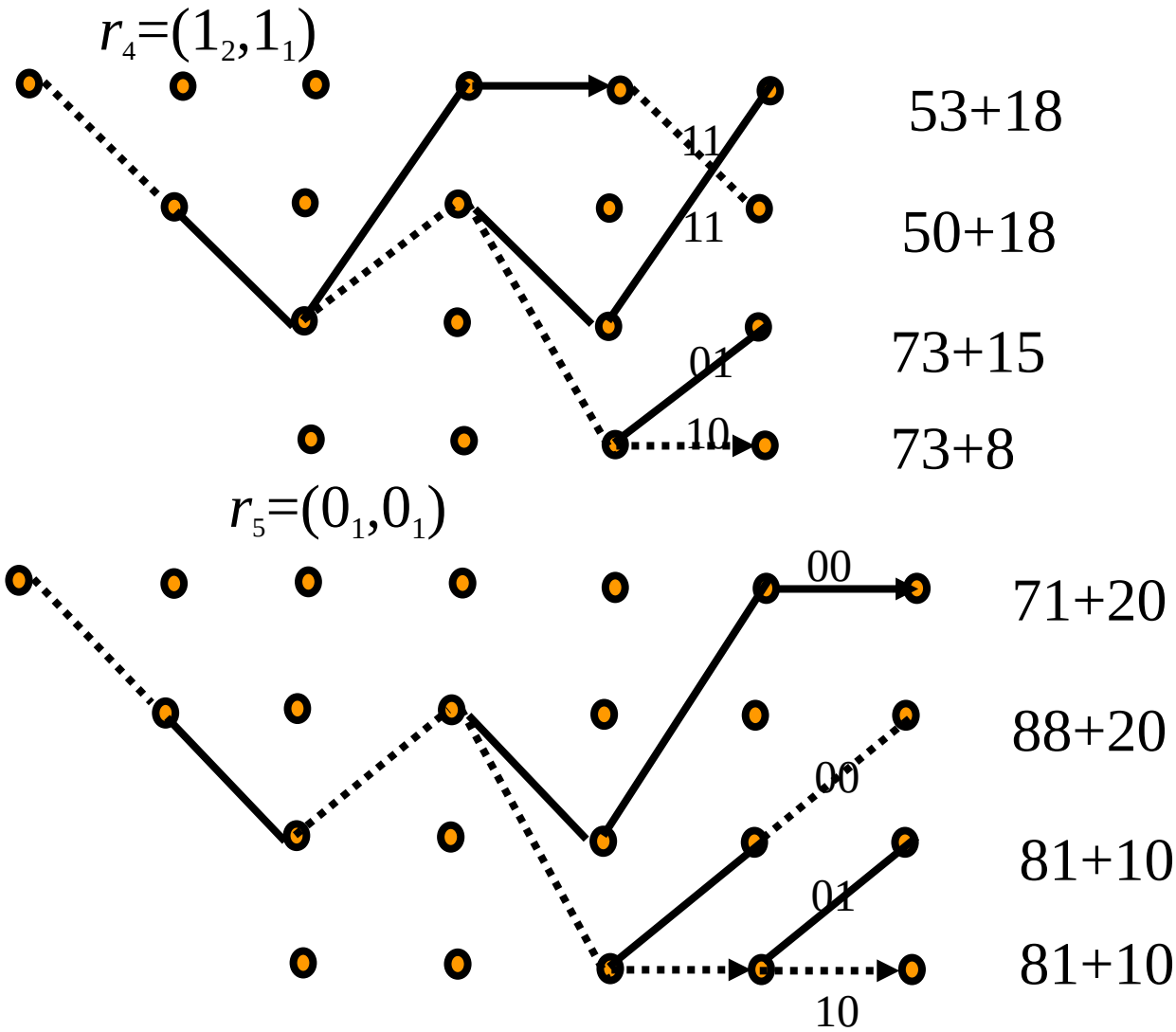
$r_2=(0_1,0_2)$



$r_3=(0_1,1_1)$

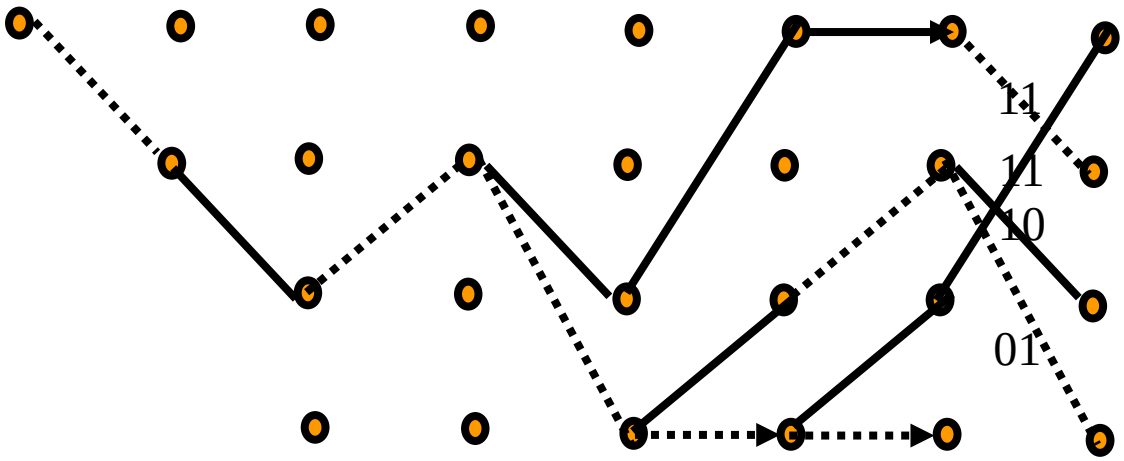


Viterbi软判决


$r_4=(1_2,1_1)$
11
11
01
10
53+18
50+18
73+15
73+8
$r_5=(0_1,0_1)$
00
00
01
10
71+20
88+20
81+10
81+10

Viterbi软判决

$r_6=(1_1,1_2)$



$\hat{m}$

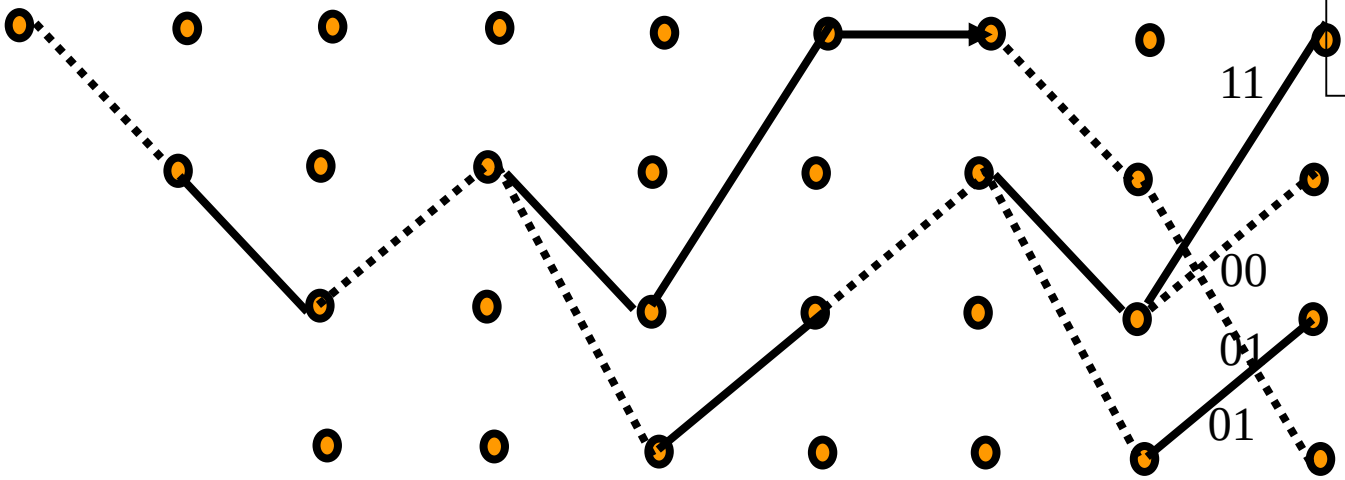
91+18 (1011100)

91+18 (1010001)

108+15 (1011010)

108+8 (1011011)

$r_7=(0_2,1_1)$



$\hat{m}$

$\boxed{\text{123+15 (10110100)}}$

123+8 (10110101)

116+18 (10110110)

109+18 (10100011)

Viterbi软判决

$\boldsymbol{m}$=10110100 $\xleftrightarrow{\text{译码正确}}$ $\hat{m}$ =10110100

软判决

$\boldsymbol{C}$=(11 10 00 01 01 00 10 11)

$\boldsymbol{R}$=(1_1,$\underline{0_2}$,1_1,0_1,0_1,0_2,0_1,1_1,$\underline{1_2}$,1_1, 0_1,0_1, 1_1,$\underline{1_2}$, $\underline{0_2}$,1_1)

$\boldsymbol{m}$=10110100 $\xleftrightarrow{\text{译码错误}}$ $\hat{m}$ =10110110

硬判决

$\boldsymbol{C}$=(11 10 00 01 01 00 10 11)

$\boldsymbol{R}$=(10 10 00 01 11 00 11 01)

软判决具有更好的纠错性能（性能提升约2dB），但译码复杂度有一定程度的增加

信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

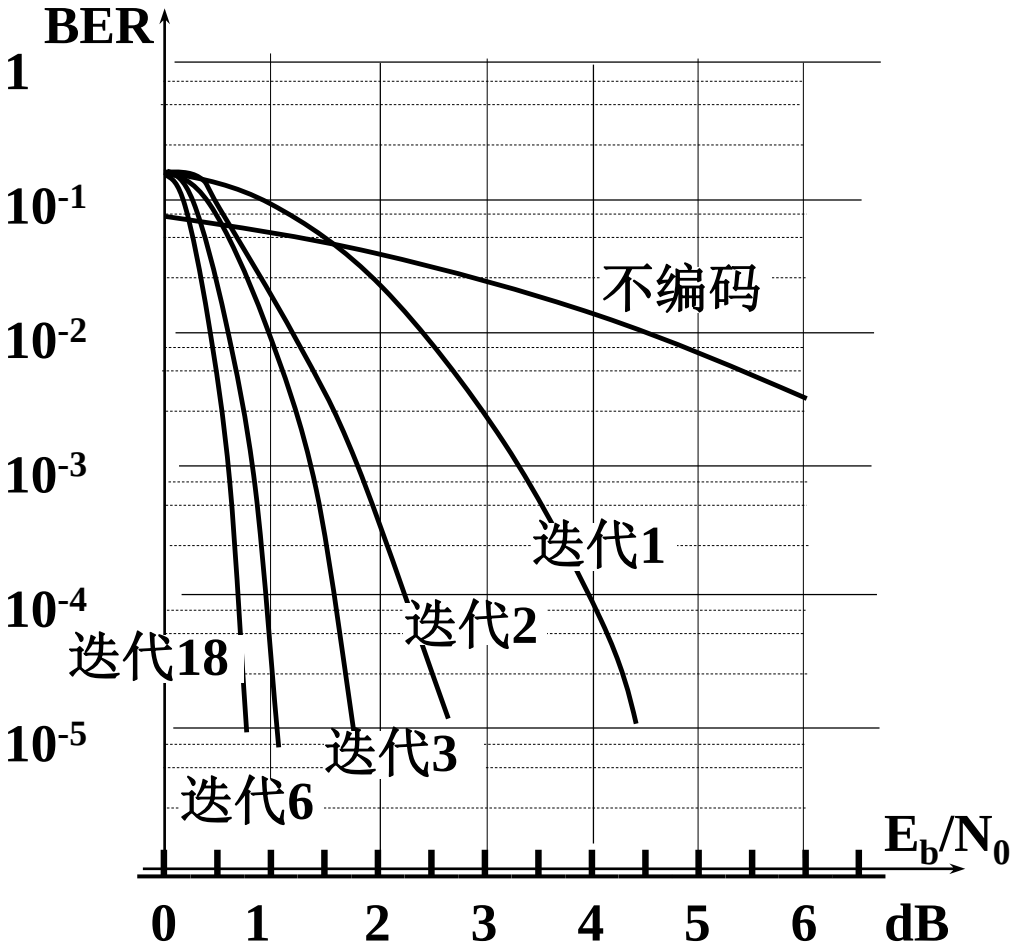
Turbo码

- 香农1948年提出信道编码定理后，经过近30年的努力，虽然新的信道编码不断提出，与香农容量限的差距不断缩小，但仍有不小差距（如采用RS+卷积码时差距为2.5dB）
- 1993 年，在日内瓦召开的ICC会议上，法国布列塔尼国立高等电信学校C.Berrou，A. Glavieux 和P. Thitimajshiwa公布了他们发明的Turbo 码，该码具有逼近香农容量限的性能（差距约0.7dB）
- Turbo码又称并行级联卷积码(Parallel Concatenated Conventional Code, PCCC)

C.Berrou（左）和A.Glavieux（右）

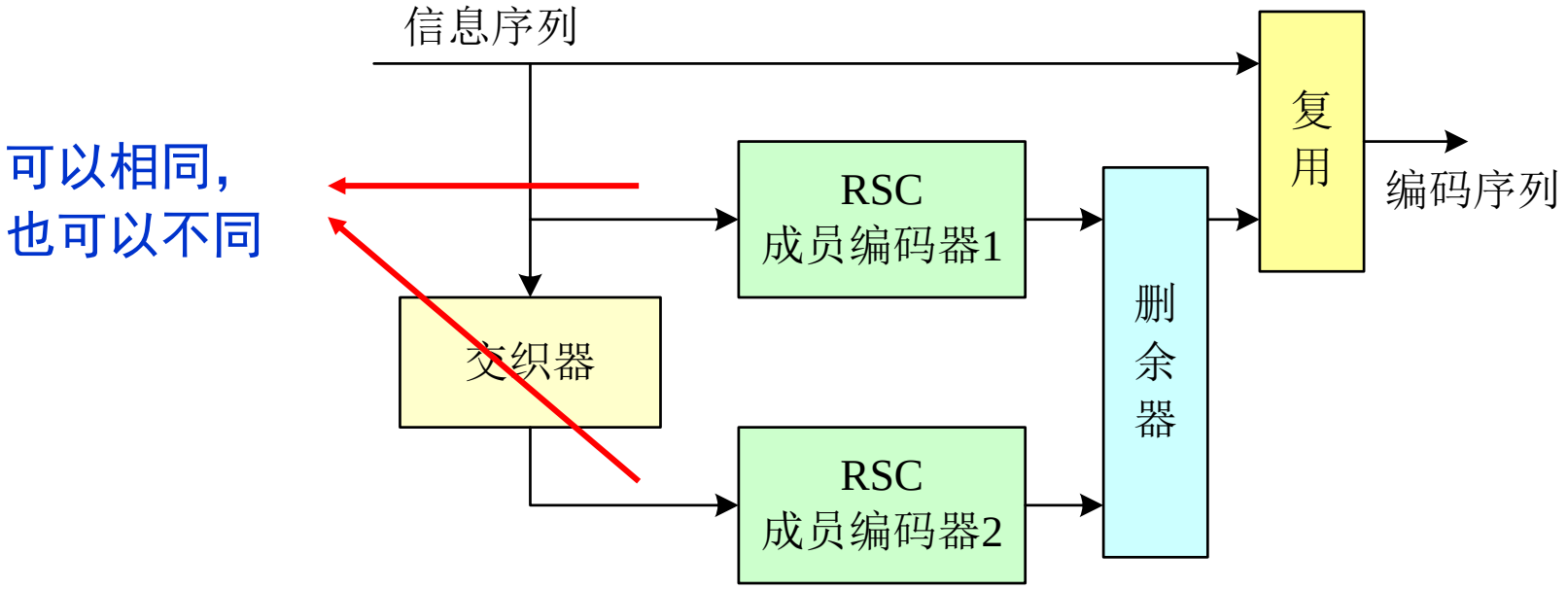
Turbo码的性能

- **C.Berrou**等给出了Turbo码在不同迭代次数下的误比特率(BER)性能曲线
  - 采用（2,1,5）RSC编码器
  - 长度为256×256非均匀交织器
- 随着迭代次数的增加，Turbo码的性能逐步逼近香农限。



不同迭代次数时的BER曲线

Turbo码的结构与编码



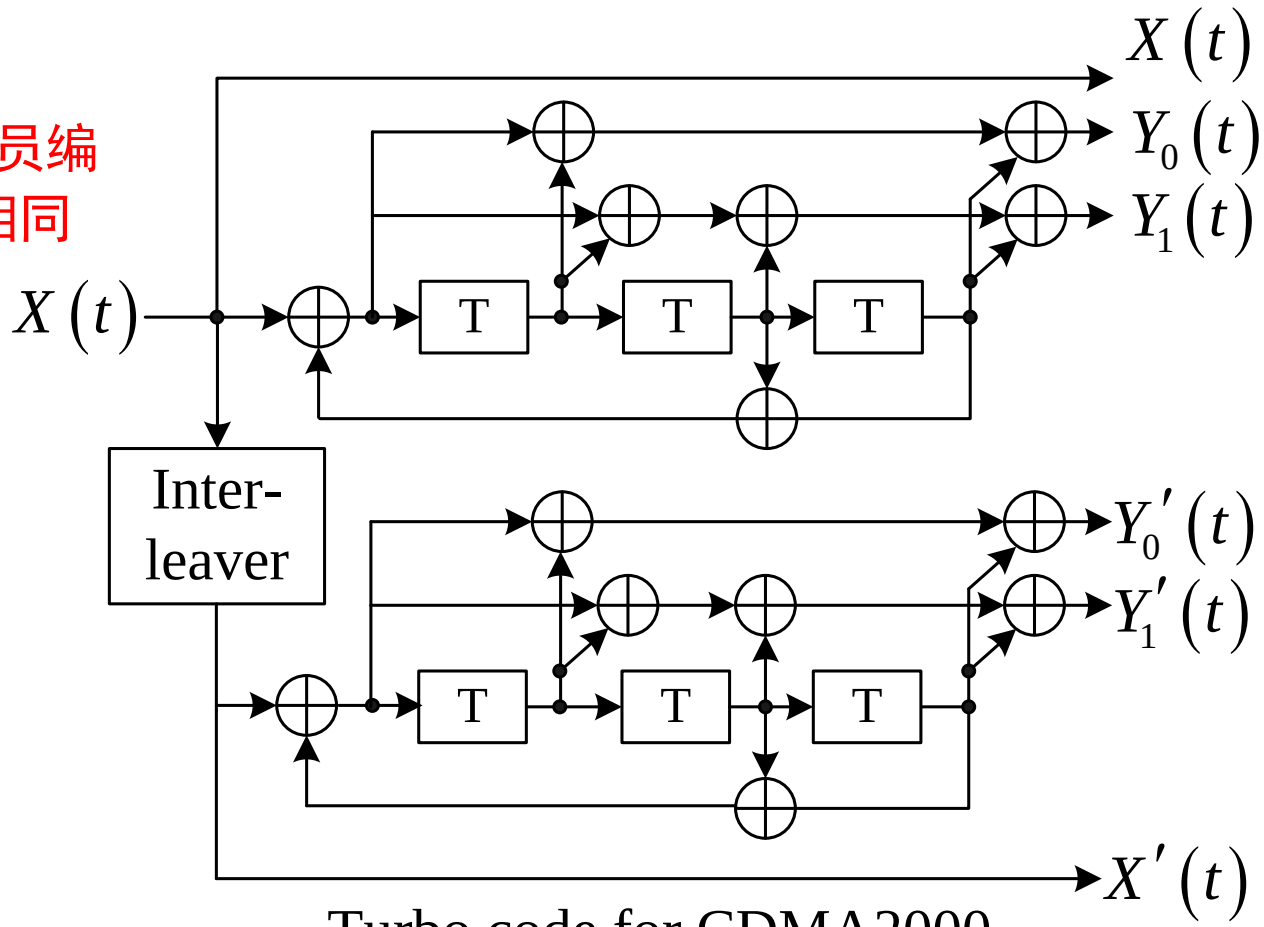
- Turbo码编码器包括两个并联的递归系统卷积码（RSC）成员编码器和一个交织器。
- 第一个编码器直接对信息序列进行编码，第二个编码器则对经过交织的信息序列进行编码。
- 两个编码器生成的校验比特在删余器中按一定规则进行删除，最后与信息比特复用从而得到编码序列。

递归系统卷积码（RSC）

- 卷积码可以分为两类：系统卷积码和非系统卷积码
- 系统卷积码的生成码字中包括输入的信息序列
- 递归系统卷积码指的是有输出反馈回输入端的系统卷积码。
- 递归系统卷积码的约束长度K=3～5时性能最好
- CDMA2000标准中的Turbo码采用约束长度为4的递归系统卷积码作为成员码

CDMA2000标准中的Turbo码结构

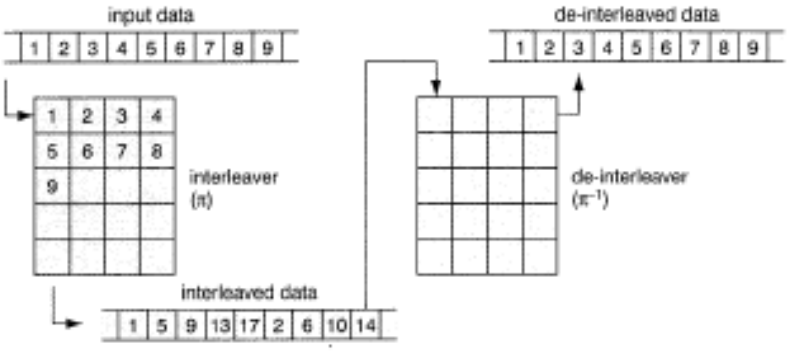
两个成员编码器相同



Turbo code for CDMA2000

Turbo码中的交织器

- 交织器将进入成员编码器2的信息比特序列进行重新排列，使两个成员编码器的输入序列尽可能不相关，从而在某种程度上实现了随机编码。
- 常用的交织器有以下几种：
  - 行列交织器
  - 非均匀交织器
  - 随机交织器
  - S交织器
  - 确定性交织器
  - .....

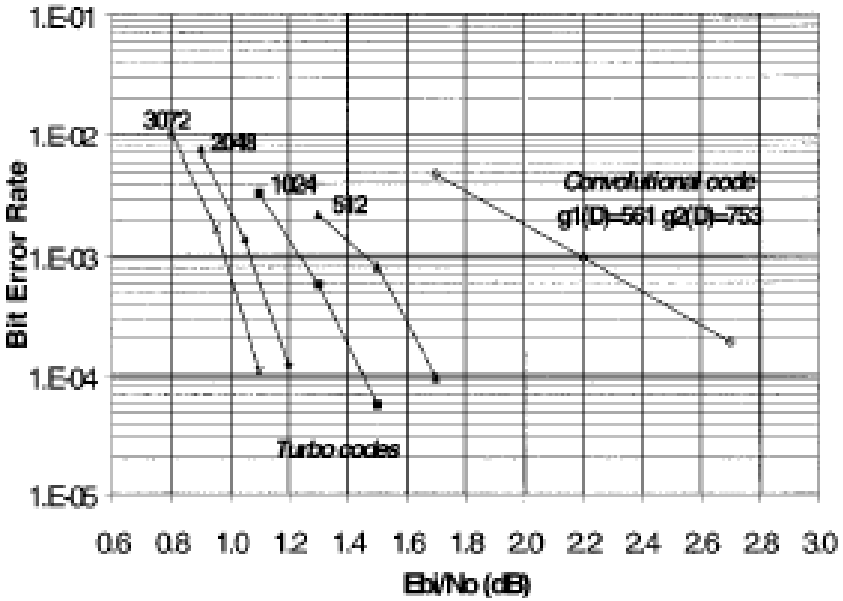


这里的交织器在Turbo编码器内部，其作用与编码器外的交织器作用不同。

- 在CDMA2000采用确定性交织器。

确定性交织器性能接近随机交织器，实现比较简单。

交织长度对Turbo码性能的影响



交织器越大，性能越好。

删余器(Puncturing)

- 删余器按照一定的规则删除部分比特，从而实现码率的提升

2节拍（每输入2bit） Turbo码删除图案

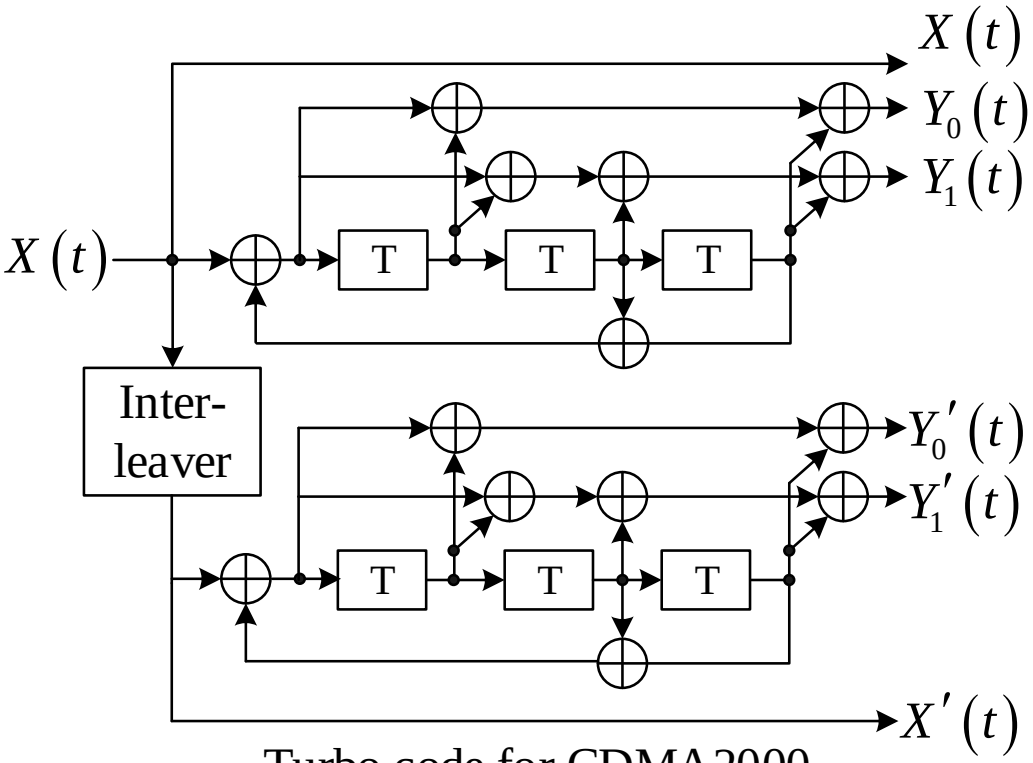
Turbo码率	1/2	1/3	1/4
X(t)	00	00	00
X’(t)	00	00	00
$Y_0(t)$	10	11	11
$Y_1(t)$	00	00	10
$Y'_0(t)$	01	11	01
$Y'_1(t)$	00	00	11

删除图案中的‘0’表示删除

(如R=1/2时，$Y_0(t)$的删除图案为10，表示前后输出的2 bit删其中1个，$Y_1(t)$的删除图案为00，表示全部删除)

Turbo编码举例

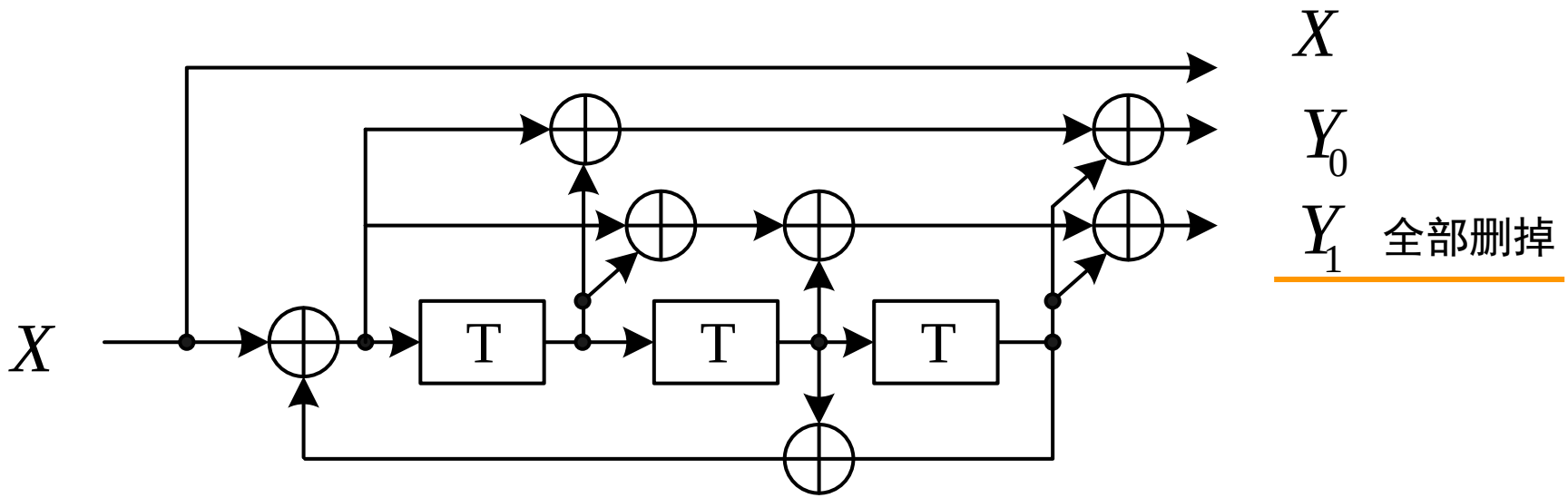
- CDMA2000标准中采用的成员编码器是约束长度为4的八状态RSC编码器，生成多项式：



Turbo code for CDMA2000

Turbo编码举例(R=1/2和1/3)

RSC1输入信息序列（未交织）X=[1,0,1,0,0,1,0,0]



X, Y_0

成员编码器1输出=[1 1, 0 1, 1 0, 0 0, 0 1, 1 0, 0 0, 0 1]

Turbo编码举例

输入序列

1	0	1	0	0	1	0	0

1	0
1	0
0	1
0	0

交织器

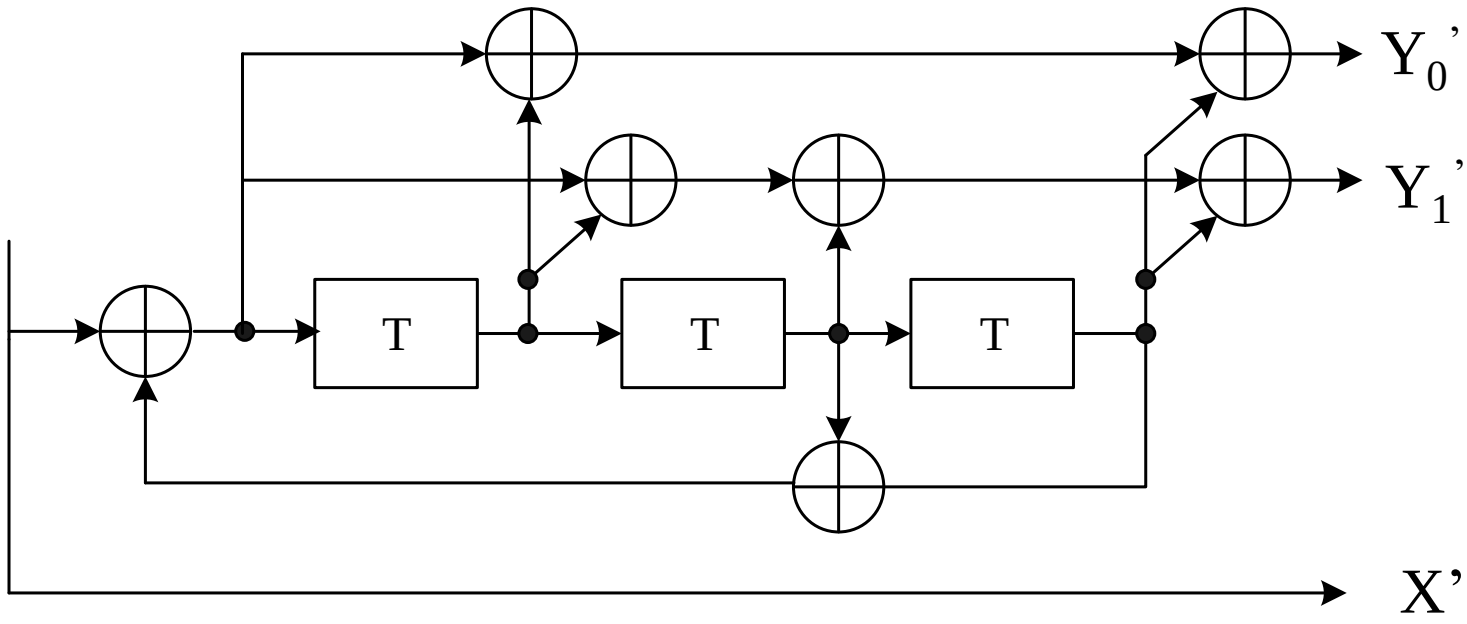
按行写入
按列读出

交织序列

1	1	0	0	0	0	1	0

Turbo编码举例

- RSC2输入信息序列（交织后）[1, 1, 0, 0, 0, 0, 1, 0]



X', Y_0'

- RSC2输出=[1 1, 1 0, 0 0, 0 0, 0 1, 0 0, 1 0, 0 0]

Turbo编码举例

- 删除和复用

  信息序列=[1,0,1,0,0,1,0,0]

  成员编码器1输出=[1 1, 0 1, 1 0, 0 0, 0 1, 1 0, 0 0, 0 1]

  成员编码器2输出=[1 1, 1 0, 0 0, 0 0, 0 1, 0 0, 1 0, 0 0]

- 码率为R＝1/3时,

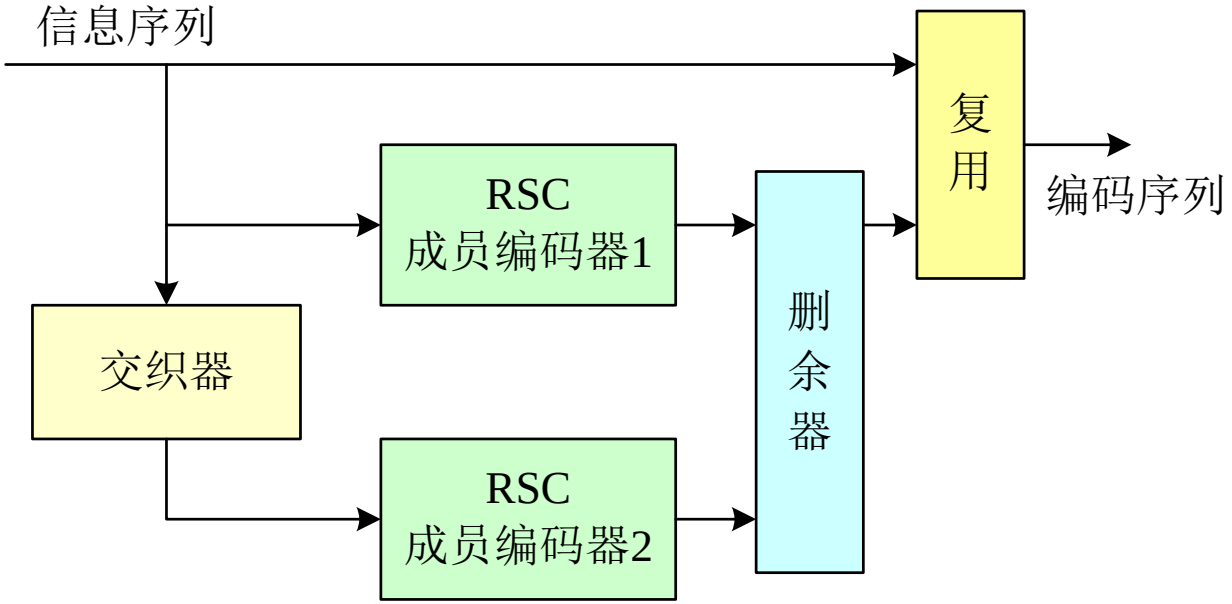
  编码输出=[1 1 1, 0 1 0, 1 0 0, 0 0 0, 0 1 1, 1 0 0, 0 0 0, 0 1 0]

- 码率为R＝1/2时,

  编码输出=[1 1, 0 0, 1 0, 0 0, 0 1, 1 0, 0 0, 0 0]

Turbo编码举例

码率为R＝1/3时的输出

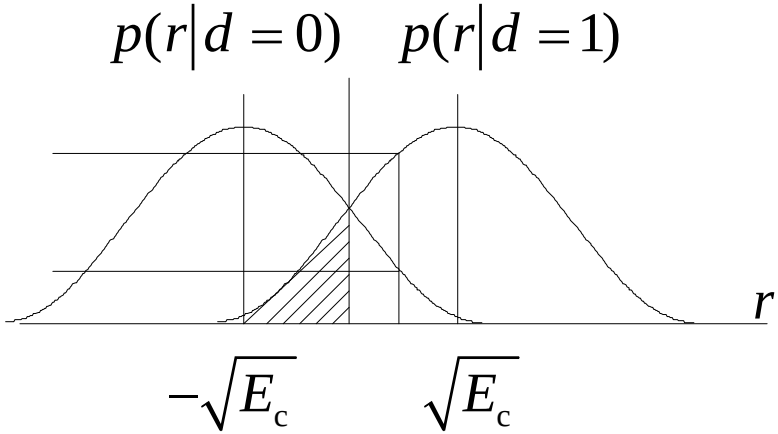


软信息

- 调制时对解调信号进行硬判决后译码，会导致信息的损失。
- 软判决Viterbi算法对解调信号进行量化，可以提高性能
- 量化比特数越多，信息损失越少，性能越好。
- 采用对数似然比（软信息）表示解调接收信号具有最佳的性能

对数似然比（LLR）

$$L(r|d)=\log\frac{p(r|d=1)}{p(r|d=0)}$$



- 在进行硬判决时，如 $L(r|d)>0$，即 $p(r|d=1)>p(r|d=0)$，则应判决d=1；反之则判决d=0；$|L(r|d)|$ 越大，表明判决的可靠性也越大。因此 $L(r|d)$ 的符号决定着硬判决值，而绝对值确定判决可靠性。

信道信息

经过信道传输后接收信号r的后验概率

$$L(d\,|\,r)=\log\frac{P(d=1\,|\,r)}{P(d=0\,|\,r)}=\log\frac{p(r\,|\,d=1)P(d=1)}{p(r\,|\,d=0)P(d=0)}$$

$$=\log\frac{p(r\,|\,d=1)}{p(r\,|\,d=0)}+\log\frac{P(d=1)}{P(d=0)}$$

$$=L(r\,|\,d)+L(d)$$

信道信息：已知d条件下对信道输出r的度量

先验信息，显然当$P(d=1)=P(d=0)=1/2$时，$L(d)=0$

信道信息（信道软输出）

$$L(r|d)=\frac{P(r|d=1)}{P(r|d=0)}$$

简便起见，常令E_c=1

$$L_c(r)=\log_e\frac{p(r\mid d=1)}{p(r\mid d=0)}=\log_e\frac{\frac{1}{\sqrt{2\pi}\sigma}\exp(-\frac{(r-\sqrt{E_c})^2}{2\sigma^2})}{\frac{1}{\sqrt{2\pi}\sigma}\exp(-\frac{(r+\sqrt{E_c})^2}{2\sigma^2})}=\frac{2}{\sigma^2}\sqrt{E_c}r=\frac{2r}{\sigma^2}$$

在AWGN信道中，由 $\sigma^2=\frac{1}{2}N_0$

也可表示为 $L_c(r)=\frac{4r}{N_0}$

外信息

- 假定编码比特c_1、c_2、c_3满足约束关系$c_1 \oplus c_2 \oplus c_3 = 0$
- 假定c_1、c_2和c_3对应接收信号为0.2，-1.8和-1.1。（假定E_c=1）
- 根据接收信号解调判决，可得c_1=1，c_2=0，c_3=0。

显然不满足约束关系

- 而由约束关系$c_1 = c_2 \oplus c_3$，根据c_2和c_3的值可以推断出c_1=0。
- 做出这个判断所依据的信息来源于c_1相关的比特，而不是c_1本身的接收信号，因此称之为外信息。

则总信息为：

$$L(\hat{c}) = L_c(r) + L_e(c) + L(c)$$

总信息　信道信息　外信息　先验信息

信道译码主要是获得外信息的过程

外信息的计算

$c = c_1 \oplus c_2$，且c_1, c_2相互独立，可证明

$$L_e(c) = L(c_1 \oplus c_2) = -\mathrm{sgn}\left[L(c_1)\right] \cdot \mathrm{sgn}\left[L(c_2)\right] \cdot \Phi\left[\Phi(\left|L(c_1)\right|) + \Phi(\left|L(c_2)\right|)\right]$$

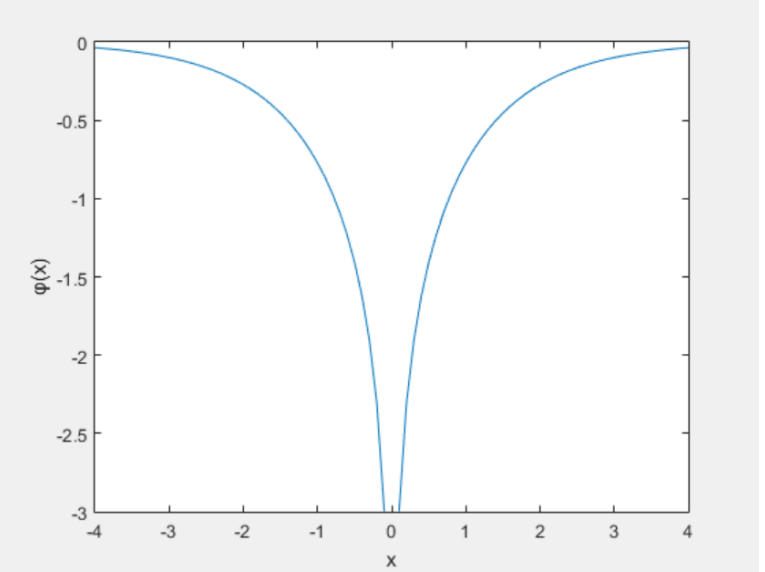
其中$\Phi(x) = \ln(\left|\tanh(\frac{x}{2})\right|) = \ln\left|\frac{e^x - 1}{e^x + 1}\right|$

推论：

如$c = c_1 \oplus c_2 \cdots \oplus c_n$，且$c_1, c_2, \cdots c_n$相互独立，则

$$L_e(c) = -\prod_{i=1}^{n} \mathrm{sgn}(L(c_i)) \cdot \Phi(\sum_{i=1}^{n} \Phi(\left|L(c_i)\right|))$$

x的绝对值越大，$\Phi(x)$绝对值越小



外信息

1）假定发端01等概，则先验信息为0

2）根据c_1、c_2 和c_3 对应接收信号，计算信道信息。

假定信噪比E_c/N_0=3dB=2，则N_0=0.5，则c_1、c_2 和c_3信道信息分别为1.6，-14.4和-8.8。

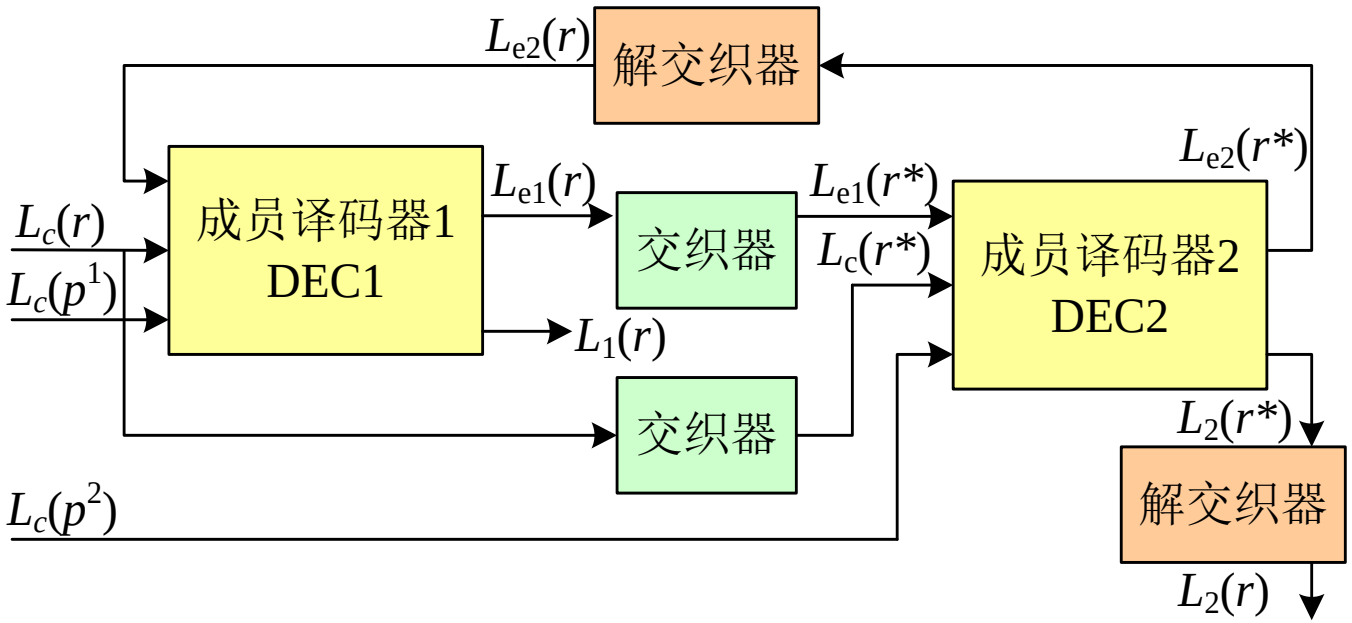
- 由外信息计算公式可得，c_1从c_2 和c_3获得的外信息为-8.796。
- c_1的总信息为1.6 -8.796= -7.196， c_1 判决为0。
- 通过外信息，纠正了c_1的错误。

满足约束关系

Turbo码译码器的结构

- 译码器由两个成员译码器、交织器和解交织器组成。
- 两个成员译码器分别对应编码器中的两个成员编码器。
- 交织器与编码器中的交织器完全相同。
- 由于信息比特在2个成员编码器中都进行了编码，因此译码过程中，2个成员译码器都能得到信息比特的外信息。
- 一个成员译码器输出的外信息将传递给另一个成员译码器。
- 每一次迭代，外信息将在两个成员译码器之间进行一次交换，整个译码需要进行多次迭代。

Turbo码译码器结构



- $L_c(r)$, $L_c(p^1)$, $L_c(p^2)$分别为信息比特、成员编码器1和2的校验比特的信道信息
- $L_{e1}(r)$： 从DEC1获得的信息比特d的外信息
- $L_{e2}(r^*)$：从DEC2获得的交织的信息比特d^* 的外信息

Turbo译码

- 发端的信息比特序列 [1 0 1 1 0 0 1]

根据解调输出确定信道信息	L_c	+1.2	$\boxed{+1.0}$	+0.4	+1.0	$\boxed{+1.0}$	–1.2	+0.6
DEC1译码后获得的总信息	L_1	+1.7	$\boxed{+0.8}$	$\boxed{-0.1}$	+2.5	$\boxed{+0.5}$	–2.7	+2.1
DEC1译码后获得的的信息比特的外信息	L_{e1}	+0.5	–0.2	–0.5	+1.5	–0.5	–1.5	+1.5
DEC2的译码后获得的总信息	L_2	+2.7	$\boxed{+0.3}$	+1.1	+3.3	–0.9	–4.2	+2.9
DEC2的译码后获得的信息比特的外信息	L_{e2}	+1.0	–0.5	+1.2	+0.8	–1.4	–1.5	+0.8

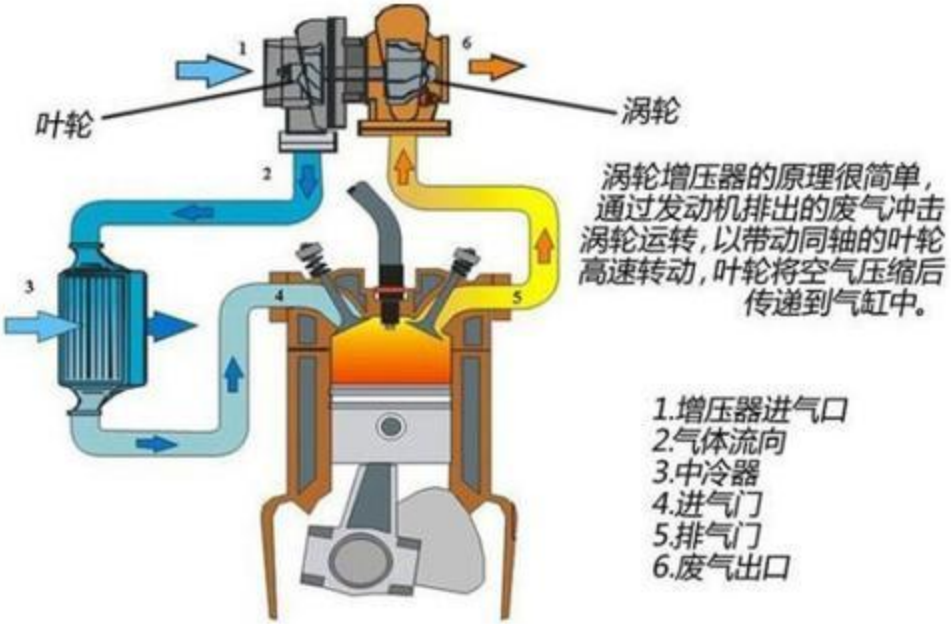
如直接用L_2做译码判决，仍存在1个错误，因此一般需要多次迭代。

$$L_c \rightarrow L_1 \qquad L_1 - L_c = L_{e1}$$

$$L_c + L_{e1} \rightarrow L_2 \qquad L_2 - L_c - L_{e1} = L_{e2}$$

Turbo码名称的由来

- Turbo码采用迭代译码方式，通过不断地迭代，不断收集外信息，从而提高了译码性能
- 原理类似涡轮增压（Turbo）发动机



Turbo码译码算法

- 一类源自Viterbi算法，追求码字（路径）的错误概率最小，如软输出Viterbi算法（SOVA）。
- 另一类源自BCJR（Bahl-Cocke-Jelinek-Raviv）算法，追求比特错误概率最小，如最大后验概率（MAP）算法。
- MAP算法性能要好于SOVA，但运算复杂度较大，需要较大存储空间，而且需要对SNR进行估计。

软输出算法(SOVA)

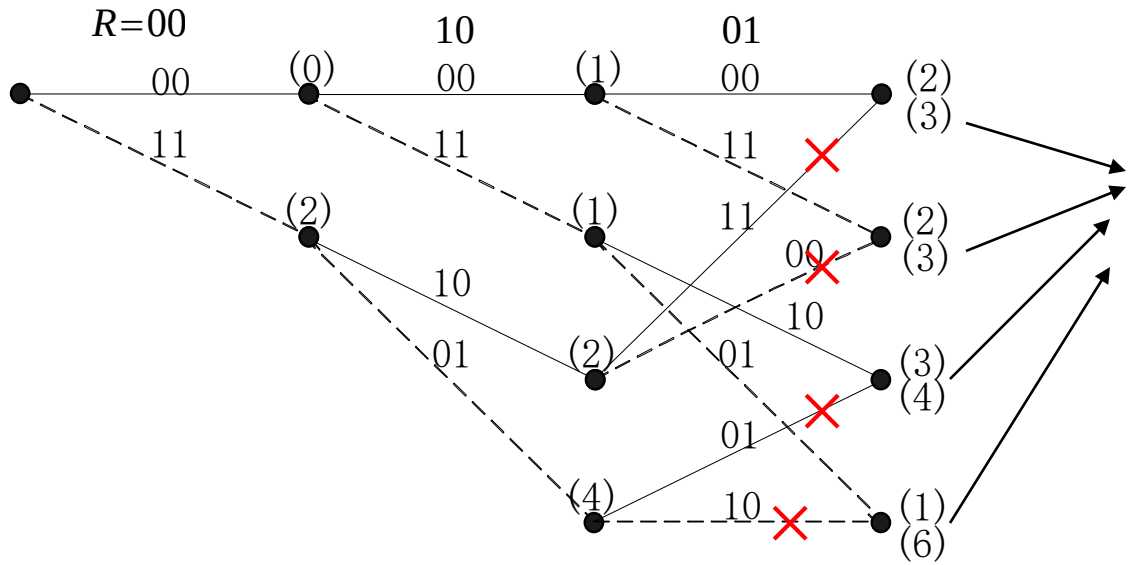
- Viterbi算法是在网格图中搜索具有最大似然的路径。为降低复杂度，在每个节点进行加-比-选，舍弃了似然度较小的路径，并未得到译码结果中各比特的可靠性（即选择的可靠性），无法直接用于Turbo译码

Andrew J.Viterbi

Joachim Hagenauer

- SOVA算法由Viterbi算法发展而来。不同之处在于进行“加-比-选”时，要比较幸存路径和竞争路径度量之差，用度量差作为选择可靠性的量度

Viterbi译码



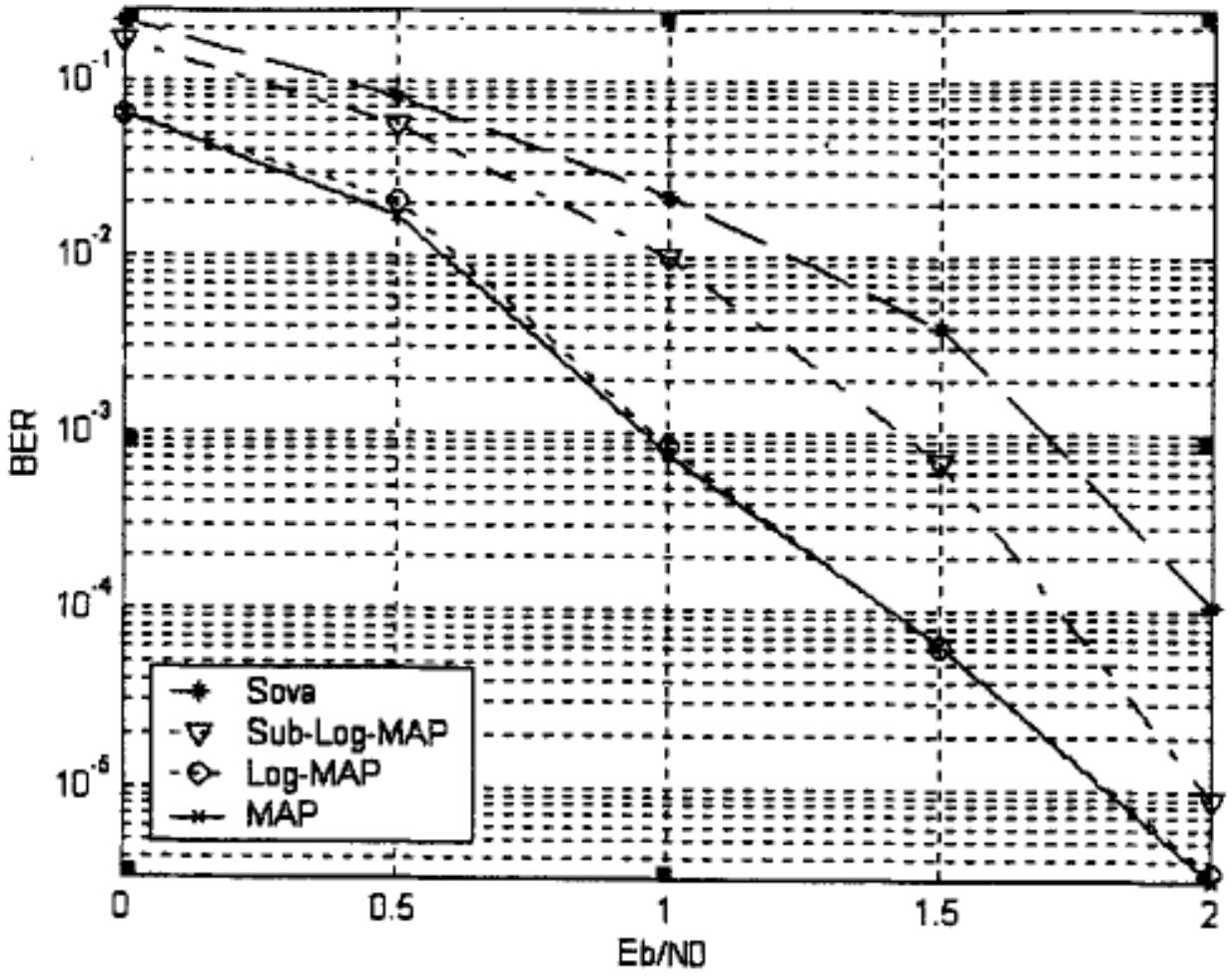
显然，当两个路径的度量差值越大，选择的可靠性越大

MAP(Maximum a Posteriori最大后验概率)算法

- 译码的方法是对于发送序列 $\boldsymbol{x}$，对应接收序列 $\boldsymbol{r}$，寻找$\boldsymbol{x}$使得P $(\boldsymbol{x}/\boldsymbol{r})$ 最大，即MAP算法是找出传输的编码序列中概率最大的信息比特。

Fred Jelinek

SOVA与MAP性能比较



Turbo码的缺点

- 译码复杂度大

  采用迭代译码算法，要进行交织和解交织，译码过程非常复杂

- 有地板(floor)效应

  当信噪比较大后，误码率随信噪比下降速度开始放缓

- 理论分析困难

  至今尚未有对Turbo码编译码方案的完整严格的理论分析，在研究中多采用仿真分析的方法

- 灵活性较差

  码长受限于交织器的大小，码率受限于成员编码器的码率与删余器的删除图案。

- 并行译码困难，译码时延大

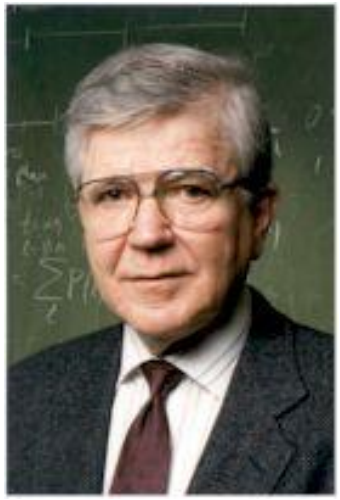
  逐个分支走网格图，难以并行译码，译码时延较大

信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

低密度奇偶校验码（LDPC码）

- 1962年Gallager首先提出LDPC码，故LDPC码亦称为Gallager 码，但由于相关理论的欠缺和当时数字信号处理能力的制约，很快沉寂下去。
- 在此后的约二十年，LDPC码几乎为人们所遗忘。



R. G. Gallager

LDPC码

- 1981年Tanner从图的观点提供了一种对LDPC码的全新解释，但他的工作又被忽视了十四年。

R.M,Tanner

Mackay

- 随着数字信号处理能力的提高和相关理论的发展完善，受Turbo提出的鼓舞，1995年Mackay和Neal重新提出LDPC码，并证明LDPC码在采用BP迭代译码时，具有逼近Shannon限的性能，是极为优秀的信道编码。

LDPC码名字的由来

- 我们熟悉的奇偶校验码是在k个二进制信息位后加上一个奇偶监督位，使得整个码字的码重（'1'的个数）恒为奇数（或恒为偶数）。严格上应称其为单奇偶校验码（Single Parity Check，SPC）。
- 低密度奇偶校验码（Low-density parity-check，LDPC）显然也属于奇偶校验码，但其校验比特不止一个。
- 由于其校验矩阵具有稀疏特性（低密度），因故得名。

LDPC码校验矩阵

$$
\begin{bmatrix}
1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1
\end{bmatrix}
$$

稀疏特性

$$
\begin{bmatrix}
1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\
1 & & & & 1 & & & & 1 & & & & 1 & & & & & & & \\
 & 1 & & & & 1 & & & & 1 & & & & & & & & & & \\
 & & 1 & & & & 1 & & & & & & & 1 & & & & 1 & & \\
 & & & 1 & & & & & & & 1 & & & & 1 & & & & 1 & \\
 & & & & & & & 1 & & & & 1 & & & & 1 & & & & 1 \\
1 & & & & & 1 & & & & & & 1 & & & & & & 1 & & \\
 & 1 & & & & & 1 & & & & 1 & & & & & 1 & & & & \\
 & & 1 & & & & & 1 & & & & & 1 & & & & & & 1 & \\
 & & & 1 & & & & & 1 & & & & & 1 & & & 1 & & & \\
 & & & & 1 & & & & & 1 & & & & & 1 & & & & & 1
\end{bmatrix}
$$

LDPC码校验矩阵及含义

H=

c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}	c_{13}	c_{14}	c_{15}	c_{16}	c_{17}	c_{18}	c_{19}	c_{20}
1	0	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	0	1	0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	1	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	1	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	1	0	0	1
0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	1
0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	0	1	0

$$c_1 \oplus c_3 \oplus c_8 \oplus c_9 = 0 \qquad c_5 \oplus c_6 \oplus c_7 \oplus c_{10} = 0 \qquad c_1 \oplus c_4 \oplus c_5 \oplus c_{11} = 0$$

校验矩阵每行代表一个校验关系（约束关系）

低密度奇偶校验码LDPC码

- 定义：
  - 校验矩阵：稀疏性，M 行 $\times$ N 列；
  - 每列包含j 个元素1；
  - 每行包含k 个元素1；
  - 码长为N 比特
  - LDPC码可记为 (N, j, k)
  - 码率

$$R = 1 - \frac{M}{N} = 1 - \frac{j}{k}$$

例如 (20, 3, 4)

$$R = 1 - \frac{15}{20} = 1 - \frac{3}{4} = \frac{1}{4}$$

LDPC码的图形表示

- 校验矩阵与Tanner图的对应关系:
  - 校验矩阵中的列—>比特节点(variable node)
  - 校验矩阵中的行—>校验节点(check node)
  - 校验矩阵中的非零元素—>边
  - 校验矩阵列重—>比特节点度(d_v)
  - 校验矩阵行重—>校验节点度(d_c)

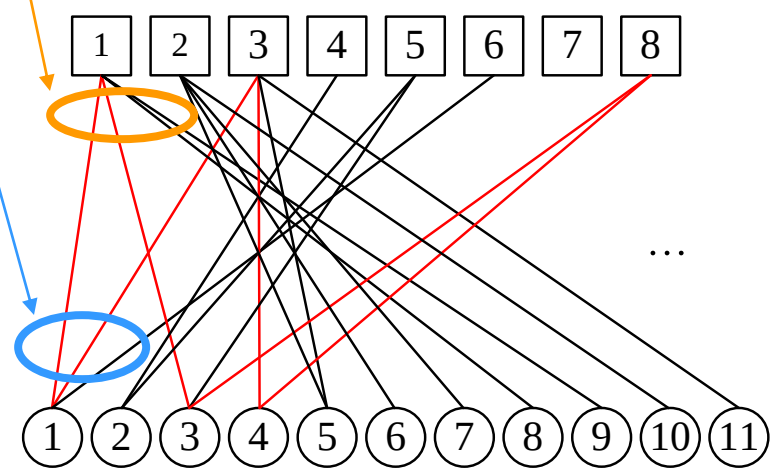
Tanner提出可以用图表示LDPC码

d_v=3, d_c=4, N=20, k=5

LDPC码的图形表示

N为码长

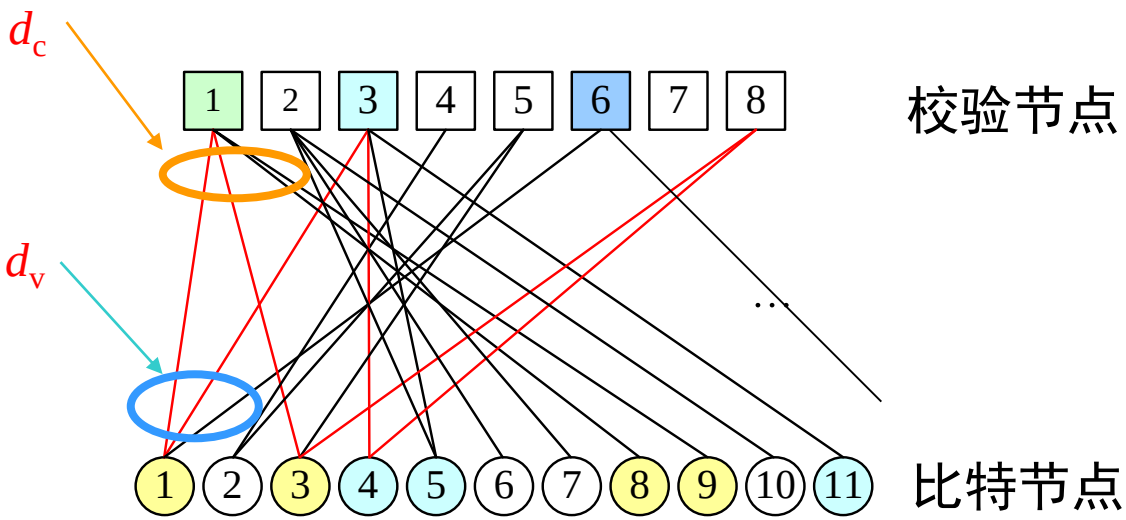
k为信息比特数



校验节点

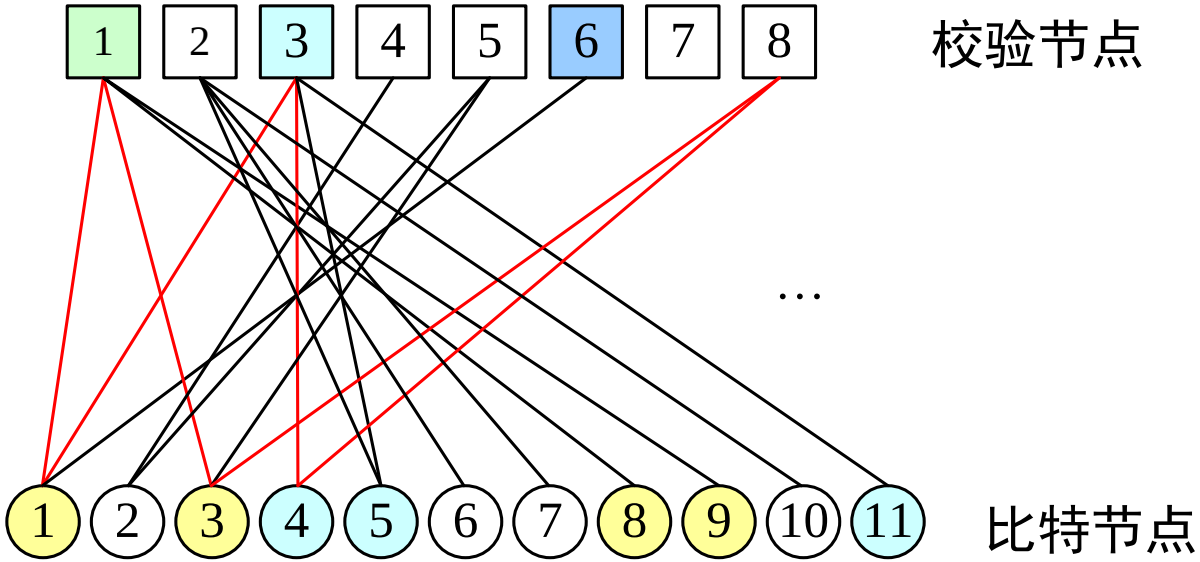
比特节点

$$H=\begin{array}{c}
\begin{array}{cccccccccccccccccccc}
c_1 & c_2 & c_3 & c_4 & c_5 & c_6 & c_7 & c_8 & c_9 & c_{10} & c_{11} & c_{12} & c_{13} & c_{14} & c_{15} & c_{16} & c_{17} & c_{18} & c_{19} & c_{20}
\end{array} \\
\begin{bmatrix}
1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0
\end{bmatrix}
\end{array}$$



环(cycle)与围长(girth)

- 在图中，从某一节点出发经过若干条不重复的边回到该节点，其轨迹称为环，经过的边数称为环长，而最小环长被称为围长。
- 围长在很大程度上决定着LDPC码的性能，一般而言围长越长，纠错性能越好，译码收敛越快。
- 在LDPC设计中，围长一般为6～10。更大的围长对性能的改善基本可忽略不计，而且增大了设计的难度，甚至根本不可实现。



正则码与非正则码

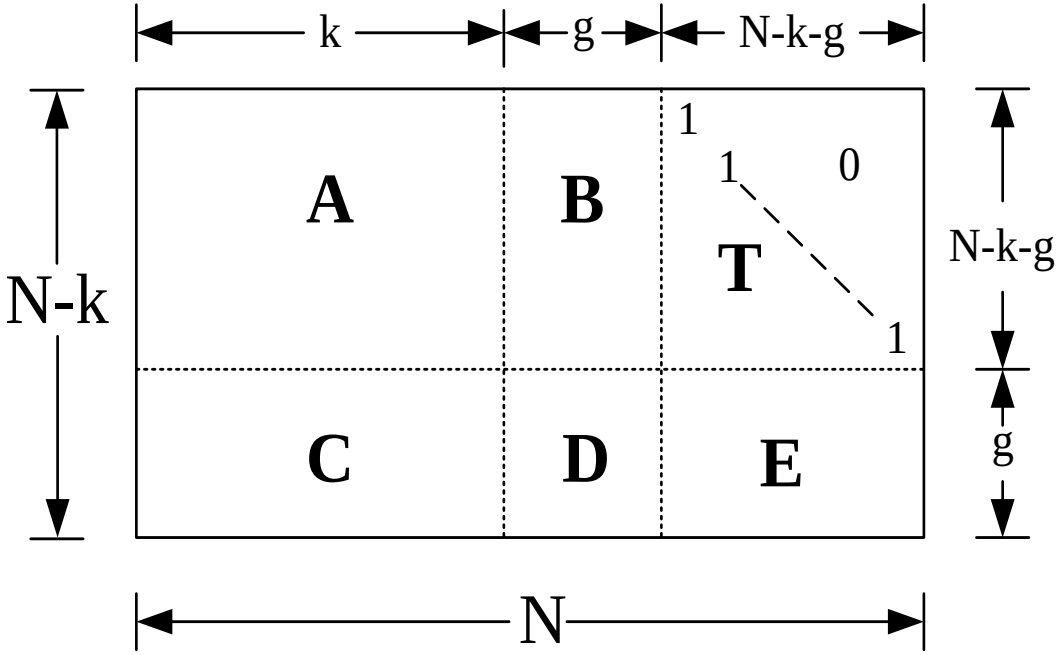
- 正则码的各行（列）的重量相同，否则为非正则码。
- 最佳非正则码的纠错性能要优于最佳正则码（波浪效应），但编译码复杂度和描述与研究的难度也较大。

利用生成矩阵进行编码

- LDPC码是线性分组码的一种，最容易想到的编码方式是对校验矩阵**H**进行行变换，得到 $\mathbf{H}=\left[\mathbf{P}\middle|\mathbf{I}_{N-k}\right]$，则生成矩阵 $\mathbf{G}=\left[\mathbf{I}_k\middle|\mathbf{P}^T\right]$
- 利用生成矩阵**G**进行编码，即码字$\boldsymbol{C}=\boldsymbol{s}\mathbf{G}$。
- 该方法在对**H**进行行变换时会破坏矩阵的稀疏性。该方法的编码复杂度为$O(N^2)$，当码长N较大（N>100）时，复杂度很大。
- 采用Richardson编码方法，实现准线性编码复杂度。
- 基于差分编码的LDPC码，实现线性编码复杂度。
- 构造QC-LDPC码，即具有循环特性的LDPC码，利用移位寄存器进行编码，虽然不降低复杂度，但能提升编码速度。

Richardson编码方法

- 减少编码复杂度的关键是尽可能地保持矩阵的稀疏性。Richardson提出通过对**H**矩阵进行**行列**交换，使得**H**矩阵右上角成为一个尽量大的下三角子矩阵。



Richardson编码方法

- 将码字$\boldsymbol{C}$分为三部分，$\boldsymbol{C}$=（$\boldsymbol{s}, \boldsymbol{p}_1, \boldsymbol{p}_2$），其中$\boldsymbol{s}$为信息比特序列，长度为$k$，$\boldsymbol{p}_1$和$\boldsymbol{p}_2$均为校验比特，长度分别为g，N-k-g。

则

$$\boldsymbol{p}_1^{\mathrm{T}} = -\boldsymbol{\Phi}^{-1}\left(-\boldsymbol{E}\boldsymbol{T}^{-1}\boldsymbol{A} + \boldsymbol{C}\right)\boldsymbol{s}^{\mathrm{T}}$$

$$\boldsymbol{p}_2^{\mathrm{T}} = -\boldsymbol{T}^{-1}\left(\boldsymbol{A}\boldsymbol{s}^{\mathrm{T}} + \boldsymbol{B}\boldsymbol{p}_1^{\mathrm{T}}\right)$$

其中 $\boldsymbol{\Phi} = -\boldsymbol{E}\boldsymbol{T}^{-1}\boldsymbol{B} + \boldsymbol{D}$

显然 $\left|\boldsymbol{\Phi}\right| \neq 0$

- Richardson方法尽可能地保持**H**矩阵的稀疏性，因此有效地降低了编码复杂度，其复杂度为O(N＋g^2)。而减小g是进一步降低复杂度的关键，也就是说右上角的下三角矩阵越大，编码复杂度越小。

编码实例

H=

c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}	c_{13}	c_{14}	c_{15}	c_{16}	c_{17}	c_{18}	c_{19}	c_{20}
1	0	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	0	1	0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	1	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	1	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	1	0	0	1
0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	1
0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	0	1	0

编码实例

- 该校验矩阵已经进行了**行列交换**，g=3

检验可得 $|\Phi| \neq 0$，可进行编码。

当$\boldsymbol{s}$=(1,0,0,0,0)，

可得

$\boldsymbol{p}_1$=(1,1,0),

$\boldsymbol{p}_2$=(1,0,1,1,1,1,0,0,0,0,1,1)。

因此

$\boldsymbol{C}$=（$\boldsymbol{s}, \boldsymbol{p}_1, \boldsymbol{p}_2$）=(1,0,0,0,0,1,1,0,1,0,1,1,1,1,0,0,0,0,1,1)

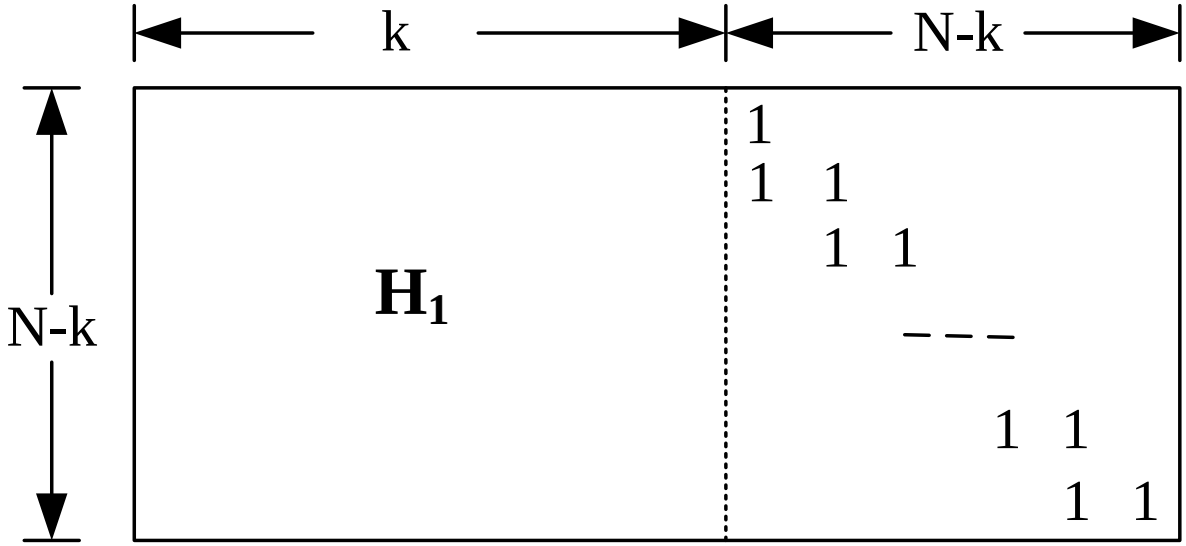
校验 $H \cdot C^{\mathrm{T}} = 0$,编码结束。

可差分编码的LDPC码

- Richardson方法适用于所有的LDPC码，但进行编码前需进行行列交换，而且其复杂度并不是严格随码长N线性增加（ O（N＋g^2) ）。
- RA（repeat accumulated) LDPC码具有特殊的结构，因此可采用差分编码方式。其编码复杂度较低，而且严格随码长线性增加。

可差分编码的LDPC码

RA码的校验矩阵如下：



信息比特序列为$\boldsymbol{s}$，校验比特 $\boldsymbol{p}=(p_1\ p_2\ \ldots\ p_{N-k})$，差分编码如下:

由 $HC^T=0$ 得： $p_1=\boldsymbol{s}\cdot\boldsymbol{H}_1(1,:)$

$$p_i=\boldsymbol{s}\cdot\boldsymbol{H}_1(i,:)+p_{i-1}\quad 2\le i\le N-k$$

准循环LDPC码（QC-LDPC）

- 实际系统中使用的LDPC码大多数是长码，存储校验矩阵需要占用较大的存储容量，而且编码器实现难度也较大
- 准循环LDPC（Quasi-Cyclic Low-Density Parity-Check Codes，QC-LDPC）码是一类具有特殊结构的LDPC码，其校验矩阵**H**由$m\times n$个子矩阵（称为标准置换矩阵）$\mathbf{H}_{1,1}$、$\mathbf{H}_{1,2}$、…$\mathbf{H}_{1,n}$、…、$\mathbf{H}_{m,n}$构成

$$\mathbf{H}=\begin{bmatrix}\mathbf{H}_{1,1} & \mathbf{H}_{1,2} & \cdots & \mathbf{H}_{1,n}\\ \mathbf{H}_{2,1} & \mathbf{H}_{2,2} & \cdots & \mathbf{H}_{2,n}\\ \vdots & \vdots & \ddots & \vdots\\ \mathbf{H}_{m,1} & \mathbf{H}_{m,2} & \cdots & \mathbf{H}_{m,n}\end{bmatrix}$$

准循环LDPC码（QC-LDPC）

- 标准置换矩阵$\mathbf{H}_{i,j}$, i=1…m, j=1…n为$Z_c \times Z_c$的方阵，是由基矩阵$\mathbf{H}_{BG}$基础上扩展而来，即将$\mathbf{H}_{BG}$矩阵中的'0'扩展为全零矩阵，'1'扩展为单位矩阵循环右移$P_{i,j}$位的矩阵

$$\mathbf{I}_1 = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \end{bmatrix}_{Z_c \times Z_c}$$

单位矩阵循环右移$P_{i,j} = 1$位得到的标准置换矩阵$\mathbf{I}_1$

- 优点：

➢根据基矩阵$\mathbf{H}_{BG}$及$P_{i,j}$的值，可以完全确定校验矩阵，降低了存储占用

➢可以利用循环移位的特性，不仅降低了编译码器硬件实现的难度，而且可进行并行编译码，提升了编译码器的吞吐量

➢准循环LDPC在5G等系统中得到了广泛的应用

LDPC码的译码方式

LDPC码的译码方式主要包括有:

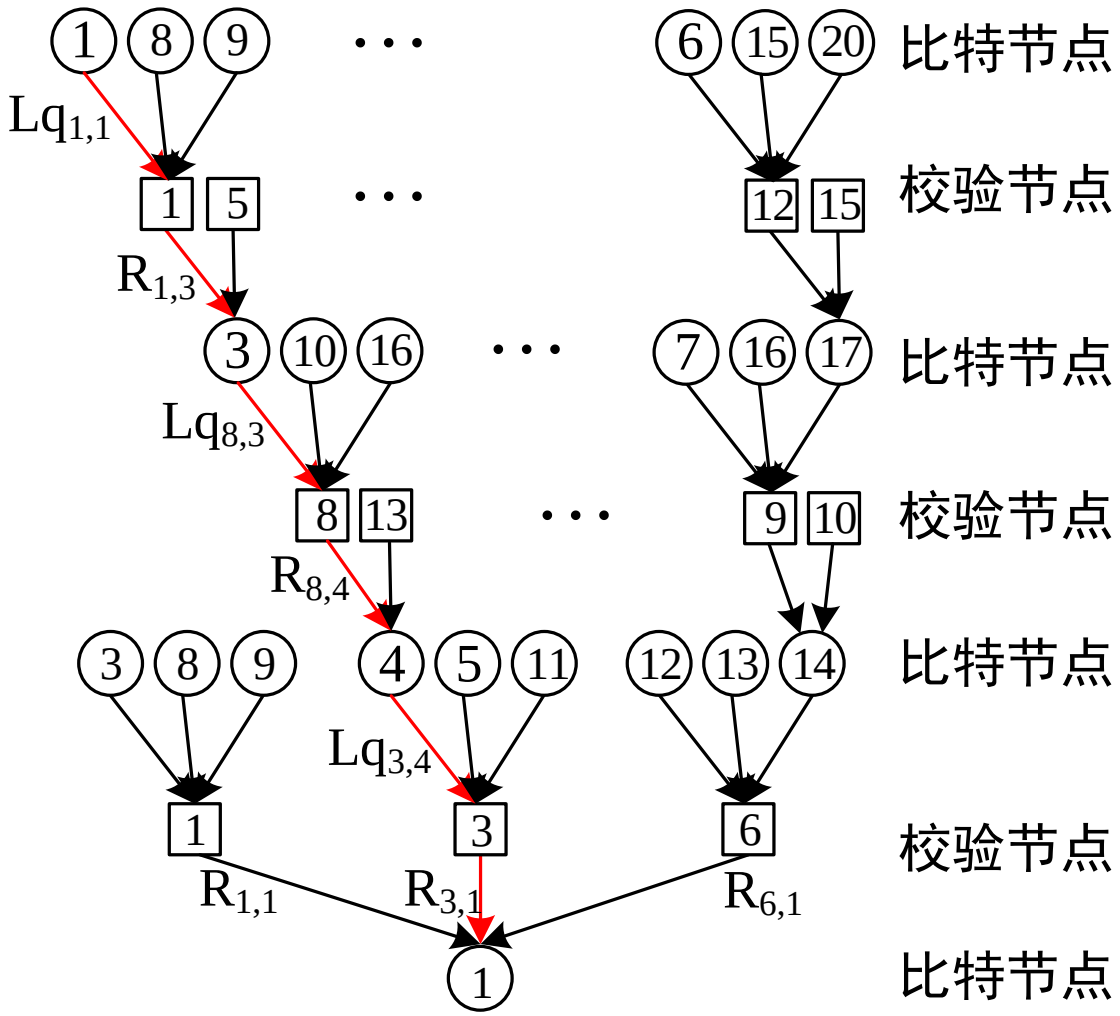
- Gallager方法
- 信度传播（BP: Belief Propagation）算法，

  也称和积算法（SPA: Sum-Product algorithm)
- 比特翻转（BF：Bit Flipping）
- 密度演进（Density evolution）算法等

其中最有代表性的是BP算法和BF算法。

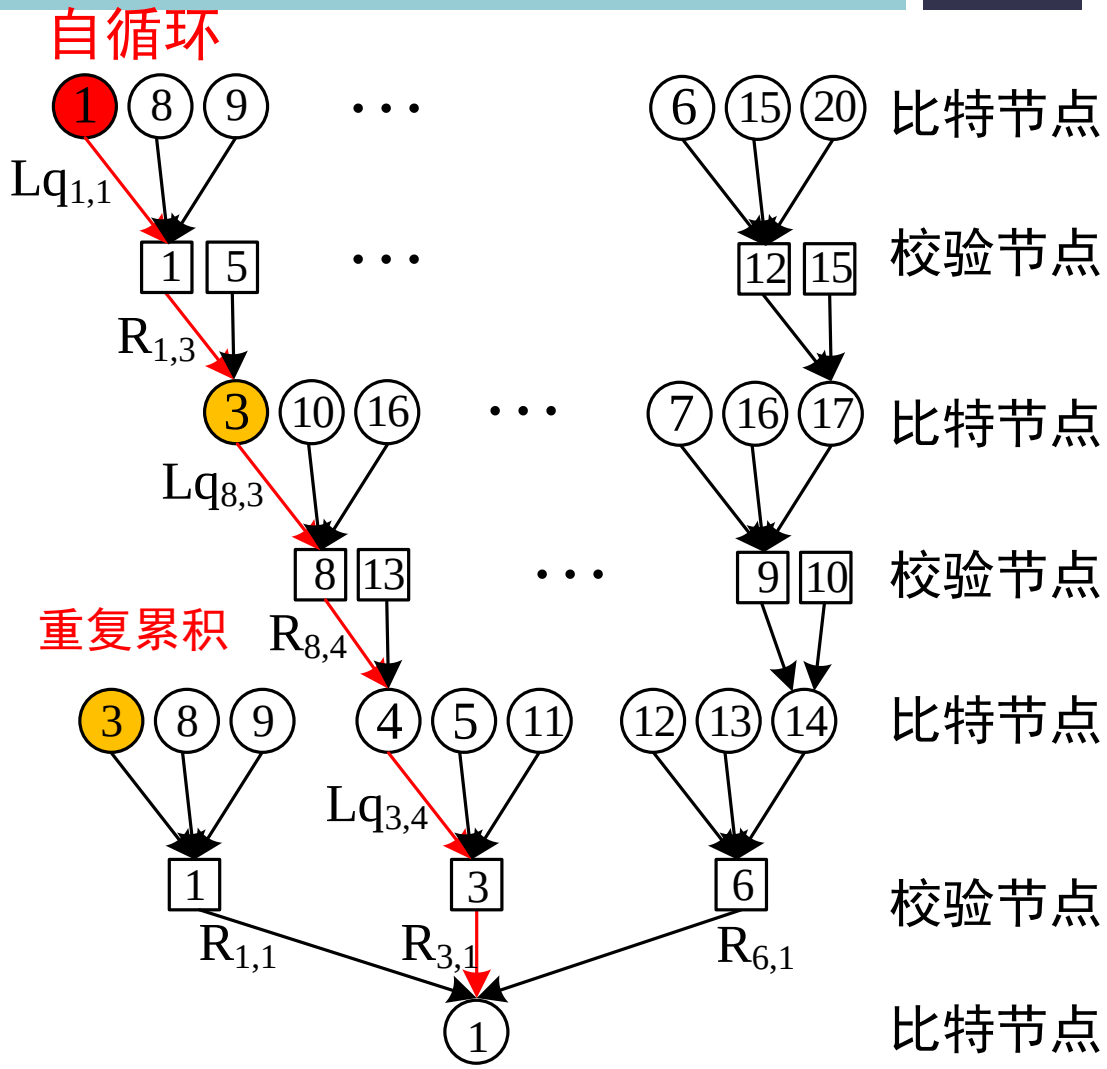
BP算法简介

- 比特节点进行译码时，其外信息来自与之相连的所有校验节点
- 校验节点所提供的外信息来自与之相连的所有比特节点（接收信息的比特节点除外）
- 比特节点传递给校验节点的信息来源于其自信息和与之相连的校验节点提供的外信息（接收信息的校验节点除外）



BP算法简介

- 每经过一次迭代，比特节点能够从中获得信息的比特节点就越多
- Tanner图全联通时，经过足够多次迭代后，比特节点可以从所有其他的比特节点处获得信息，其自身的信息也会传递给所有其他比特节点
- 由于Tanner图中存在环，因此存在信息的自循环和重复累积



BP算法具体步骤

1）初始化

比特节点j传递给校验节点i的信息 $\lambda_{i,j}^{(0)} = \dfrac{2r_j}{\sigma^2}$,

2）校验节点更新（更新校验节点传递给比特节点的信息）

第l轮迭代

$$\mu_{i,j}^{(l)} = -\prod_{k\in A(i)\setminus j} \operatorname{sgn}(\lambda_{i,k}^{(l-1)}) \cdot \Phi\Big(\sum_{k\in A(i)\setminus j} \Phi\big(\big|\lambda_{i,k}^{(l-1)}\big|\big)\Big)$$

$A(i)$表示与校验节点i相连的比特节点集合　　不包括比特节点j

3）比特节点更新（更新比特节点传递给校验节点的信息）

$$\lambda_{i,j}^{(l)} = \frac{2r_j}{\sigma^2} + \sum_{m\in B(j)\setminus i} \mu_{m,j}^{(l)}$$

$B(j)$表示与比特节点j相连的校验节点集合　　不包括校验节点i

BP算法具体步骤

4）译码判决

第j个比特（节点）的全部信息 $L\left(\hat{c}_j\right)=\dfrac{2r_j}{\sigma^2}+\sum\limits_{m\in B(j)}\mu_{m,j}^{(l)}$

根据各比特节点的后验LLR进行判决，得到码字序列

$$\hat{C}=\left[\hat{c}_j\right], j=1\cdots N$$

$$\hat{c}_j=\begin{cases}0 & L\left(\hat{c}_j\right)<0\\ 1 & L\left(\hat{c}_j\right)\geq 0\end{cases}$$

5）校验

如果 $\mathbf{H}\cdot\hat{C}^T=\mathbf{0}$，则表示译码成功，结束译码；

否则，进一步判断是否达到允许的最大迭代次数，如尚未到达，则回到步骤2，开始下一轮迭代，否则宣布译码失败，结束译码。

BP改进算法——最小BP算法

对复杂度最高的校验节点更新进行近似计算

$$\mu_{i,j}^{(l)} = -\prod_{k\in A(i)-j} \operatorname{sgn}(\lambda_{i,k}^{(l-1)})\cdot\Phi(\sum_{k\in A(i)-j}\Phi(\left|\lambda_{i,k}^{(l-1)}\right|))$$

$$\approx \prod_{k\in A(i)-j} \operatorname{sgn}(\lambda_{i,k}^{(l-1)})\cdot\min\left(\left|\lambda_{i,k}^{(l-1)}\right|\right)$$

从而避免了指数和对数运算。然而，由于

$$\min_{k\in A(i)\setminus j}\left(\left|\lambda_{i,k}^{(l-1)}\right|\right) > -\Phi\left(\sum_{k\in A(i)\setminus j}\Phi\left(\left|\lambda_{i,k}^{(l-1)}\right|\right)\right)$$

即采用最小BP算法得到的存在一定的误差，这导致LDPC码性能一定的恶化。

归一化BP算法和偏移BP算法

归一化BP算法：除以α（一般取1.2~1.3）

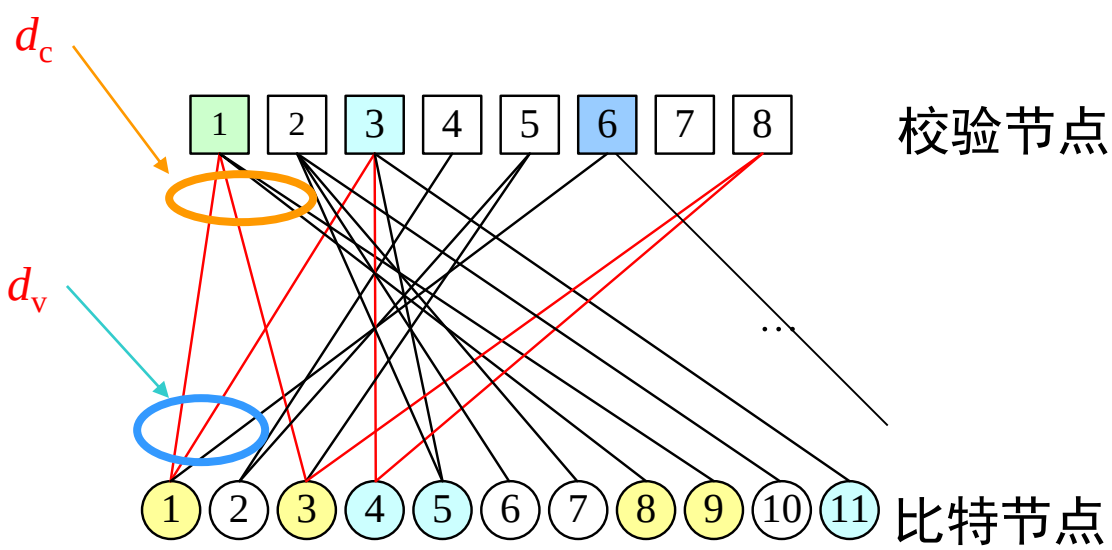
$$\mu_{i,j}^{(l)} \approx \prod_{k\in A(i)\setminus j} \operatorname{sgn}\left(\lambda_{i,k}^{(l-1)}\right)\cdot \min_{k\in A(i)\setminus j}\left(\left|\lambda_{i,k}^{(l-1)}\right|\right)\Big/\alpha$$

偏移BP算法：减去β（一般取0.1~0.2）

$$\mu_{i,j}^{(l)} \approx \prod_{k\in A(i)\setminus j} \operatorname{sgn}\left(\lambda_{i,k}^{(l-1)}\right)\cdot\left(\min_{k\in A(i)\setminus j}\left(\left|\lambda_{i,k}^{(l-1)}\right|\right)-\beta\right)$$

LDPC码构造优化

- 避免短环，增大围长，减小BP算法中信息自反馈造成的影响
- 从校验节点的角度看，d_c度数越小越好，因为度数小，它对与之相连的信息节点提供的信息越有价值。
- 从比特节点的角度看，d_v度数越大越好，因为它从校验节点得到的信息越多，他的译码越准确。
- 合理设定d_c和d_v的分布，是构造具有良好性能的LDPC码的重要方面



当前研究热点

- LDPC码的构造
- 低复杂度LDPC译码算法
- LDPC码的性能界
- 非规则码及非二进制LDPC码
- LDPC编译码器的硬件实现

LDPC码 vs Turbo码

- 优点：
  - 相同码率和码长的长LDPC码纠错性能略优于Turbo码，而且LDPC码具有极强的检错能力。
  - LDPC码设计（码长和码率）更为灵活。
  - 译码复杂度随码长增长较慢，这使得长码的使用成为可能。
  - 译码算法（主要指BP算法）为并行算法，而且无须交织（解交织），译码时延小。
  - 描述简单，能够进行严格的理论分析。
- 缺点：
  - 编码复杂度一般大于Turbo码。
  - 校验矩阵H的存储需占用一定空间。

信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

极化码（Polar码）简介

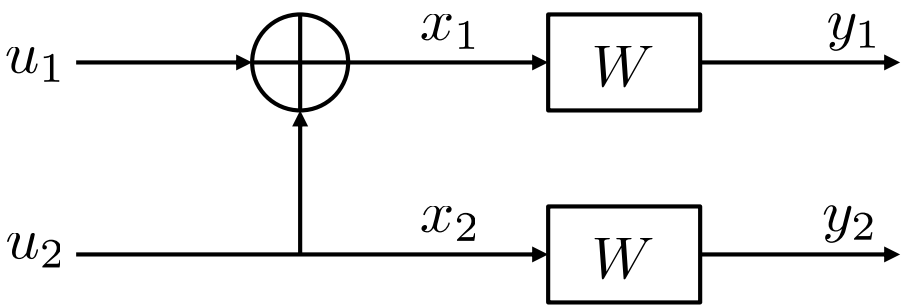
- 2008 年，土耳其的 Erdal Arikan 教授于首次提出了极化码。
- 极化码是目前唯一被严格证明可以达到香农信道容量限的信道编码。
- 极化码具有规则的编码结构、明确的构造方法和低复杂度的编译码算法，同时还具有丰富的代数结构和良好的分析特性。
- 极化码的应用：
  - 2016 年 11 月 17 日，在 3GPP RAN1 87 次会议上，华为公司主推的极化码方案成为 5G 控制信道编码方案
  - 在数字存储系统、光纤通信等领域有着广泛的应用前景。

信道极化（Channel Polarization）

- 信道极化可分为两个过程：信道合并（Channel Combining）、信道分裂（Channel Splitting）。
- 信道合并：将 $N = 2^n$ 个相互独立的二进制离散无记忆信道 (Binary-Discrete Memoryless Channel，B-DMC) 递归合并，得到新的信道组合：$W^N : X^N \to Y^N$
- 合并后的信道转移概率：$W^N\left(y_1^N \mid x_1^N\right) = \prod_{i=1}^{N} W(y \mid x)$
  - $W(y \mid x)$ 是B-DMC信道的状态转移概率，$x \in \{0, 1\}$

信道极化

- n=1, N=2 信道合并：



$$\begin{cases} x_1 = u_1 \oplus u_2 \\ x_2 = u_2 \end{cases}$$

- 合成信道的转移概率为：

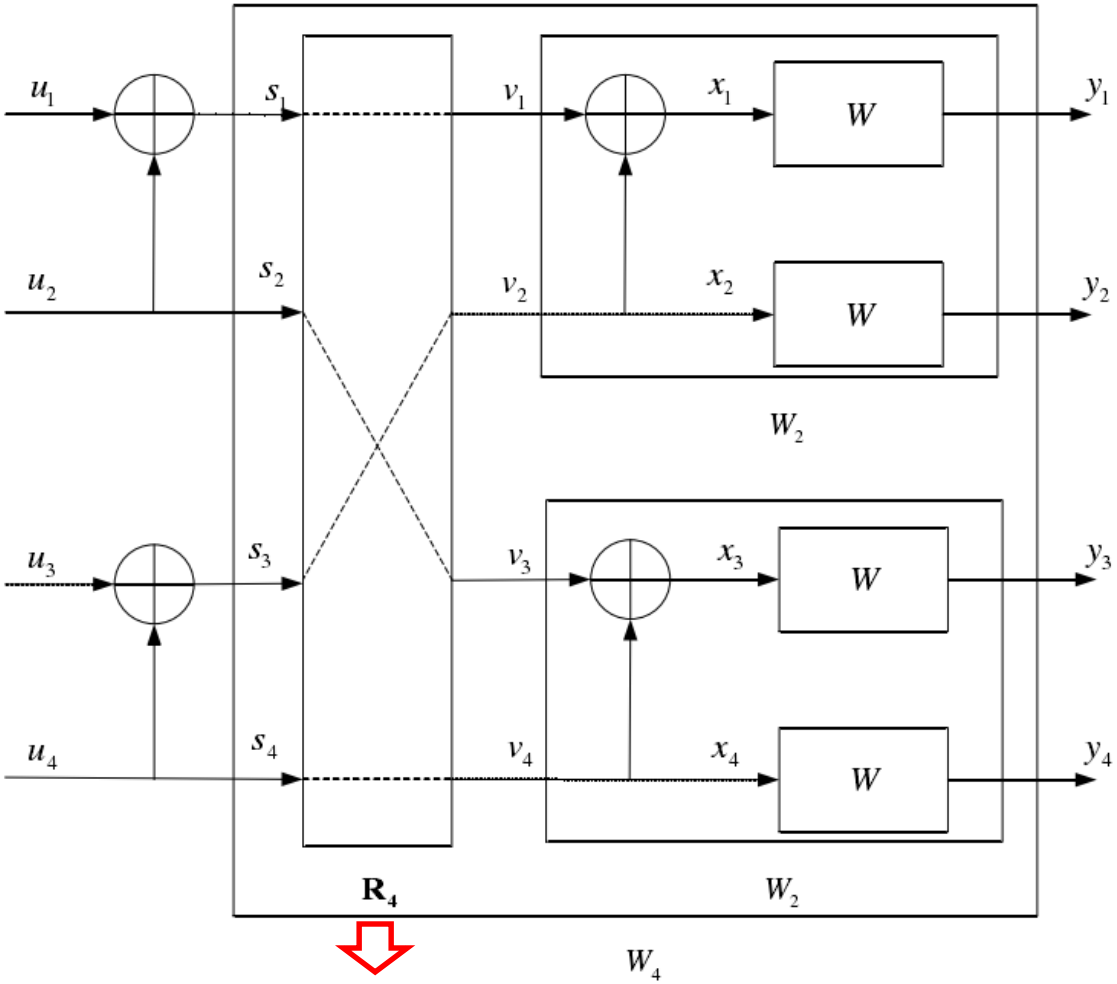
$$W_2\left(y_1, y_2 \mid u_1, u_2\right) = W\left(y_1 \mid u_1 \oplus u_2\right) W\left(y_2 \mid u_2\right)$$

- 由于合成的信道实际上是两个BEC信道独立的复用，因此拆分信道容量和为 $I\left(W^-\right) + I\left(W^+\right) = 2I(W)$
- Arikan证明：$\begin{cases} I\left(W^-\right) = I(W)^2 \\ I\left(W^+\right) = 2I(W) - I(W)^2 \end{cases}$

$$0 \le I(W) \le 1, I\left(W^-\right) \le I(W) \le I\left(W^+\right)$$

信道极化

- n=2, N=4 信道合并



比特翻转操作

信道极化

- n=2, N=4 信道合并
  - 合成信道的转移概率为：

$$W_4\left(y_1^4 \mid u_1^4\right) = W_2\left(y_1^2 \mid u_1 \oplus u_2, u_3 \oplus u_4\right) W_2\left(y_3^4 \mid u_2, u_4\right)$$

$$\begin{cases} x_1 = u_1 \oplus u_2 \oplus u_3 \oplus u_4 \\ x_2 = u_3 \oplus u_4 \\ x_3 = u_2 \oplus u_4 \\ x_4 = u_4 \end{cases} \Rightarrow \quad W_4\left(y_1^4 \mid u_1^4\right) = W^4\left(y_1^4 \mid u_1^4 \boldsymbol{G}_4\right)$$

$$\boldsymbol{G}_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

信道极化

- 各子信道并不是相互独立的。信道索引值越大的极化子信道，包含的信息越丰富，信道容量越大，信道可靠性越高。
- 信道索引值大的极化子信道变成信道容量为1的完美信道，而索引值小的退化为信道容量为 0 的信道，接近纯噪声信道。
- 已有研究证明：码长趋于无穷大时，信道容量趋近于 1 的极化信道数量约为 $N \times I(W)$ 个，而信道容量趋近于 0 的极化信道数目约为$N \times (1-I(W))$个。因此可达到信道容量。

极化码编码

- 码长$N=2^n$，信息序列 $\boldsymbol{u}_1^N=(u_0, u_1, \ldots, u_N)$，码字序列：

$$\boldsymbol{x}_1^N=\boldsymbol{u}_1^N\boldsymbol{G}_N$$

生成矩阵 $\boldsymbol{G}_N=\underline{\boldsymbol{B}_N}\,\underline{\boldsymbol{F}^{\otimes n}}$ ⇨ 对 $\boldsymbol{F}=\begin{bmatrix}1 & 0\\ 1 & 1\end{bmatrix}$ 做n次克罗内克积运算

⇩

比特翻转操作　　即 $\boldsymbol{F}^{\otimes n}=\boldsymbol{F}\otimes\boldsymbol{F}^{\otimes(n-1)}$

克罗内克积（Kronecker Product）运算：

$$\boldsymbol{A}\otimes\boldsymbol{B}=\begin{pmatrix}A_{11}\boldsymbol{B} & \cdots & A_{1n}\boldsymbol{B}\\ \vdots & \ddots & \vdots\\ A_{m1}\boldsymbol{B} & \cdots & A_{mn}\boldsymbol{B}\end{pmatrix}\quad \boldsymbol{A}=\begin{pmatrix}A_{11} & \cdots & A_{1n}\\ \vdots & \ddots & \vdots\\ A_{m1} & \cdots & A_{mn}\end{pmatrix}$$

极化码编码

- 步骤1：计算巴氏参数，选取信息位
  - 巴氏参数$Z(W)$也称为Z参数，用于表征系统的可靠度

$$Z(W) \triangleq \sum_{y \in Y} \sqrt{W(y \mid 0) W(y \mid 1)}$$

  - 物理意义：在信道W上，等概率发送0和1，采用最大似然判决时的错误概率上限。当$Z(W) \approx 0$时，$I(W) \approx 1$；当$Z(W) \approx 1$时，$I(W) \approx 0$。
  - 巴氏参数的递归计算：$W_1^{(1)}$为BEC信道删除概率ε，

$$Z\left(W_N^{(2i)}\right) = Z\left(W_{N/2}^{(i)}\right)^2$$

$$Z\left(W_N^{(2i-1)}\right) = 2Z\left(W_{N/2}^{(i)}\right) - Z\left(W_{N/2}^{(i)}\right)^2$$

  - BSC信道：$Z_0 = 2\sqrt{p_\varepsilon\left(1 - p_\varepsilon\right)}$
  - AWGN信道：$Z_0 = \mathrm{e}^{-\frac{1}{2\sigma^2}}$

极化码编码

- 步骤1：计算巴氏参数，选取信息位
  - 若码长N=8，信息比特K=4，BEC删除概率ε=0.5
  - 子信道1~8的巴氏参数分别为：0.9961、0.8789、0.8086、0.3164、0.6836、0.1914、0.1211、0.0039。
  - 由于信息位共4位，因此选取巴氏参数较小的4个子信道，即第4、6、7、8号子信道用来传输信息比特；其余巴氏参数较大的4个子信道，即第1、2、3、5号子信道，用来传输冻结比特。

极化码编码

- 步骤2：计算生成矩阵
  - 若码长N=8，则n=3

$$\boldsymbol{F}^{\otimes 3} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

极化码编码

步骤3：计算B_N，进行比特翻转操作

（1）8个子信道的索引值为(1, 2, 3, 4, 5, 6, 7, 8)。

（2）将索引值减1后转换为3位二进制数（000，001，010，011，100，101, 110, 111)。

（3）将二进制索引值进行比特翻转，得到(000，100，010，110，001，101, 011, 111)。

（4）将翻转后的二进制索引值转换为十进制后加1，得到最终的索引值 (1, 5, 3, 7, 2, 6, 4, 8)。

状态	索引值							
比特翻转前	1	2	3	4	5	6	7	8
比特翻转后	1	5	3	7	2	6	4	8

极化码编码

- 步骤3：计算$\boldsymbol{B}_N$，进行比特翻转操作
  - 根据比特翻转前后的索引值,将$F^{\otimes 3}$矩阵第2行和第5行互换，第4行和第7行互换，其余四行不变，得到生成矩阵：

$$\boldsymbol{G}_N = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

极化码编码

- 步骤4：计算编码序列，完成编码
  - 假设信息序列为(1, 1, 1, 1)，将其放到信息位上，加上其他4个全0冻结比特，得8位信息序列为(0, 0, 0, 1, 0, 1, 1, 1)
  - 编码用生成矩阵第4，6，7，8行：

$$\boldsymbol{C} = (1,1,1,1) \times \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} = (0,1,1,0,1,0,0,1)$$

极化码译码

- 极化码支持多种译码算法，包括Arikan最早提出的串行消除（Successive Cancellation，SC）算法，由其衍生出的SC列表（Successive Cancellation List，SCL）译码算法和基于并行结构的信度传播算法等。
- SC译码算法复杂度低，当码长趋于无穷时，可以达到香农容量限，且不存在差错平底的问题，应用较多。
- SC译码算法核心思想：连续地消除前面的消息比特对后面消息比特的干扰，从而大幅提升译码后续消息比特的可靠性。有错误比特传播问题。

极化码性能

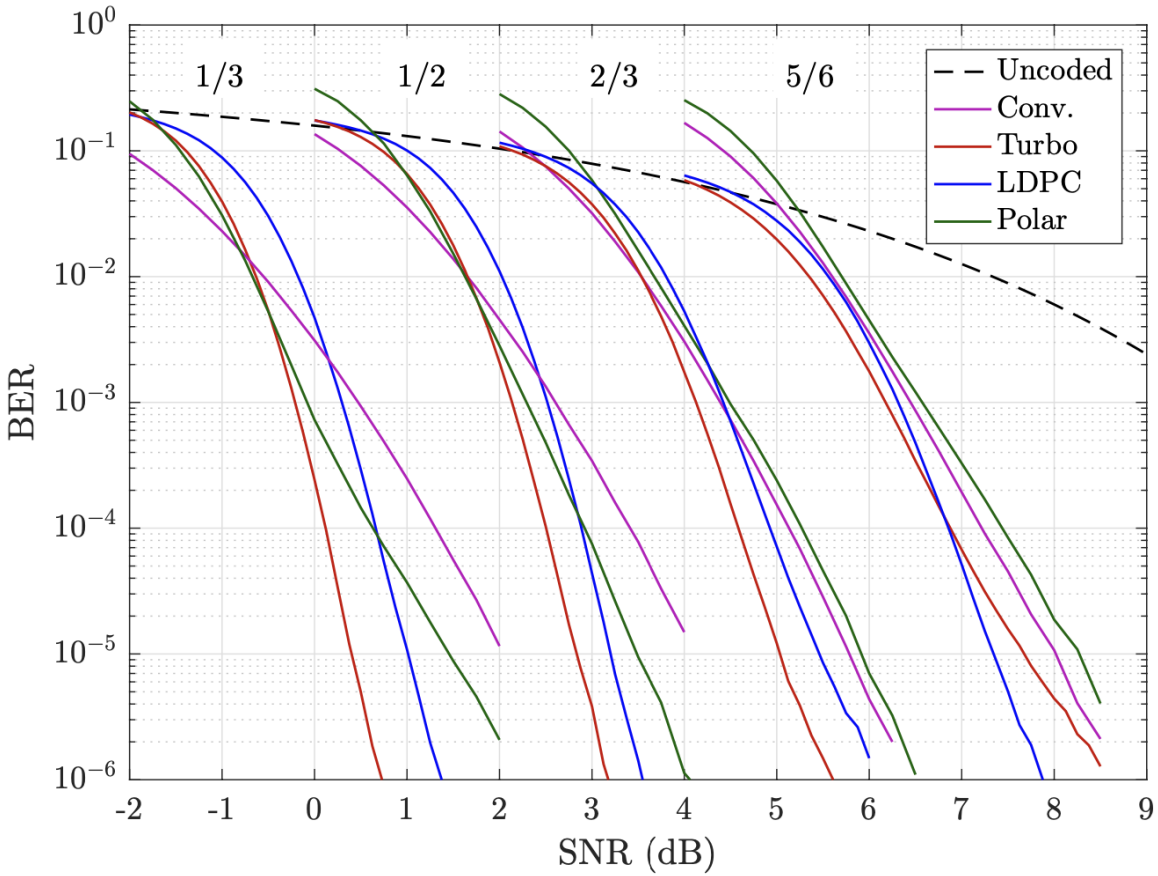
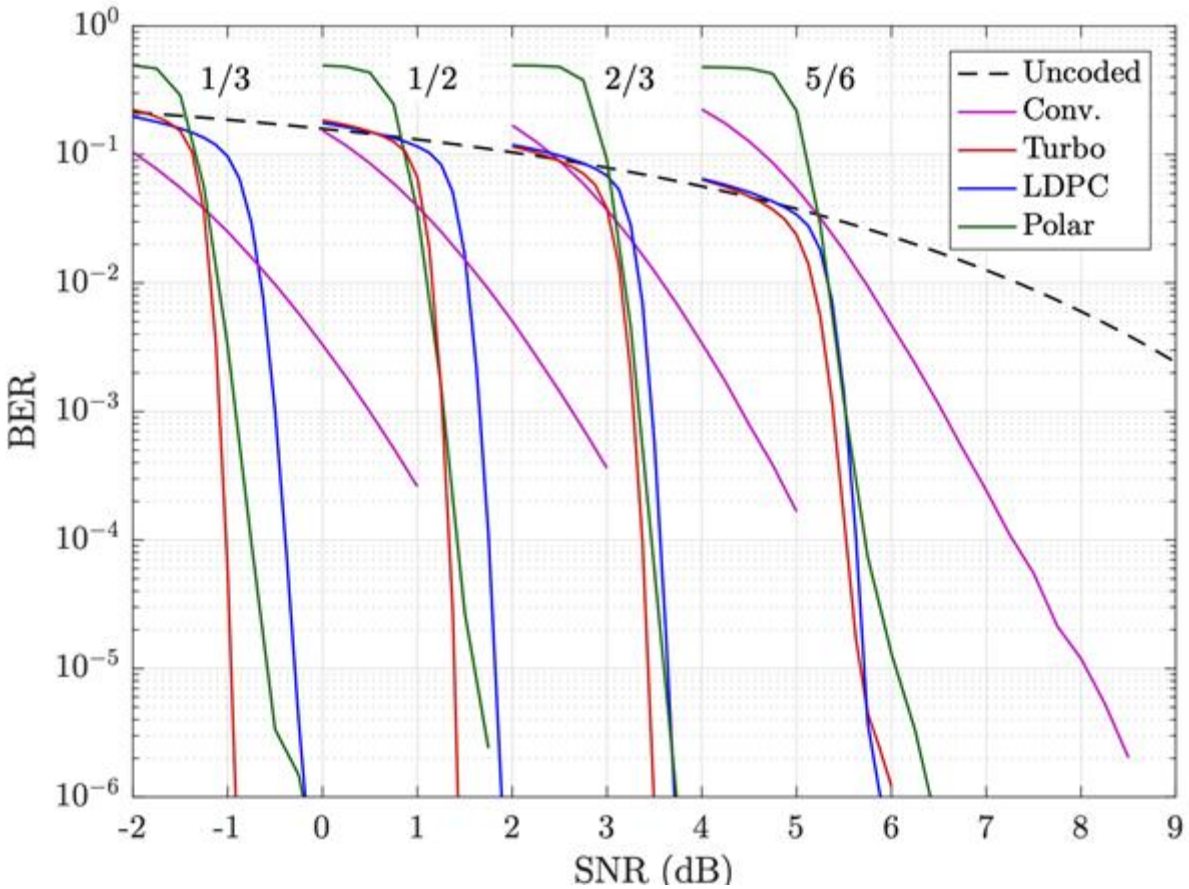


Fig. 9. BER comparison for different code rates, $K = 256$ (For LDPC, $K = 252$ for $R = 1/2$, and $1/3$, and $K = 260$ for $R = 5/6$.)

B. Tahir, S. Schwarz and M. Rupp, "BER comparison between Convolutional, Turbo, LDPC, and Polar codes," *2017 24th International Conference on Telecommunications (ICT)*, Limassol, Cyprus, 2017.

性能比较



码长为8192比特（对于LDPC码，R=1/2和1/3时,码长为8196，R=5/6时，码长为8200）

B. Tahir, S. Schwarz and M. Rupp, "BER comparison between Convolutional, Turbo, LDPC, and Polar codes," *2017 24th International Conference on Telecommunications (ICT)*, Limassol, Cyprus, 2017.

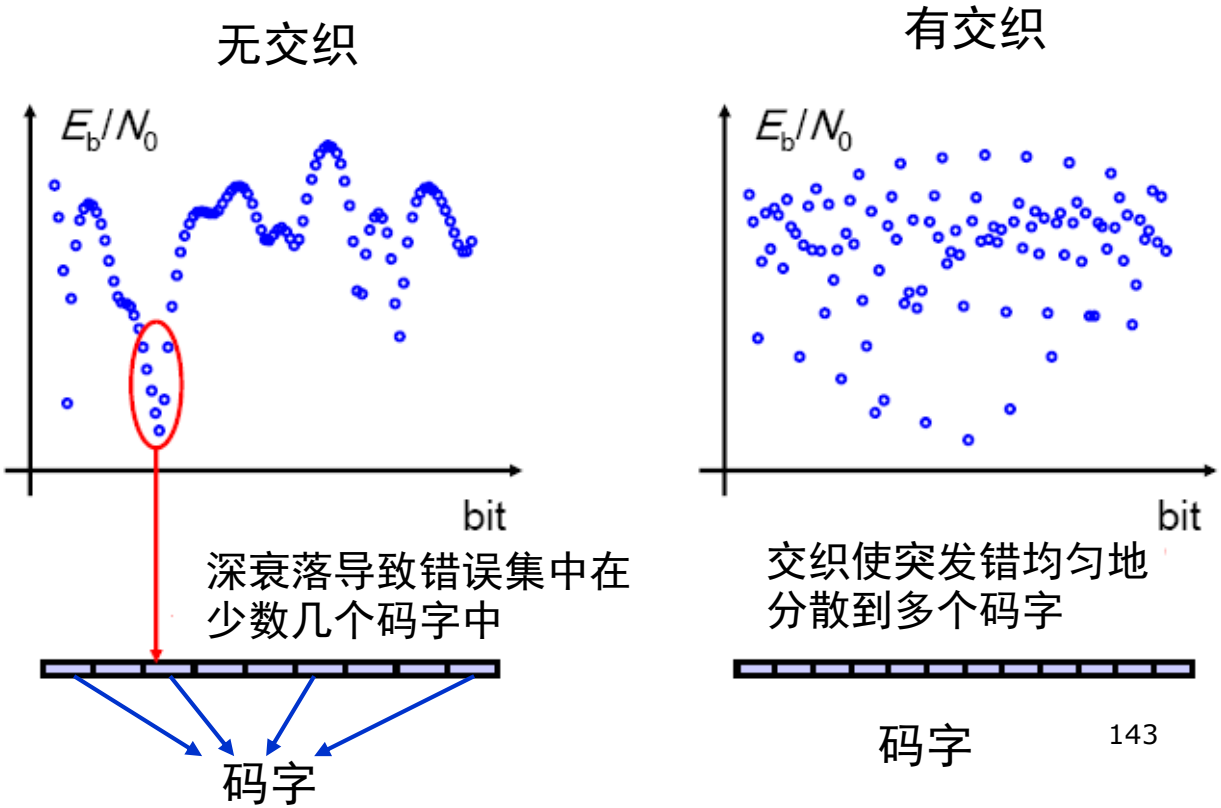
信道编码

- 概述
- 线性分组码
- 卷积码
- Turbo码
- 低密度奇偶校验码
- 极化码
- 交织与级联码

交织器（Interleaver）

- 交织是一种将数据序列的顺序进行变换的处理方法

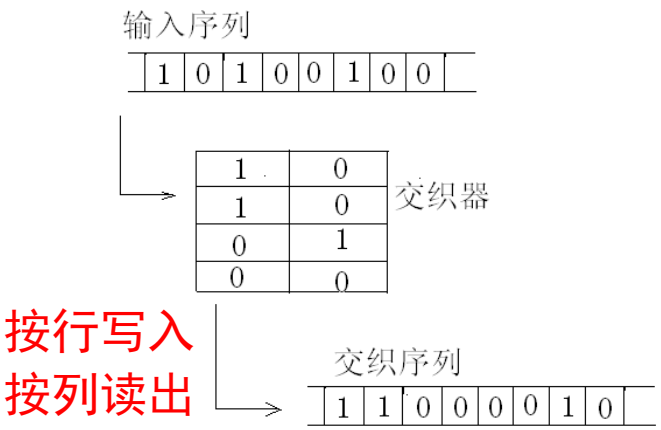
- 由于衰落、突发干扰、拥塞等原因，数据在一段时间可能发生连续错误（突发错）
- 信道编码处理突发错能力一般较差
- 交织将突发错分散成随机错误可以充分发挥信道编码的性能。

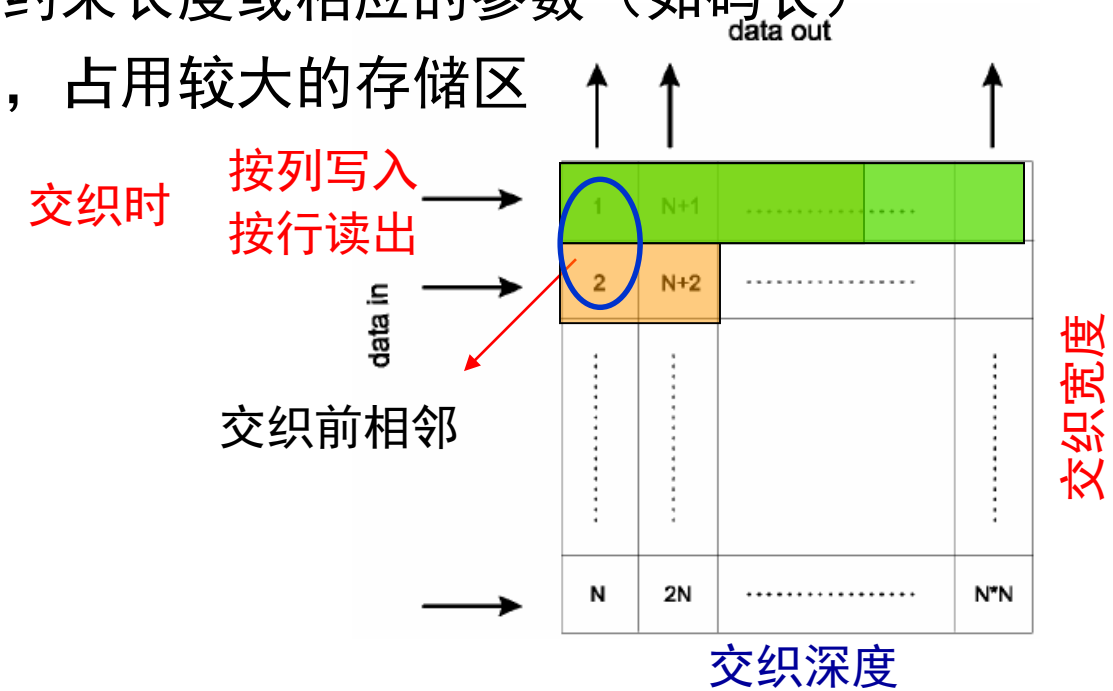


码字越短，处理突发错能力越差；而译码复杂度一般随码字指数或幂次增长

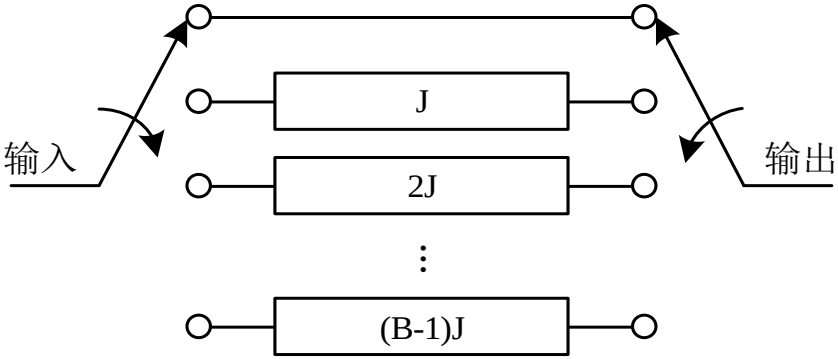
分组交织

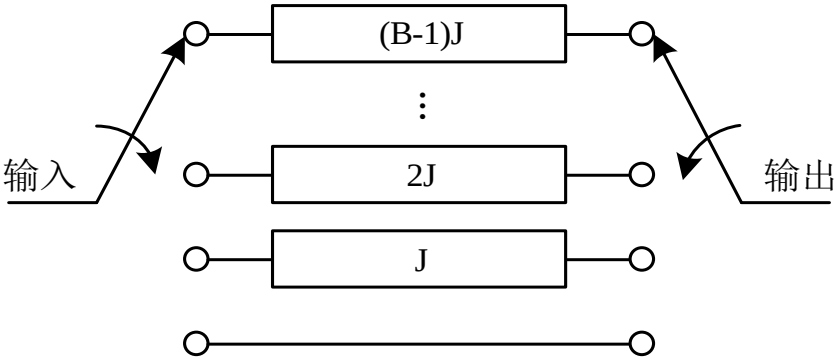
- 交织时，按列写按行读；解交织时，按行写按列读（反之亦可）
- 交织前（编码后）相邻符号在交织后的最小距离称为交织深度
- 交织后相邻的符号在交织前的最小距离称为交织宽度
- 交织深度应不小于信道上可能的突发错长度，否则解交织后仍可能存在一定的突发错误
- 交织宽度应不小于编码的约束长度或相应的参数（如码长）
- 分组交织会造成较大时延，占用较大的存储区



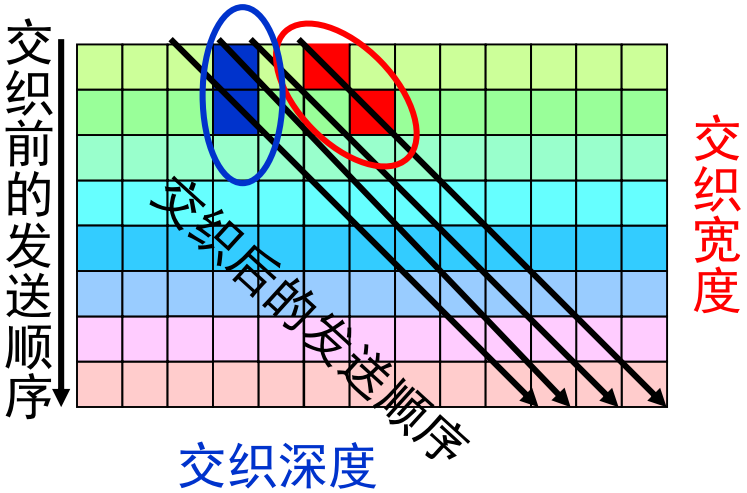


卷积交织





- B 称为交织宽度（支路），代表交织后相邻的符号在交织前的最小距离。
- BJ表示交织深度（延迟缓存器），是交织前相邻的符号在交织后的最小距离
- 交织和解交织造成最大时延为$B(B-1)J$
- 存储占用为$B(B-1)J/2$



- 卷积交织器要实现与分组交织器相同的交织深度和交织宽度（$BJ=M$，$B=N$），引起的端到端延迟和所需的存储容量，均只有分组交织器的一半，一般配合卷积码使用

交织的效果

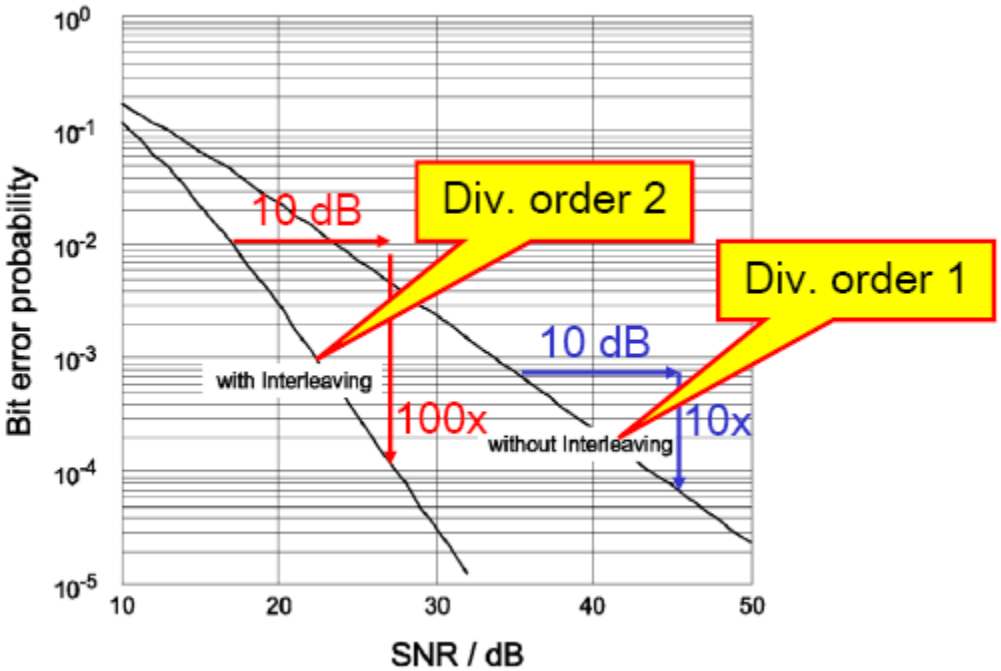
- R=1/3的重复码，采用大数判决，在平坦瑞利衰落信道
  - 无交织

$$BER \approx \int_0^\infty pdf_\gamma(\gamma) Q^2(\sqrt{\gamma}) d\gamma$$

  - 有交织

$$BER = \int_0^\infty \int_0^\infty pdf_{\gamma_1}(\gamma_1) Q(\sqrt{\gamma_1}) pdf_{\gamma_2}(\gamma_2) Q(\sqrt{\gamma_2}) d\gamma_1 d\gamma_2$$

$$= \left(\int_0^\infty pdf_\gamma(\gamma) Q(\sqrt{\gamma}) d\gamma \right)^2$$



交织具有时间分集的效果

级联码

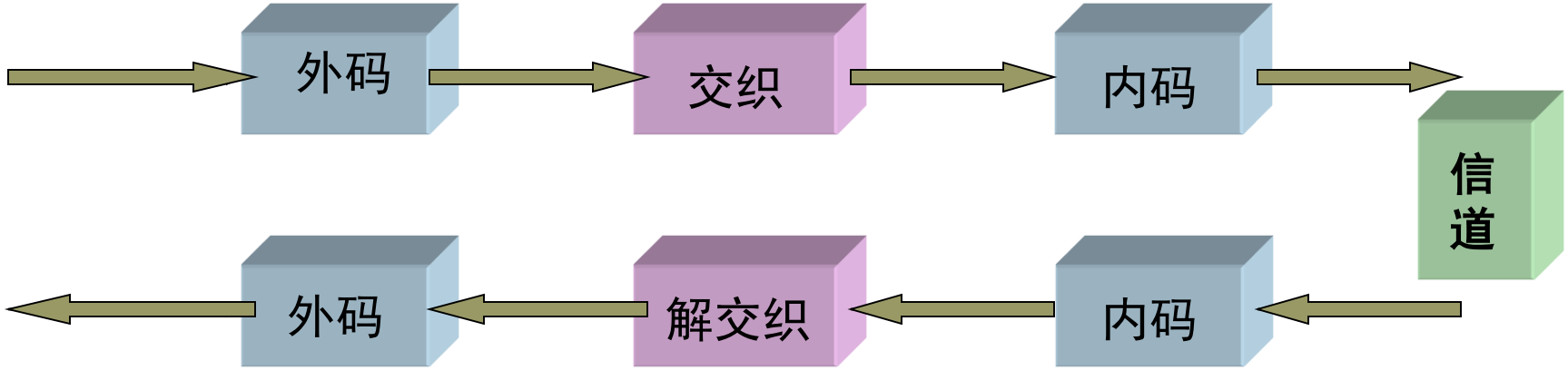
- 可将编码、信道、译码的整体看成一个广义信道。这个信道也存在错误，因此对它还可作进一步纠错编码。
- 对于有多次编码的系统，将各级编码看成一个整体，就是级联码。
- 级联码的最初想法是进一步降低残余误码率（改善渐近性能），但事实上它同样可以提高较低信噪比下的性能。
- 这是由较好构造的短码进一步构造性能更好的长码（近随机码）的一种途径。

Forney(1966)

内码与外码

- 将两个编码串联起来构成一个级联码
  - 广义信道中的编码称为内码
  - 以广义信道为信道的信道编码称为外码
  - 由于内码译码结果不可避免地会产生突发错，因此内外码之间一般都有交织器。

内外是相对信道而言的



常见的级联方式

- 卷积码为内码，RS码为外码的级联码具有良好的性能，俗称“黄金搭档”
- 为了充分利用卷积码的性能，通常采用软判决Viterbi译码。
- 卷积译码后可能存在突发错，而大部分突发错长度约等于其约束长度（或更长一些），而RS码对于突发错有很好的纠错能力，适合做外码。
- 如：（2，1，7）卷积码+（204，188，8）RS码

Reed and Solomon

Fundamentals of Wireless Communications

感谢您的观看

深圳大学
SHENZHEN UNIVERSITY